GYM MEMBERSHIP MANAGEMENT SYSTEM

A PROJECT REPORT

submitted in partial fulfilment of the requirements of the course

BCSE302L Database Systems

by

Student name	Register number
Liz Alex	24BCE2978
Malika Balaji	24BAI0114
Nehaa Prabhu	24BAI0130

Under the guidance of

Prof. Lydia Jane G



School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India

Fall Semester 2026-27 | October 2026

Source code: https://github.com/malikabalaji/gym-membership-db

Abstract

Gyms and fitness centres manage tightly linked data about members, trainers, workout plans, group classes, attendance and payments, yet many small and mid-sized gyms still keep these records in registers and disconnected spreadsheets. The result is overbooked classes, missed or untracked dues, and no reliable view of member engagement or revenue. This project designs and implements the **Gym Membership Management System**, a database solution built on MongoDB 8, a document-oriented NoSQL database management system.

The design followed the full database life cycle: requirements analysis, an Enhanced Entity Relationship model with a USER generalization, a relational schema normalized to third normal form, and a document model that embeds bounded, always-read-together data such as workout exercises and class rosters while referencing unbounded data such as attendance and payments. The implemented database has seven collections protected by `$jsonSchema` validators, nineteen secondary indexes, one read-only view, two multi-document ACID transactions and two change-stream triggers. A Python application layer of 36 CRUD functions enforces ten referential integrity rules. A reproducible dataset of 1,016 documents was loaded.

Fifteen representative queries covering selection, joins, grouping, subqueries and set operations, eight analytical aggregation pipelines and a recursive `$graphLookup` over the member referral network were executed on a live MongoDB 8.3 replica set. Indexing reduced a member's attendance history query from 200,000 documents examined in 80.5 ms to 105 documents in 2.1 ms, and a partial index served the dues report at one tenth of the size of a full index. Transaction tests confirmed complete rollback on failure, and all six automated tests passed.

The project shows that a carefully designed document database can provide the integrity guarantees usually associated with relational systems while giving a gym the flexible schema, fast dashboard reads and built-in analytics it needs.

Keywords: MongoDB, NoSQL, document database, data modelling, aggregation pipeline, indexing, ACID transactions, gym management.

Table of Contents

Abstract i

Table of Contents ii

List of Figures v

List of Tables vi

Part B: Main Report 1

Chapter 1: Introduction 1

1.1 Background of the Study 1
1.2 Motivation 1
1.3 Problem Statement 1
1.4 Aim and Objectives 2
1.5 Scope of the Project 2
1.6 Limitations 3
1.7 Methodology Overview 3

Chapter 2: Literature Review 5

2.1 Overview of the Application Domain and Its Data Needs 5
2.2 Review of Data Models 5
2.3 Review of Existing Systems and Related Work 7
2.4 Comparison of DBMS Platforms and Tools 7
2.5 Standards and Best Practices 8
2.6 Gap Identified 8
2.7 How This Project Addresses the Gap 8
2.8 Summary of the Chapter 9

Chapter 3: System Analysis and Requirements 10

3.1 Existing System and Its Drawbacks 10
3.2 Proposed System and Its Advantages 10
3.3 Feasibility Study 10
3.3.1 Technical Feasibility 10
3.3.2 Economic Feasibility 11
3.3.3 Operational Feasibility 11
3.4 Users, Roles and Access Needs 11
3.5 Requirements Specification 12
3.5.1 Functional Requirements 12
3.5.2 Data Requirements 12
3.5.3 Non-Functional Requirements 13
3.5.4 Hardware Requirements 13
3.5.5 Software Requirements 13
3.6 Business Rules and Constraints 14

Chapter 4: Database Design 15

4.1 Design Approach and Principles 15
4.2 System Architecture Diagram 16
4.3 Data Flow Diagram 17
4.4 Conceptual Design: ER / EER Diagram 18
4.4.1 Entities and Attributes 19
4.4.2 Relationships, Cardinality and Participation 19

4.4.3 Keys 20
4.4.4 Generalization and Specialization 20
4.5 Logical Design 20
4.6 Normalization 26
4.6.1 Functional Dependencies 26
4.6.3 Justified Denormalization 27
4.7 Physical Design 28
4.7.1 Data Types and Storage 28
4.7.2 Indexing Strategy 28
4.7.3 Partitioning 29
4.8 Integrity Constraints 30
Chapter 5: Implementation 31
5.1 Development Environment and Setup 31
5.2 DBMS, Tools and Libraries Used 31
5.5 Views and Indexes Created 32
5.7 Application Connectivity 33
5.8 Version Control and Project Repository 34
5.9 Challenges Faced and Solutions Adopted 34
Chapter 6: Query Design and Performance Tuning 35
6.1 Representative Queries 35
6.1.1 Selection 35
6.1.2 Joins 36
6.1.3 Grouping and Aggregation 38
6.1.4 Subqueries 39
6.1.5 Set Operations 41
6.1.6 Analytical Reports and Graph Query 42
6.2 Performance Tuning 42
6.2.1 Execution Plans with explain() 42
6.2.2 Benchmark at Scale 43
6.2.3 Other Tuning Measures 44
Chapter 7: Results and Discussion 45
7.1 Output Screenshots and Sample Reports 45
7.1.1 Sample Data 45
7.1.2 Revenue Report (A1) 45
7.1.3 Class Popularity (A2) 46
7.1.4 Retention and Collections (A3, A4) 47
7.1.5 Workload, Overview and Referral Network (A5, A6, G1, G2) 47
7.1.6 Integrity, Transactions and Triggers 48
7.2 Evaluation Against Objectives 49
7.3 Comparison with the Existing System 50
7.4 Discussion of Findings 50
7.5 Limitations Observed 51
Chapter 8: Conclusion and Future Work 52
8.1 Summary of the Work 52
8.2 Key Contributions 52
8.3 Future Enhancements 52
8.4 Concluding Remarks 53
Part C: References and Appendices 54
References 54

Appendix A: Complete DDL Scripts.....57

Appendix B: Sample Data and DML Scripts.....61

B.1 Sample Documents.....61
B.2 Sample Data Generator.....65
B.3 DML: CRUD Operations.....70

Appendix C: Stored Procedures, Functions and Triggers (Full Code).....77

C.1 Aggregation Pipelines and View (aggregations.py).....77
C.2 Transactions (transactions.py).....80
C.3 Triggers (triggers.py).....81
C.4 Index Creation and Explain (indexes.py).....83

Appendix D: Complete Query Set with Outputs.....85

D.1 Query Set for mongosh (queries.js).....85
D.2 Outputs of the Query Set on MongoDB 8.3.7 (query_output.txt).....91
D.3 Output of the Index Benchmark (benchmark_output.txt).....94

Appendix E: Full-Size ER Diagram and Relational Schema.....95

Appendix F: Application Screenshots.....98

Appendix G: User Manual.....106

G.1 Installation.....106
G.2 Running the System.....106
G.3 Everyday Tasks.....107
G.4 Backup and Restore.....107
G.5 Troubleshooting.....107

List of Figures

Figure 4.1: Layered system architecture....16
Figure 4.2: Context diagram (level 0 DFD)....17
Figure 4.3: Level 1 data flow diagram....17
Figure 4.4: Enhanced Entity Relationship diagram (Chen notation, key attributes shown)....18
Figure 4.5: Relational schema in third normal form (12 relations)....21
Figure 4.6: MongoDB document model (orange arrows are ObjectId references)....22
Figure 6.1: Median query time with and without indexes at 200,000 documents....43
Figure 7.1: Collected revenue by month and membership tier (pipeline A1)....45
Figure 7.2: Attendance rate and fill rate by class (pipeline A2)....46
Figure 7.3: Live run, transactions T1 and T2 with commit and rollback....48
Figure 7.4: Live run, change-stream triggers TR1 and TR2....49
Figure E.1: Complete EER diagram with all attributes....95
Figure E.2: Full-size relational schema (3NF)....96
Figure E.3: Full-size MongoDB document model (rotated to fit the page)....97
Figure F.1: Validator rejecting a member with an invalid tier....98
Figure F.2: Collections created and 1,016 sample documents loaded....98
Figure F.3: CRUD demonstration, create and read operations....99
Figure F.4: CRUD demonstration, update, delete and authentication....100
Figure F.5: Aggregation reports A1 to A5....101
Figure F.6: Aggregation reports A6 to A8 and the member_dashboard view....102
Figure F.7: Referral network with $graphLookup....102
Figure F.8: Indexes, explain() comparison and text search....103
Figure F.9: Transactions T1 and T2, commit and rollback....103
Figure F.10: Correlated and EXISTS subqueries Q11 and Q12....104
Figure F.11: Set operations Q13 to Q15....104
Figure F.12: Index benchmark at 200,000 documents....105
Figure F.13: Change-stream triggers TR1 and TR2....105

List of Tables

Table 1.1: Database life cycle followed in this project....3
Table 2.1: Comparison of data models for the gym domain....6
Table 2.2: Existing gym management systems....7
Table 2.3: DBMS platforms considered....7
Table 3.1: Cost estimate....11
Table 3.2: Users, roles and access rights....11
Table 3.3: Functional requirements....12
Table 3.4: Data requirements: entities, sources, volume and growth....12
Table 3.5: Non-functional requirements....13
Table 3.6: Hardware requirements....13
Table 3.7: Software requirements....13
Table 3.8: Business rules and how they are enforced....14
Table 4.1: Entities and their attributes....19
Table 4.2: Relationships, cardinality and participation....19
Table 4.3: Collection members....23
Table 4.4: Collection trainers....23
Table 4.5: Collection workout_plans....24
Table 4.6: Collection classes....24
Table 4.7: Collection class_sessions....25
Table 4.8: Collection attendance....25
Table 4.9: Collection payments....26
Table 4.10: Functional dependencies and normal form of each relation....26
Table 4.11: Denormalization decisions....28
Table 4.12: Indexes created....28
Table 4.13: Integrity constraints and their MongoDB implementation....30
Table 5.1: DBMS, tools and libraries....31
Table 5.2: Challenges and solutions....34
Table 6.1: Output of Q1....35
Table 6.2: Output of Q2....35
Table 6.3: Output of Q3....36
Table 6.4: Output of Q4....37
Table 6.5: Output of Q5....37
Table 6.6: Output of Q6....38
Table 6.7: Output of Q7....38
Table 6.8: Output of Q8....39
Table 6.9: Output of Q9....39
Table 6.10: Output of Q10 (gym-wide average 77.1%)....40
Table 6.11: Output of Q11 (first 8 of 17)....41
Table 6.12: Output of Q12....41
Table 6.13: Report pipelines and graph queries....42
Table 6.14: Execution plans for the member attendance history query (sample data)....43
Table 6.15: Benchmark results....43
Table 7.1: Sample data loaded....45
Table 7.2: Class popularity report (A2)....46
Table 7.3: Members at risk, attendance below 60% (A3)....47
Table 7.4: Outstanding dues, first six rows of the collections worklist (A4)....47
Table 7.5: Trainer workload (A5)....47

Table 7.6: Results of integrity, transaction and trigger tests.....48
Table 7.7: Evaluation against the objectives of Section 1.4.....49
Table 7.8: Existing manual system compared with the proposed system.....50
Table G.1: Commands.....106
Table G.2: Common problems.....107

PART B: MAIN REPORT

CHAPTER 1
INTRODUCTION

1.1 Background of the Study

The fitness industry in India has grown quickly over the last decade, and a typical neighbourhood gym now offers tiered memberships, personal training, scheduled group classes such as yoga, Zumba and CrossFit, and monthly billing. Each of these services produces data that is related to the others: a member is coached by a trainer, follows a workout plan written by that trainer, enrols in classes, attends class sessions and pays a monthly fee that depends on the membership tier.

A database management system (DBMS) stores such related data once, enforces rules on it and answers questions about it efficiently [3], [4]. For decades the default choice was a relational DBMS built on Codd's relational model [1]. Since around 2009, NoSQL systems such as document, key-value, column-family and graph databases have become mainstream for applications that need flexible schemas, nested data and horizontal scaling [5], [9]. This project applies database design principles from the course to a real business problem and implements the result on MongoDB, the most widely used document database [10], [22].

1.2 Motivation

The idea for the project came from observing how gyms near our campus and in our home cities actually operate. Most use a paper register at the front desk, a spreadsheet for fees and a messaging group for class announcements. Three problems recur:

- **Revenue leakage.** Nobody can see at a glance which members have unpaid bills or how many days they are overdue, so dues are chased late or not at all.
- **Poor class planning.** Popular classes are overbooked while others run half empty, because enrolment and actual attendance are never compared.
- **Weak member retention.** Members who stop attending are noticed only after they fail to renew. Attendance data that could flag them early is not analysed.

These are data problems, and they are a good fit for a course project because they need careful modelling, strict integrity rules, analytical queries and transactional updates. Commercial gym software exists (Section 2.3), but it is subscription-priced and closed, so it cannot be studied or adapted. Building the system ourselves let us apply every stage of the database life cycle.

1.3 Problem Statement

Design and implement a database system for a single gym that stores members, trainers, personalised workout plans, class schedules, class enrolment, attendance and payments;

enforces the business rules that keep this data consistent; supports role-based access for administrators, trainers and members; and produces the operational and analytical reports a gym owner needs, such as revenue trends, class popularity, outstanding dues and members at risk of leaving. The system must remain fast as attendance and payment history grow over years.

1.4 Aim and Objectives

Aim: to build a reliable, well-designed and efficient database for gym membership management on MongoDB and to show that it meets the functional and non-functional requirements of a real gym.

The objectives, numbered for traceability in Section 7.2, are:

- **O1.** Design a document-oriented schema that captures the relationships between members, trainers, workout plans, classes, sessions, attendance and payments, derived from a formal EER model.
- **O2.** Implement full Create, Read, Update and Delete (CRUD) functionality for all core entities with referential integrity enforced.
- **O3.** Enforce data quality at the database level through schema validation, unique keys and constraints.
- **O4.** Provide analytical insight (revenue trends, class popularity, attendance rates, dues and trainer workload) through MongoDB aggregation pipelines.
- **O5.** Provide authentication and role-based access for Administrator, Trainer and Member.
- **O6.** Ensure performance at scale through an evidence-based indexing strategy.
- **O7.** Guarantee consistency of multi-document business operations through ACID transactions and triggers.
- **O8.** Deliver a reproducible, version-controlled implementation with sample data, tests and documentation.

1.5 Scope of the Project

In scope:

- Single-gym member registration and membership-tier tracking (Basic, Premium, VIP) with active, expired and frozen states.
- Trainer profiles, specialisations, availability and trainer-assigned, personalised workout plans.
- Class creation, weekly schedules, dated class sessions and capacity-controlled enrolment.
- Attendance logging per member per class session (present, late, absent).
- Monthly billing, payment recording and automatic flagging of overdue bills.
- Analytical reports, a member referral network, auditing triggers and role-based login.

Out of scope:

- Real-time check-in through biometric or IoT devices.
- Integration with an online payment gateway.
- Multi-branch or franchise support.
- A production web front end (the data and application layers are complete; a web dashboard is planned as future work).

1.6 Limitations

- The sample data is synthetic. It is realistic and internally consistent but was generated by a seeded script, not collected from a working gym.
- The system serves one gym. Multi-branch data and sharding were not implemented.
- The user interface is a command-line demonstration and mongosh scripts. Role-based screens are designed but not built.
- Referential integrity is enforced in the application layer because MongoDB has no foreign keys; a client that bypasses the application could insert dangling references (mitigated by validators and triggers).
- Performance figures were measured on a single machine and will differ on other hardware.

1.7 Methodology Overview

The project followed the classic database life cycle [3], [4]. Table 1.1 summarises each phase, what was produced and where it is documented.

Table 1.1: Database life cycle followed in this project

Phase	Activities and outputs	Report section
Requirements	Interviews of the problem domain, user roles, 8 functional and 5 non-functional requirements, business rules	Chapter 3
Conceptual design	EER diagram with 9 entities, 11 relationships and a USER generalization	4.4
Logical design	Relational schema in 3NF (12 relations), then mapping to 7 MongoDB collections	4.5, 4.6
Physical design	BSON data types, 19 indexes, validators, partial index, replica set	4.7, 4.8
Implementation	Python package `gym_db`, mongosh DDL, seed data, view, transactions, triggers	Chapter 5
Testing	6 automated tests, live demonstration, 15 query outputs, explain plans, benchmark	Chapters 6, 7
Maintenance	Git repository, JSON export for backup, README and user manual	5.8, Appendix G

The design was developed iteratively across three course reviews: Review 1 delivered the requirements and data model, Review 2 the implementation with CRUD and advanced NoSQL

features, and this final report completes the query set, performance tuning, triggers and documentation.

CHAPTER 2
LITERATURE REVIEW

2.1 Overview of the Application Domain and Its Data Needs

Gym management is a membership-and-scheduling domain similar to clubs, libraries and clinics. Its data has four characteristics that drive the choice of database:

- **Strong relationships.** Almost every record refers to a member, and many also refer to a trainer or a class. The data is relational in nature even if it is not stored in a relational DBMS.
- **Hierarchical, read-together data.** A workout plan is always read with its exercises, and a class with its weekly schedule and roster. Splitting these into separate tables forces joins on every dashboard load.
- **Fast-growing history.** Attendance grows by one record per member per session, and payments by one per member per month. These collections dominate storage and query cost after a few years.
- **Analytics.** Owners need aggregates such as revenue by month and tier, attendance rate by class and overdue dues, not just individual records.

The domain also has privacy obligations: member contact details, dates of birth and payment history are personal data under India's Digital Personal Data Protection Act, 2023 [23], so passwords must be hashed and access restricted.

2.2 Review of Data Models

Table 2.1 compares the main families of data models against the needs listed above.

Table 2.1: Comparison of data models for the gym domain

Model	Core idea	Strengths for a gym	Weaknesses for a gym
Relational	Data in tables of rows; relationships through foreign keys; SQL; ACID [1], [3]	Strong integrity (foreign keys, checks), mature tooling, ad hoc SQL	Nested data (exercises, rosters) needs extra tables and joins; schema changes need migrations
Object-relational	Relational model extended with user-defined types, arrays and JSON columns (for example PostgreSQL) [24]	Keeps SQL and constraints while allowing some nesting	Nested data still feels secondary; JSON columns are weakly validated
Document (NoSQL)	Self-describing JSON or BSON documents grouped in collections; embedding and referencing [10]	Matches hierarchical data; flexible schema; rich aggregation; indexes on nested fields; multi-document ACID since MongoDB 4.0 [13]	No foreign keys; joins ($lookup) are less central than in SQL; design needs discipline
Key-value	Opaque values looked up by key (for example Redis)	Extremely fast caching and sessions	No querying by attribute; unsuitable as the main store
Column-family	Wide rows partitioned by key (for example Cassandra) [26]	Massive write throughput across data centres	Query patterns must be known in advance; no joins; weak ad hoc analytics
Graph	Nodes and edges with properties (for example Neo4j)	Natural for networks such as referrals	Poor fit for tabular billing and reporting
NewSQL	Distributed SQL with ACID at scale (for example CockroachDB) [6], [27]	Relational guarantees plus horizontal scaling	Operational complexity far beyond one gym's needs

Reasons for the model chosen. The document model was selected, implemented on MongoDB, for five reasons:

1. **Natural fit for hierarchical data.** Exercises inside a plan and schedules and rosters inside a class are stored together and read in a single operation.
2. **Integrity is still achievable.** `$jsonSchema` validators enforce required fields, types, enumerations and ranges in the server [11], unique indexes enforce keys, and multi-document transactions provide ACID guarantees where several documents change together [13].
3. **Built-in analytics.** The aggregation framework provides grouping, joins ($lookup), faceted reports ($facet), bucketing and recursive graph traversal ($graphLookup) [12], [15], covering both the reporting needs and the referral network without a second database.
4. **Flexible evolution.** New fields, such as the `referred_by` field added after Review 1, need no table migration.
5. **Learning value and industry relevance.** MongoDB is the most widely used document database [22], and the course syllabus covers NoSQL alongside the relational model.

The relational model was not discarded. It was used as the rigorous intermediate step of the design (Section 4.5) and for normalization analysis, and the final document model is a justified denormalization of it (Section 4.6.3).

2.3 Review of Existing Systems and Related Work

Commercial gym and studio platforms such as Mindbody [21a], Glofox [21b], Zen Planner [21c] and Gymdesk [21d] offer membership billing, class booking, attendance check-in and reporting as cloud subscriptions. Independent comparisons of these platforms list the same core features: membership plans, recurring billing, scheduling and booking, and attendance and retention reports [21e]. Table 2.2 summarises what such systems provide and where they fall short for this project.

Table 2.2: Existing gym management systems

System type	What it provides	Limitation relevant to this project
Paper register and spreadsheets	Zero cost, familiar to staff	No integrity, no concurrency control, no analytics, easy to lose
Commercial SaaS (Mindbody, Glofox, Zen Planner, Gymdesk)	Billing, booking, check-in, marketing, mobile apps	Monthly per-location subscription; closed data model; data held by the vendor; little control over custom analytics
Open-source or academic gym projects	Usually a single relational schema (MySQL) with CRUD screens	Rarely include analytics, indexing evidence, transactions or a documented design life cycle

On the database side, Cattell's survey of scalable SQL and NoSQL stores [5] and Stonebraker's critique [8] frame the trade-off between relational integrity and NoSQL flexibility, while Brewer's CAP analysis [7] explains why distributed databases must balance consistency and availability. Pavlo and Aslett [6] show that NewSQL systems now offer ACID at scale. Kleppmann [9] gives the practical guidance this project followed: choose the document model when data has a tree-like structure loaded all at once, and use references for many-to-many and unbounded relationships. MongoDB's own data-modelling guidance [10] makes the same recommendation as the embed-or-reference rule used in Section 4.6.3.

2.4 Comparison of DBMS Platforms and Tools

Table 2.3: DBMS platforms considered

Criterion	MySQL 8	PostgreSQL 16	MongoDB 8	Cassandra 5
Data model	Relational	Object-relational	Document (BSON)	Wide column
Schema	Fixed, DDL migrations	Fixed plus JSONB	Flexible with $jsonSchema validation	Fixed per table
Nested data	Separate tables	Arrays, JSONB	Native embedding	Collections, UDTs
Joins	Full SQL joins	Full SQL joins	$lookup, $graphLookup	None
Transactions	ACID	ACID	Multi-document ACID on replica sets	Lightweight, per partition
Analytics	SQL GROUP BY, window functions	SQL plus extensions	Aggregation pipeline, $facet, $bucket	Limited, external tools
Cost	Free (GPL)	Free (PostgreSQL licence)	Free Community Server; Atlas free tier	Free (Apache)
Fit for this project	Good, but nesting is awkward	Good	Best fit	Poor (analytics, joins)

Supporting tools were chosen for the same reasons of fit and cost: **PyMongo** as the official Python driver [29], **mongosh** and **MongoDB Compass** for interactive work, **bcrypt** for password hashing [17], **pytest** with **mongomock** for offline tests, and **Git and GitHub** for version control.

2.5 Standards and Best Practices

- **Design methodology:** Chen's ER model [2] and the EER extensions in Elmasri and Navathe [3] for conceptual design; normalization to third normal form before any denormalization [3], [4].
- **Query language standards:** ISO/IEC 9075 SQL [20] was used to state the SQL equivalent of every query in Chapter 6 so the design can be compared with a relational implementation.
- **Document modelling:** MongoDB schema design patterns [10], [25]: embed bounded one-to-few data, reference unbounded one-to-many data, and copy frequently read fields (the extended reference pattern).
- **Data quality:** server-side `$jsonSchema` validation [11] with `validationLevel: strict` and `validationAction: error`.
- **Security:** passwords stored as salted bcrypt hashes as recommended by OWASP [18] and NIST SP 800-63B [19]; hashes never returned by queries; personal data handled in line with the DPDP Act, 2023 [23].
- **Performance:** indexes designed from query patterns and verified with `explain("executionStats")` [16]; partial indexes for selective predicates [14].
- **Reliability:** replica set for transactions and failover; majority write concern for transactions [13]; regular backups with mongodump and JSON export.
- **Engineering:** version control, automated tests and reproducible, seeded sample data.

2.6 Gap Identified

- Commercial systems are closed and subscription-priced, so a gym owner cannot run custom analytics such as at-risk members or referral reach on their own data.
- Spreadsheet and register systems provide no integrity, concurrency control or history.
- Academic gym projects typically stop at a relational CRUD application. They rarely document the full design life cycle, justify the choice of data model, measure index performance or use transactions for multi-step business operations.
- Little published student work shows how to obtain relational-grade integrity (keys, constraints, referential actions, ACID) on a document database.

2.7 How This Project Addresses the Gap

- An **open, documented database design** that follows the full life cycle from EER to 3NF to a justified document model.

- **Relational-grade integrity on MongoDB**: validators as CHECK and NOT NULL constraints, unique indexes as keys, application-level referential actions, atomic conditional updates for capacity, and multi-document transactions.
- **Analytics inside the database**: eight report pipelines, a view and a recursive referral query, with no extra BI tool.
- **Measured performance**: explain plans and a 200,000-document benchmark that justify each index.
- **Reproducibility**: public repository, seeded data, tests and mongosh scripts so any evaluator can rebuild every result.

2.8 Summary of the Chapter

The gym domain combines strongly related, hierarchical data with fast-growing history and analytical needs. After comparing relational, object-relational, NoSQL and NewSQL models, the document model on MongoDB was chosen because it matches the hierarchical data, provides validation, transactions and a powerful aggregation framework, and costs nothing. Existing commercial systems are closed and academic projects rarely demonstrate integrity, performance evidence or transactions on a NoSQL platform; this project addresses that gap.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System and Its Drawbacks

In the gyms we studied, the existing system is manual: members sign a register at the front desk, fees are entered in a spreadsheet, class lists are kept on paper or in a messaging group and trainers keep workout plans in notebooks. Its drawbacks are:

- **No single source of truth.** The same member appears in several sheets with different spellings and phone numbers.
- **No integrity.** Nothing prevents a class from being overbooked, a payment from being recorded twice, or a member from being deleted while dues are outstanding.
- **No concurrency control.** Two staff editing the spreadsheet at once overwrite each other.
- **No history or audit trail.** Changes to fees or memberships leave no record of who changed what and when.
- **No analytics.** Revenue by month, attendance rates and overdue lists take hours to compile by hand.
- **Security risks.** Spreadsheets and registers expose personal data to anyone with access.

3.2 Proposed System and Its Advantages

The proposed system stores all gym data in one MongoDB database with seven collections, accessed through a Python application layer and mongosh scripts. Its advantages are:

- **Consistency:** validators, unique keys and ten referential integrity rules keep the data correct.
- **Safe concurrency:** atomic single-document updates and multi-document transactions prevent lost updates and overbooking.
- **Instant reports:** eight aggregation pipelines answer the owner's main questions in milliseconds.
- **Auditability:** a change-stream trigger records every change to members and payments.
- **Security:** bcrypt password hashes, role-based login and projections that never return hashes.
- **Scalability:** indexes keep queries fast as history grows; the replica set allows failover and later sharding.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All required technology is mature, free and well documented. MongoDB 8 Community Server runs on Windows, macOS and Linux, and was installed and run during the project on macOS (Homebrew) and Linux. Python 3.12 with PyMongo provides full access to queries,

aggregation, transactions and change streams. The team had the required skills from the course (data modelling, SQL and NoSQL) and from programming courses. The only special requirement, a replica set for transactions, is met by a single-node replica set on a laptop. The project is technically feasible.

3.3.2 Economic Feasibility

Table 3.1: Cost estimate

Item	Development cost	Running cost per year
MongoDB Community Server, Python, PyMongo, mongosh, Compass	INR 0 (open source)	INR 0
Hosting on the gym's existing PC or MongoDB Atlas free tier	INR 0	INR 0 (Atlas paid tier optional)
Development effort (3 students, course project)	No cash cost	Not applicable
Comparable commercial gym SaaS	Not applicable	Recurring monthly subscription per location

The system has no licence cost, and its only running costs are electricity and the existing PC. Compared with a commercial subscription, it pays back immediately. It is economically feasible.

3.3.3 Operational Feasibility

Front-desk staff need only three tasks: register members, record payments and mark attendance. These map to single functions in the application layer, and the planned web dashboard will expose them as forms. Reports are generated on demand, removing manual compilation. Trainers already plan workouts and take roll calls, so the system digitises an existing habit rather than introducing a new one. Members gain visibility of their plan, schedule and dues. The system is operationally feasible.

3.4 Users, Roles and Access Needs

Table 3.2: Users, roles and access rights

Role	Main tasks	Data accessed	Access level
Administrator	Register and update members and trainers; create classes and sessions; record payments; run reports	All collections, all reports	Create, read, update, delete on all data
Trainer	Create and update workout plans for assigned members; take attendance; view class rosters	Own members, own classes, own plans, attendance of own sessions	Read and update own data; create plans and attendance
Member	View profile, plan, class schedule, attendance and dues; request enrolment	Own documents only (through member_dashboard view)	Read own data; enrol in classes
System (triggers, sweep)	Flag overdue bills, write the audit log, reactivate memberships on payment	payments, members, audit_log	Automated updates

3.5 Requirements Specification

3.5.1 Functional Requirements

Table 3.3: Functional requirements

ID	Requirement	Implemented by
FR1	The administrator can create, read, update and delete members, trainers, classes and payment records	`crud.py`
FR2	A trainer can create, view and update personalised workout plans for assigned members	`create_workout_plan`, `add_exercise`, `progress_exercise`
FR3	A member can view their profile, workout plan, class schedule, attendance history and payment status	`member_dashboard` view, `member_classes`, `attendance_history`, `member_dues`
FR4	The system supports class scheduling with capacity limits and member enrolment	`create_class`, `create_session`, `enroll_member`
FR5	Attendance is recorded per member per class session	`mark_attendance`, transaction T2
FR6	Payments are tracked, and pending bills past their due date are flagged as overdue	`create_invoice`, `record_payment`, `flag_overdue`
FR7	The system produces aggregated reports on revenue, class popularity, attendance and workload	Pipelines A1 to A8
FR8	Users log in with email and password, with role-based access	`authenticate` (bcrypt)

3.5.2 Data Requirements

Table 3.4: Data requirements: entities, sources, volume and growth

Entity	Main attributes	Source	Sample volume	Expected growth
Member	Identity, contact, tier, status, dates, trainer, referrer	Registration form	60	About 20 per month
Trainer	Identity, specialisations, certification, availability	HR records	8	A few per year
Workout plan	Goal, duration, status, exercises	Trainer	76	About 1.3 per member per year
Class	Name, category, capacity, schedule, roster	Administrator	8	Rarely changes
Class session	Date, time, status	Generated from schedule	64 (4 weeks)	About 16 per week
Attendance	Session, member, status, check-in time	Trainer roll call	580	About 240 per week (largest)
Payment	Amount, mode, status, billing period, dates	Front desk	220	1 per member per month

Attendance and payments are unbounded and grow with time; the other entities are small and stable. At the observed rates the database would hold about 12,000 attendance and 1,000 to

2,000 payment documents after one year, still well within a single MongoDB server. The design nonetheless keeps attendance and payments in their own collections so that growth never inflates member or class documents.

3.5.3 Non-Functional Requirements

Table 3.5: Non-functional requirements

ID	Category	Requirement and how it is met
NFR1	Performance	Dashboard queries under 100 ms on years of data. Met with 19 indexes; member history in 2.1 ms on 200,000 rows (Section 6.2).
NFR2	Scalability and concurrency	Many staff and members can work at once without lost updates. Met with atomic conditional updates, MVCC in WiredTiger and transactions with snapshot isolation.
NFR3	Security	Passwords hashed with bcrypt; hashes never returned; role-based access; audit trail of changes to members and payments.
NFR4	Usability and availability	Simple commands for daily tasks; replica set allows automatic failover when deployed with three nodes; target availability during gym hours.
NFR5	Consistency, backup and recovery	Validators, keys and referential rules; ACID transactions; journaling for crash recovery; nightly mongodump and JSON export for backup and restore.

3.5.4 Hardware Requirements

Table 3.6: Hardware requirements

Component	Minimum	Used in development
Processor	64-bit dual core (x86-64 or ARM64)	Apple M-series (ARM64) and x86-64 Linux
Memory	4 GB RAM	8 GB and above
Storage	2 GB free (MongoDB plus data)	SSD
Network	Local network for staff PCs	Localhost

3.5.5 Software Requirements

Table 3.7: Software requirements

Software	Version	Purpose
MongoDB Community Server	8.0 or later (tested 8.3.7 and 8.3.11)	DBMS, run as replica set rs0
mongosh	2.x	Shell for DDL and query scripts
Python	3.10 or later	Application layer
PyMongo, bcrypt	4.x, 4.x	Driver and password hashing
pytest, mongomock	8.x, 4.x	Automated tests
MongoDB Compass (optional)	Latest	Visual inspection
Git	2.x	Version control
Operating system	macOS, Windows 10 or later, Linux	Any

3.6 Business Rules and Constraints

Table 3.8: Business rules and how they are enforced

No.	Rule	Enforced by
BR1	Every member has exactly one membership tier (Basic, Premium or VIP) and one status (active, expired, frozen)	Validator enum
BR2	Monthly fee is INR 1,500 for Basic, 2,500 for Premium and 4,000 for VIP; each bill equals the tier fee	`TIER_FEES`, `create_invoice`
BR3	Member and trainer emails are unique	Unique indexes `uq_email`
BR4	A member is assigned to at most one trainer, and that trainer must exist	`create_member`, `assign_trainer`
BR5	A workout plan is issued by the member's assigned trainer, and a member has only one active plan	`create_workout_plan`
BR6	Only active members can enrol; enrolment never exceeds class capacity; no duplicate enrolment	Atomic update in `enroll_member`
BR7	One attendance record per member per session	Unique index `uq_session_member`, upsert
BR8	A pending bill past its due date becomes overdue	`flag_overdue` sweep
BR9	Paid bills cannot be changed or deleted	`record_payment`, `delete_payment`
BR10	A member with unpaid bills cannot be deleted; a trainer with members or classes cannot be deleted	`delete_member`, `delete_trainer`
BR11	Renewing a membership records the payment and extends the end date together, or not at all	Transaction T1
BR12	Paying a bill reactivates an expired membership	Trigger TR2

CHAPTER 4
DATABASE DESIGN

4.1 Design Approach and Principles

The design moved from the general to the specific, following the life cycle in Table 1.1. A technology-independent EER model was drawn first (Section 4.4) and mapped to a relational schema in third normal form (Sections 4.5 and 4.6), which removes redundancy and gives a clear statement of every key and dependency. The relational schema was then transformed into a MongoDB document model by deliberate, documented denormalization (Section 4.6.3). Four principles guided every decision:

- **Model for the queries.** Data read together is stored together; the most frequent screens (member dashboard, class roster, workout plan) need one read.
- **Embed bounded, reference unbounded.** Exercises, schedules and rosters are small and bounded, so they are embedded. Attendance and payments grow without limit, so they are referenced [9], [10].
- **Integrity first.** Every rule from Section 3.6 is enforced by the database (validators, unique indexes, transactions, triggers) or by a single application function, never by convention.
- **Evidence for performance.** Each index serves a named query and is verified with explain plans (Section 6.2).

4.2 System Architecture Diagram

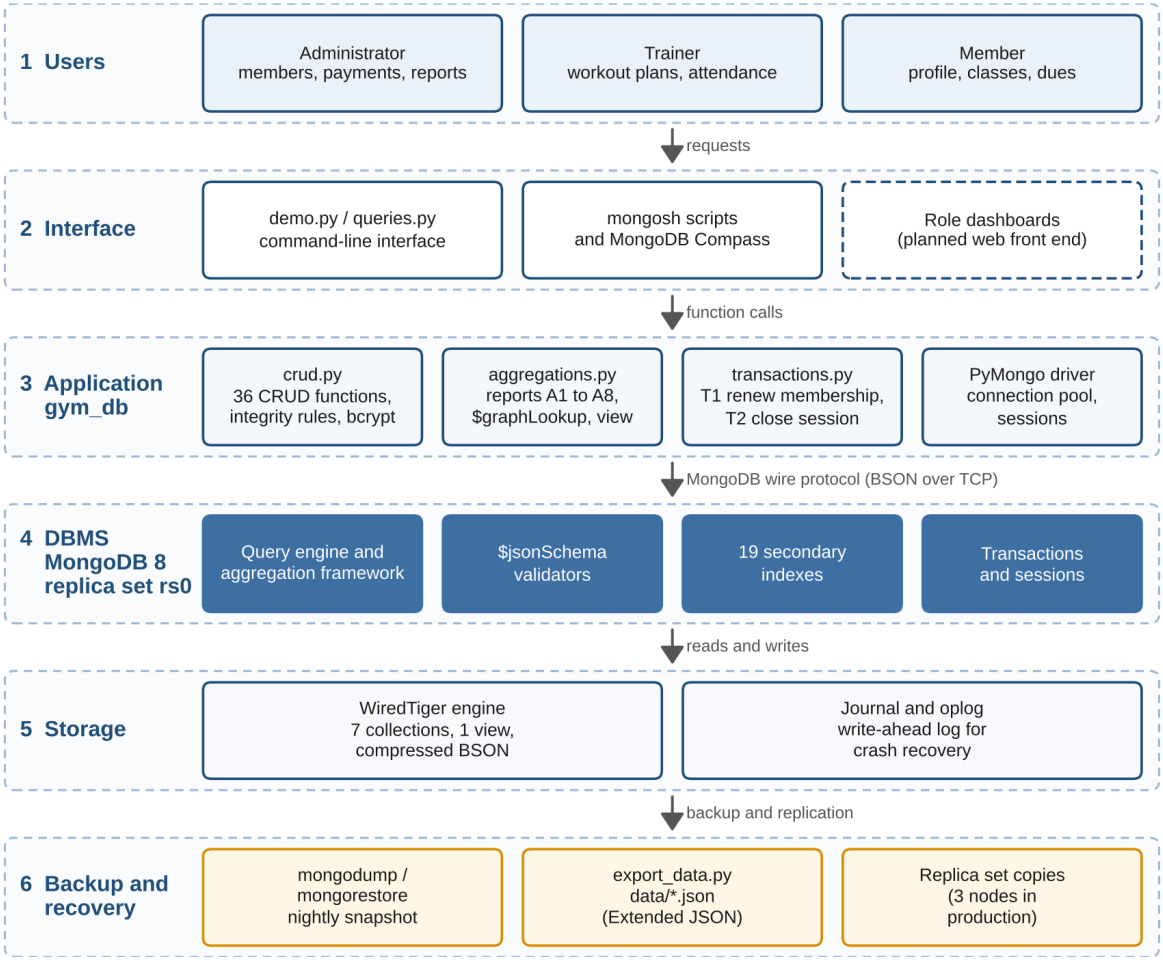


Figure 4.1: Layered system architecture

The architecture in Figure 4.1 has six layers:

- **Users:** administrators, trainers and members, each with the access rights in Table 3.2.
- **Interface:** the command-line programs `demo.py` and `queries.py`, mongosh scripts and MongoDB Compass. A role-based web dashboard is designed as the next step.
- **Application:** the `gym_db` Python package. `crud.py` holds the 36 CRUD functions and integrity rules, `aggregations.py` the reports and view, `transactions.py` the ACID operations and `triggers.py` the change-stream triggers. PyMongo manages the connection pool and client sessions.
- **DBMS:** MongoDB 8 running as replica set `rs0`. The query engine and aggregation framework execute requests; validators reject invalid documents; indexes accelerate queries; the session and transaction manager provides snapshot isolation.
- **Storage:** the WiredTiger engine stores compressed BSON documents and indexes; the journal (write-ahead log) and oplog give crash recovery and replication.
- **Backup and recovery:** mongodump snapshots, the JSON export in `data/` and replica set copies.

4.3 Data Flow Diagram

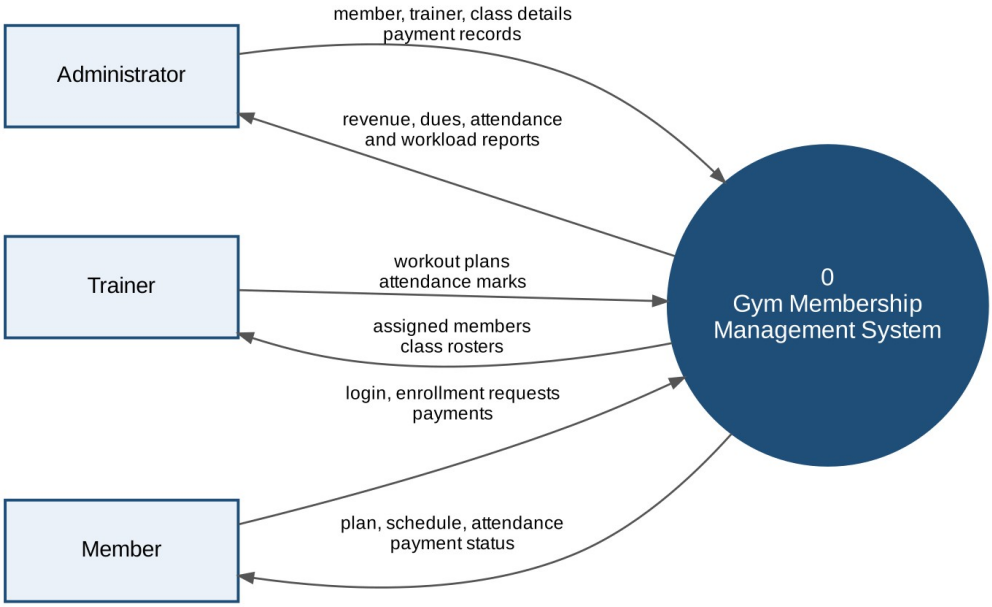


Figure 4.2: Context diagram (level 0 DFD)

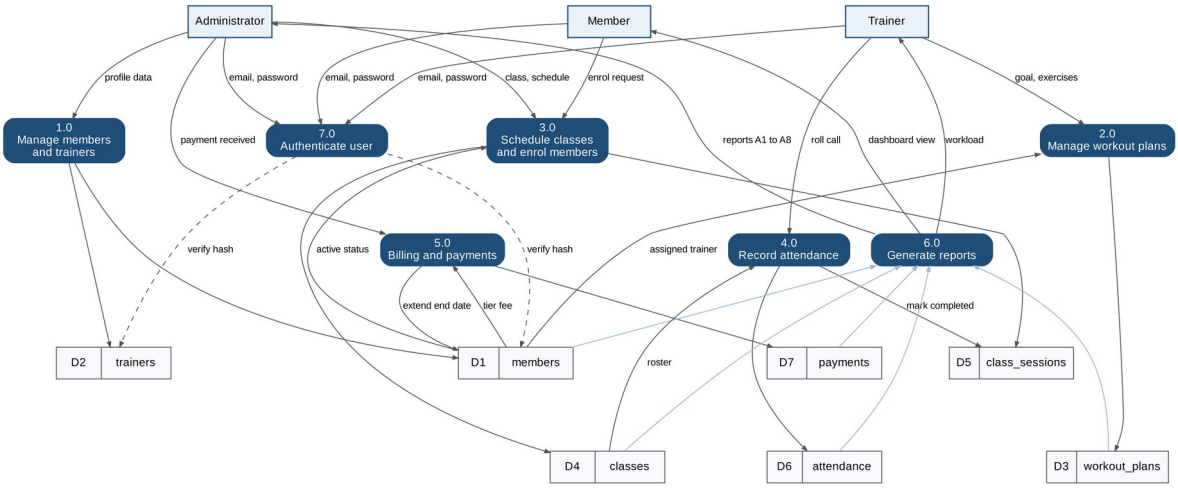


Figure 4.3: Level 1 data flow diagram

The context diagram (Figure 4.2) shows the system as a single process exchanging data with its three external entities. The level 1 diagram (Figure 4.3) decomposes it into seven processes: managing members and trainers, workout plans, classes and enrolment, attendance, billing, reporting and authentication. The seven data stores D1 to D7 are exactly the seven MongoDB collections.

4.4 Conceptual Design: ER / EER Diagram

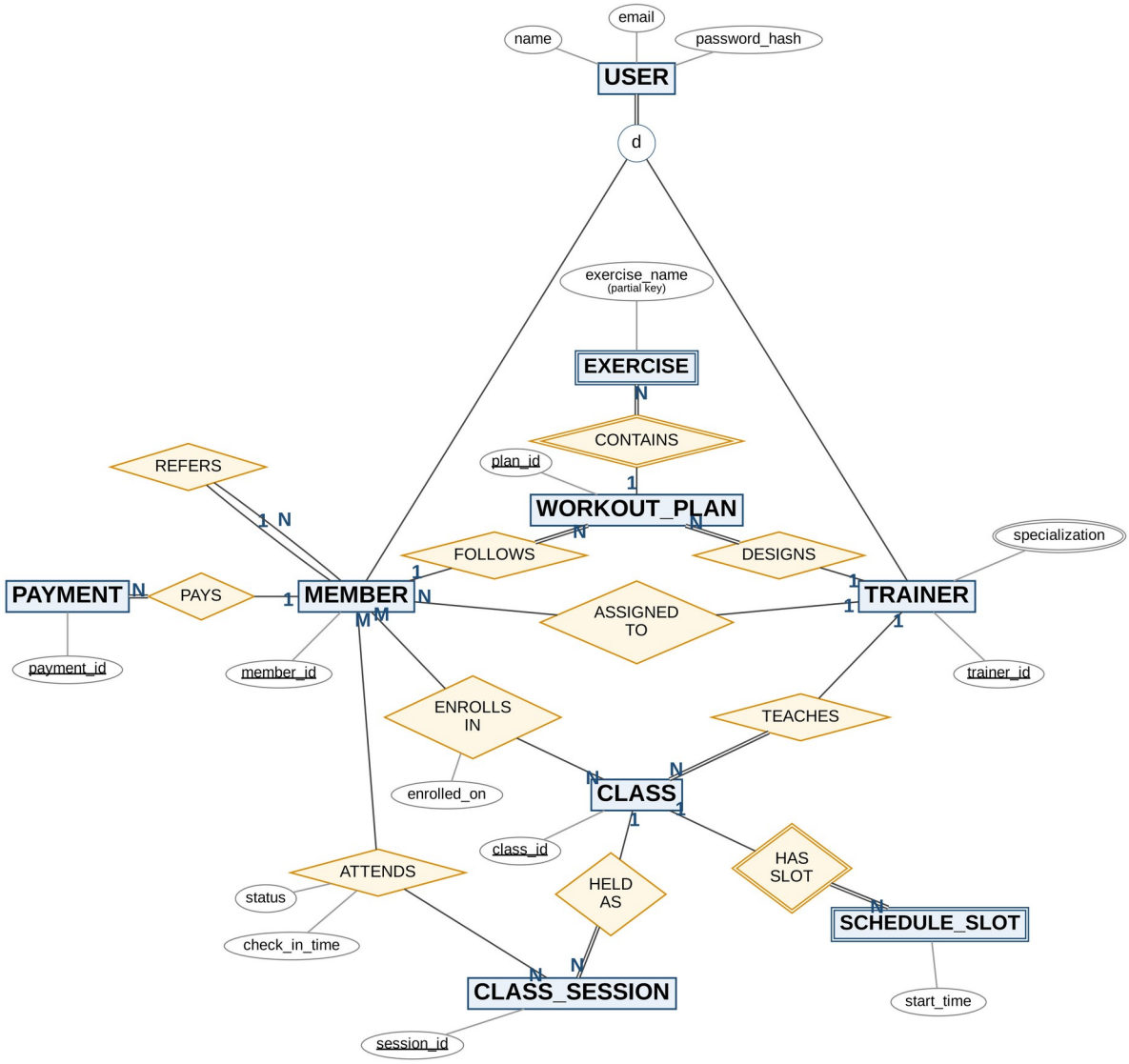


Figure 4.4: Enhanced Entity Relationship diagram (Chen notation, key attributes shown)

Figure 4.4 uses Chen notation. Rectangles are entities (double rectangles are weak entities), diamonds are relationships (double diamonds are identifying relationships), ellipses are attributes (underlined for keys, double for multivalued), double lines show total participation and the circle labelled d is a disjoint specialization. For readability the figure shows key attributes, relationship attributes and one multivalued attribute; every attribute is listed in Table 4.1 and drawn in the complete diagram, Figure E.1 in Appendix E.

4.4.1 Entities and Attributes

Table 4.1: Entities and their attributes

Entity	Type	Attributes (key underlined in the diagram)
USER	Superclass	name, email, phone, password_hash, role
MEMBER	Subclass of USER	member_id (key), dob, gender, membership (composite: type, status, start_date, end_date), address (composite), emergency_contact (composite)
TRAINER	Subclass of USER	trainer_id (key), certification, experience_years, specialization (multivalued), availability (multivalued, composite: day, start_time, end_time)
WORKOUT_PLAN	Strong	plan_id (key), goal, duration_weeks, status, created_at, updated_at
EXERCISE	Weak (owner WORKOUT_PLAN)	exercise_name (partial key), day, sets, reps, weight_kg, notes
CLASS	Strong	class_id (key), class_name, category, capacity
SCHEDULE_SLOT	Weak (owner CLASS)	day_of_week, start_time (partial key), end_time, room
CLASS_SESSION	Strong	session_id (key), date, start_time, end_time, status
PAYMENT	Strong	payment_id (key), amount, currency, payment_mode, status, billing period, payment_date, due_date

4.4.2 Relationships, Cardinality and Participation

Table 4.2: Relationships, cardinality and participation

Relationship	Entities	Cardinality	Participation
ASSIGNED_TO	TRAINER, MEMBER	1 : N	Partial on both sides (a new member may have no trainer yet)
REFERS	MEMBER, MEMBER (recursive)	1 : N	Partial (most members are not referred)
FOLLOWS	MEMBER, WORKOUT_PLAN	1 : N	Total for WORKOUT_PLAN (every plan belongs to a member)
DESIGNS	TRAINER, WORKOUT_PLAN	1 : N	Total for WORKOUT_PLAN
CONTAINS (identifying)	WORKOUT_PLAN, EXERCISE	1 : N	Total for EXERCISE
TEACHES	TRAINER, CLASS	1 : N	Total for CLASS
ENROLLS_IN (attribute enrolled_on)	MEMBER, CLASS	M : N	Partial on both sides
HAS_SLOT (identifying)	CLASS, SCHEDULE_SLOT	1 : N	Total for SCHEDULE_SLOT
HELD_AS	CLASS, CLASS_SESSION	1 : N	Total for CLASS_SESSION
ATTENDS (attributes status, check_in_time)	MEMBER, CLASS_SESSION	M : N	Partial on both sides
PAYS	MEMBER, PAYMENT	1 : N	Total for PAYMENT

4.4.3 Keys

- **Primary keys.** Every document has the system-generated `_id` (ObjectId), used as the internal primary key and as the target of every reference. Each strong entity also has a human-readable business key (`member_id` such as MEM1001, `trainer_id` TRN01, `plan_id` WP5001, `class_id` CLS01, `session_id` SES9001, `payment_id` PAY7001) that is generated sequentially, so it is unique, and is used in the user interface. For members and trainers, uniqueness is also enforced by a unique index.
- **Candidate keys.** `email` is a second candidate key of MEMBER and TRAINER.
- **Partial keys.** EXERCISE is identified by its owner plan plus `exercise_name`; SCHEDULE_SLOT by its owner class plus `day_of_week` and `start_time`.
- **Composite key of a relationship.** ATTENDS is identified by (session, member), enforced by the unique index `uq_session_member`.
- **Foreign keys.** Implemented as ObjectId references (Section 4.8).

4.4.4 Generalization and Specialization

MEMBER and TRAINER share name, email, phone, password_hash and role, so they are specializations of the superclass USER. The specialization is **disjoint** (a person is either a member or a trainer in this system, marked d) and **total** (every user is one of the two, shown by the double line). It was mapped using the "one table per subclass" option [3]: the shared attributes are repeated in both `members` and `trainers`, because members and trainers are always queried separately and have different relationships. Login searches both collections, and the `role` field records which subclass a document belongs to.

4.5 Logical Design

The EER model was first mapped to the relational schema in Figure 4.5 using the standard mapping algorithm [3]: strong entities became relations, weak entities became relations with composite keys, multivalued attributes became separate relations, the two M:N relationships became ENROLLMENT and ATTENDANCE, and 1:N relationships became foreign keys on the N side.

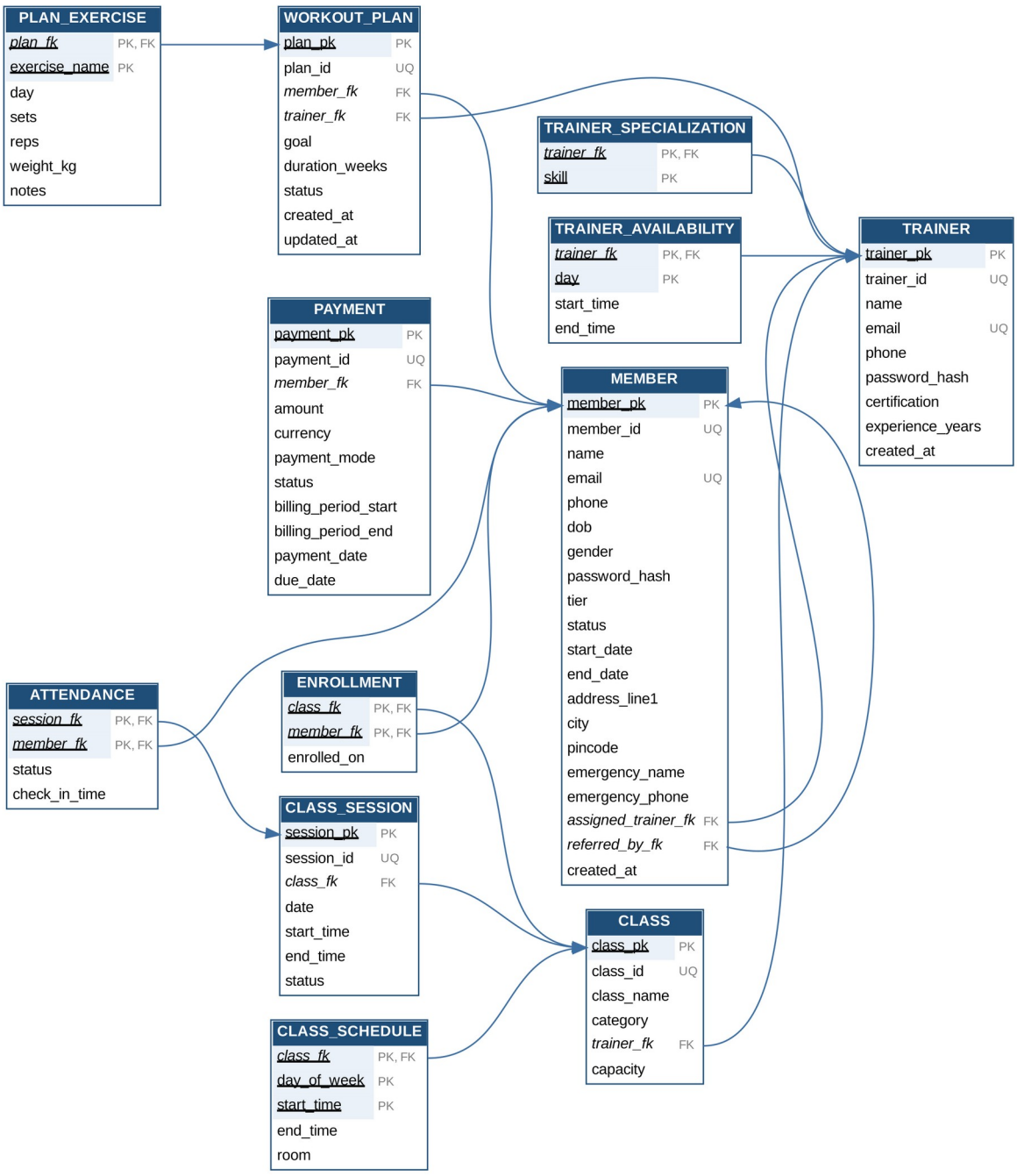


Figure 4.5: Relational schema in third normal form (12 relations)

The relational schema was then mapped to seven MongoDB collections (Figure 4.6). Five relations were embedded into their owners (TRAINER_SPECIALIZATION and TRAINER_AVAILABILITY into trainers, PLAN_EXERCISE into workout_plans, CLASS_SCHEDULE and ENROLLMENT into classes), and the remaining relations became collections.

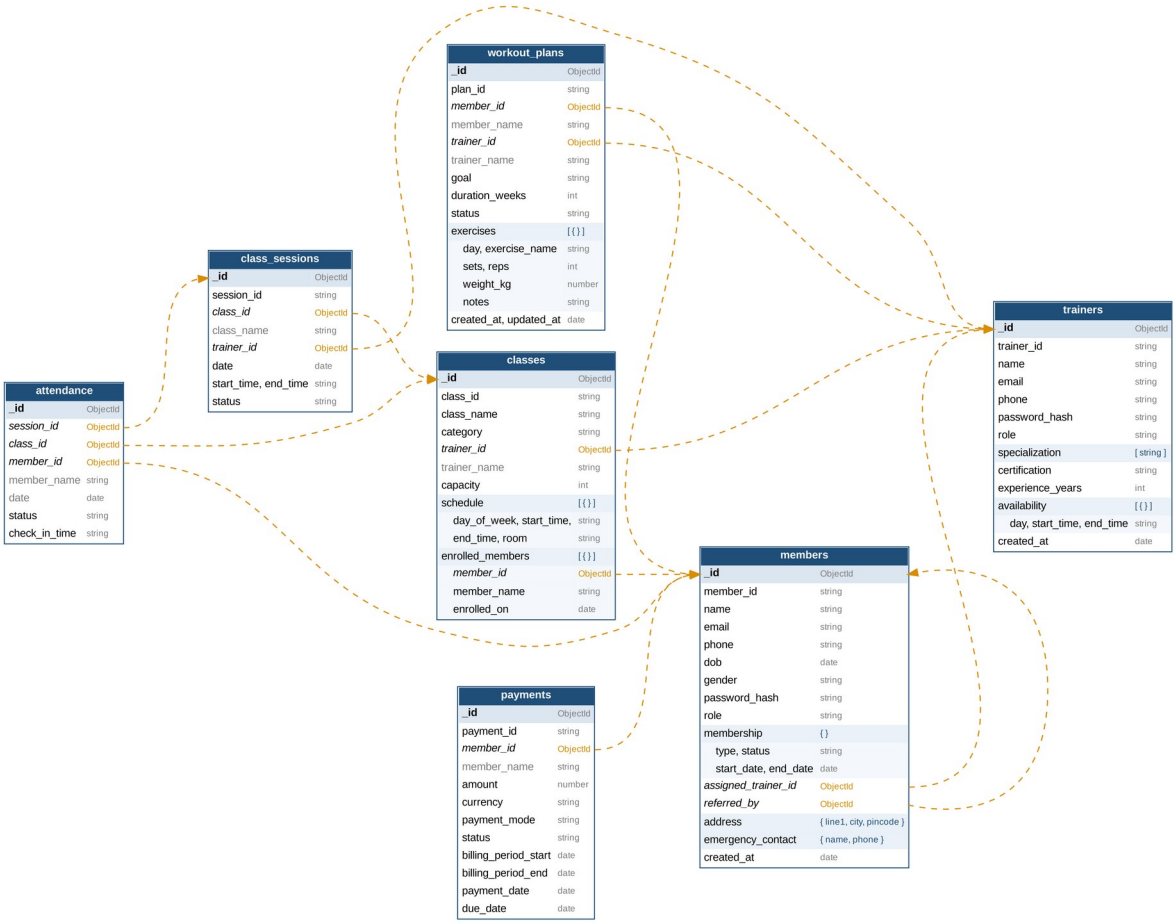


Figure 4.6: MongoDB document model (orange arrows are ObjectId references)

Tables 4.3 to 4.9 give the logical design of every collection. Size gives the maximum length or the storage size of the BSON type.

Table 4.3: Collection members

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
member_id	string	7	Required, unique, pattern ^MEM[0-9]{4}$	Business key
name	string	2 to 100	Required, min length 2	Full name
email	string	100	Required, unique, email pattern	Login name
phone	string	10	Pattern ^[0-9]{10}$	Mobile number
dob	date	8 B	BSON date	Date of birth
gender	string	6	Enum Male, Female, Other	Gender
password_hash	string	60	Required	bcrypt hash
role	string	6	Required, enum member	Subclass marker
membership.type	string	7	Required, enum Basic, Premium, VIP	Tier
membership.status	string	7	Required, enum active, expired, frozen	Status
membership.start_date, end_date	date	8 B each	Required	Membership period
assigned_trainer_id	ObjectId or null	12 B	References trainers._id	Assigned trainer
referred_by	ObjectId or null	12 B	References members._id	Referring member
address	object	about 120	line1, city, pincode	Postal address
emergency_contact	object	about 60	name, phone	Emergency contact
created_at	date	8 B	Default: time of insert	Audit field

Table 4.4: Collection trainers

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
trainer_id	string	5	Required, unique, pattern ^TRN[0-9]{2}$	Business key
name, email, phone, password_hash	string	as members	Required (except phone), email unique	Shared USER attributes
role	string	7	Required, enum trainer	Subclass marker
specialization	array of string	up to 10	Items must be strings; $addToSet prevents duplicates	Skills (multivalued)
certification	string	30	None	Main certificate
experience_years	int	4 B	Minimum 0	Years of experience
availability	array of object	up to 7	Each item needs day, start_time, end_time	Weekly availability
created_at	date	8 B	Default: time of insert	Audit field

Table 4.5: Collection workout_plans

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
plan_id	string	6	Required, unique in practice	Business key
member_id	ObjectId	12 B	Required, references members._id	Plan owner
member_name	string	100	Copy of members.name	Denormalized for display
trainer_id	ObjectId	12 B	Required, references trainers._id	Plan author
trainer_name	string	100	Copy of trainers.name	Denormalized for display
goal	string	20	Required	Muscle Gain, Weight Loss and so on
duration_weeks	int	4 B	1 to 52	Plan length
status	string	9	Required, enum active, completed	One active plan per member
exercises	array of object	1 to about 10	At least 1 item; sets and reps at least 1	Embedded exercises (day, exercise_name, sets, reps, weight_kg, notes)
created_at, updated_at	date	8 B each	Default: time of change	Audit fields

Table 4.6: Collection classes

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
class_id	string	5	Required	Business key
class_name, category	string	30	Required	Name and category
trainer_id	ObjectId	12 B	Required, references trainers._id	Instructor
trainer_name	string	100	Copy	Denormalized
capacity	int	4 B	Required, 1 to 50	Maximum enrolment
schedule	array of object	up to 14	day_of_week, start_time, end_time, room	Weekly slots
enrolled_members	array of object	up to capacity	member_id references members._id; size never exceeds capacity	Roster with member_name, enrolled_on

Table 4.7: Collection class_sessions

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
session_id	string	7	Required	Business key
class_id	ObjectId	12 B	Required, references classes._id	Parent class
class_name	string	30	Copy	Denormalized
trainer_id	ObjectId	12 B	References trainers._id	Instructor of the session
date	date	8 B	Required	Session date
start_time, end_time	string	5	HH:MM	Time slot
status	string	9	Required, enum scheduled, completed, cancelled	Lifecycle

Table 4.8: Collection attendance

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
session_id	ObjectId	12 B	Required, references class_sessions._id; unique with member_id	Session
class_id	ObjectId	12 B	Required, references classes._id	Copy for class reports
member_id	ObjectId	12 B	Required, references members._id	Member
member_name	string	100	Copy	Denormalized
date	date	8 B	Required, copy of session date	For date-sorted history
status	string	7	Required, enum present, absent, late	Outcome
check_in_time	string or null	5	HH:MM	Arrival time

Table 4.9: Collection payments

Field	Data type	Size	Constraint	Description
_id	ObjectId	12 B	Primary key	Internal identifier
payment_id	string	7	Required	Business key
member_id	ObjectId	12 B	Required, references members._id	Payer
member_name	string	100	Copy	Denormalized
amount	number	8 B	Required, minimum 0	Bill amount in INR
currency	string	3	Enum INR	Currency
payment_mode	string or null	4	Enum cash, card, UPI, null	Null until paid
status	string	7	Required, enum paid, pending, overdue	Lifecycle
billing_period_start, billing_period_end	date	8 B each	Dates	Period billed
payment_date	date or null	8 B	Set when paid	Date paid
due_date	date	8 B	Required	Payment deadline

4.6 Normalization

4.6.1 Functional Dependencies

The relational schema of Figure 4.5 was checked relation by relation. Table 4.10 lists the functional dependencies (FDs), with the left-hand side of each FD a key of its relation.

Table 4.10: Functional dependencies and normal form of each relation

Relation	Functional dependencies	Normal form
MEMBER	member_pk → all attributes; member_id → member_pk; email → member_pk	BCNF
TRAINER	trainer_pk → all attributes; trainer_id → trainer_pk; email → trainer_pk	BCNF
TRAINER_SPECIALIZATION	(trainer_fk, skill) is the whole key; no non-trivial FD	BCNF
TRAINER_AVAILABILITY	(trainer_fk, day) → start_time, end_time	BCNF
WORKOUT_PLAN	plan_pk → plan_id, member_fk, trainer_fk, goal, duration_weeks, status, created_at, updated_at	BCNF
PLAN_EXERCISE	(plan_fk, exercise_name) → day, sets, reps, weight_kg, notes	BCNF
CLASS	class_pk → class_id, class_name, category, trainer_fk, capacity	BCNF
CLASS_SCHEDULE	(class_fk, day_of_week, start_time) → end_time, room	BCNF
ENROLLMENT	(class_fk, member_fk) → enrolled_on	BCNF
CLASS_SESSION	session_pk → class_fk, date, start_time, end_time, status; (class_fk, date, start_time) → session_pk	BCNF
ATTENDANCE	(session_fk, member_fk) → status, check_in_time	BCNF
PAYMENT	payment_pk → member_fk, amount, currency, payment_mode, status, dates	3NF (see note)

Checking each normal form:

- **First normal form.** All attributes are atomic in the relational schema: multivalued attributes (specialization, availability), repeating groups (exercises, schedule slots) and the M:N rosters were moved to their own relations, and composite attributes (membership, address) were flattened.
- **Second normal form.** Every relation with a composite key was checked for partial dependencies. In PLAN_EXERCISE, sets, reps and weight depend on the whole key (the same exercise can appear in different plans with different values); in ATTENDANCE, status depends on both session and member; in ENROLLMENT, enrolled_on depends on both. No partial dependencies exist.
- **Third normal form.** No non-key attribute depends on another non-key attribute. For example, trainer name is not stored in CLASS (it depends on trainer_fk, which would be transitive), and class_id is not stored in ATTENDANCE (it depends on session_fk). Since every determinant is a candidate key, the relations are also in Boyce-Codd normal form.
- **Note on PAYMENT.** amount is determined by the member's tier at billing time (BR2). This is not a transitive dependency on the member, because the tier can change later while the historical amount must not, so amount is correctly stored on the bill.

4.6.3 Justified Denormalization

The 3NF schema avoids redundancy but would need many joins on every screen. Following MongoDB schema design guidance [10] and Kleppmann's analysis of document models [9], the following controlled denormalizations were made. Each one is justified by a frequent query and protected by a rule that keeps the copies consistent.

Table 4.11: Denormalization decisions

Denormalization	Reason	How consistency is kept
Embed PLAN_EXERCISE in workout_plans	Plan and exercises are always read together; at most about 10 exercises (bounded)	Atomic single-document updates ($push, positional $inc, $pull)
Embed TRAINER_SPECIALIZATION and TRAINER_AVAILABILITY in trainers	Small, bounded lists read with the profile	$addToSet prevents duplicate skills
Embed CLASS_SCHEDULE and ENROLLMENT in classes	Roster and timetable are shown with the class; roster bounded by capacity (at most 50)	Capacity and duplicate check inside the atomic update filter
Copy member_name, trainer_name, class_name into referencing documents	Dashboards and reports avoid a $lookup per row	`update_member` propagates a rename to all copies
Store class_id and date in attendance (derivable from session)	Class reports and date-sorted history without joining sessions	Values are copied from the session when the row is created and sessions never move
Keep attendance and payments as separate collections (not embedded)	Unbounded growth would bloat member documents beyond the 16 MB document limit and slow every member read	References plus indexes on member_id

4.7 Physical Design

4.7.1 Data Types and Storage

Documents are stored as BSON, a binary JSON encoding with typed fields. The physical types chosen are: **ObjectId** (12 bytes) for every primary and foreign key, giving compact, index-friendly, roughly time-ordered identifiers; **string** for business keys and enumerations; **int** (4 bytes) for counts such as capacity and sets; **double** for money and weights; and **date** (8-byte UTC) for every date so that range queries and date arithmetic work natively. Times of day are stored as HH:MM strings because they are compared only within a day.

MongoDB uses the WiredTiger storage engine with block compression (Snappy by default) for collections and prefix compression for indexes, a journal for crash recovery and, in a replica set, the oplog for replication. Each document is limited to 16 MB [10]; the largest document in the design, a full class with 50 roster entries, is under 10 KB. The 1,016 sample documents occupy well under 1 MB.

4.7.2 Indexing Strategy

Indexes were designed from the query patterns of the CRUD layer and reports. Table 4.12 lists all nineteen secondary indexes.

Table 4.12: Indexes created

Collection	Index name	Keys	Type	Query served
members	uq_email	email	Unique	Login, duplicate prevention

Collection	Index name	Keys	Type	Query served
members	uq_member_id	member_id	Unique	Lookup by business key
members	ix_trainer	assigned_trainer_id	Single field	Trainer's members, workload report
members	ix_tier_status	membership.type, membership.status	Compound	Member list filters
members	ix_referred_by	referred_by	Single field	$graphLookup connectToField
trainers	uq_email	email	Unique	Login
trainers	uq_trainer_id	trainer_id	Unique	Lookup by business key
workout_plans	ix_member_status	member_id, status	Compound	Active plan of a member
workout_plans	ix_trainer	trainer_id	Single field	Trainer's plans
classes	ix_trainer	trainer_id	Single field	Workload, delete checks
classes	ix_enrolled_member	enrolled_members.member_id	Multikey	Classes of a member, rename propagation
classes	tx_class_search	class_name, category	Text	Class search
class_sessions	ix_class_date	class_id, date	Compound	Timetable of a class
attendance	ix_member_date	member_id, date (descending)	Compound	Member attendance history
attendance	uq_session_member	session_id, member_id	Unique compound	One mark per member per session
attendance	ix_class_status	class_id, status	Compound	Class popularity, absentees
payments	ix_member_status	member_id, status	Compound	Dues of a member
payments	ix_due_date	due_date	Single field	Overdue sweep
payments	px_unpaid_dues	status, due_date (only unpaid bills)	Partial	Dues reports

Compound indexes follow the equality, sort, range rule: the equality field comes first (member_id) and the sort field second (date descending), so the member history query is answered in index order with no in-memory sort. The partial index stores only pending and overdue bills, which keeps it small as paid history grows.

4.7.3 Partitioning

Partitioning (sharding) was not needed: one gym produces only thousands of documents per year, far within one server's capacity. The design is shard-ready, however. If the system were extended to many branches, attendance and payments, the two large collections, could be sharded on a hashed `member_id` (or on `branch_id, member_id`), which spreads writes evenly and keeps each member's history on one shard. Replication is used instead of partitioning for availability: the database runs as replica set `rs0`.

4.8 Integrity Constraints

Table 4.13: Integrity constraints and their MongoDB implementation

Constraint type	Relational meaning	Implementation in this project
Primary key	Unique, not null identifier	`_id` ObjectId on every document (automatic unique index)
Unique	No duplicate values	Unique indexes on email, member_id, trainer_id and (session_id, member_id)
Not null	Value must be present	`required` arrays in each $jsonSchema validator
Check	Value must satisfy a condition	Validator `enum` (tiers, statuses, modes), `pattern` (IDs, phone, email), `minimum` and `maximum` (capacity 1 to 50, duration 1 to 52, amount at least 0), `minItems` (at least 1 exercise)
Default	Value supplied when missing	Set by the application: status active, currency INR, created_at now, payment_mode null
Foreign key	Value must match a parent key	ObjectId references checked before every insert and update (`_require`, `create_*` functions)
Referential action: RESTRICT	Block deleting a parent with children	Members with unpaid bills, trainers with members or classes
Referential action: CASCADE	Delete children with the parent	Deleting a member deletes plans and roster entries; deleting a class deletes future sessions
Referential action: SET NULL	Clear the reference	Deleting a member sets `referred_by` to null in members they referred
Referential action: CASCADE UPDATE	Propagate a changed value	A member rename updates every denormalized `member_name`
Assertion across documents	Rule spanning documents	Capacity check inside the atomic enrolment update; transactions T1 and T2; triggers TR1 and TR2

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment and Setup

The system was developed on macOS with MongoDB installed through Homebrew and on Linux with MongoDB 8.3.7, in both cases as a single-node replica set so that transactions and change streams are available. The setup is:

```
# 1. Install MongoDB and make it a replica set (macOS shown)
brew tap mongodb/brew && brew trust mongodb/brew && brew install mongodb-community
#    add to /opt/homebrew/etc/mongod.conf:   replication:
#                                              replSetName: rs0
brew services restart mongodb-community
mongosh --eval "rs.initiate()"

# 2. Get the code and install Python dependencies
git clone https://github.com/malikabalaji/gym-membership-db.git
cd gym-membership-db
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3. Build, seed and demonstrate
python demo.py              # creates gym_db, loads data, runs every section
python queries.py           # the 15 representative queries of Chapter 6
python benchmark.py         # index benchmark of Section 6.2
python -m pytest -q         # automated tests
```

5.2 DBMS, Tools and Libraries Used

Table 5.1: DBMS, tools and libraries

Component	Version	Role in the project
MongoDB Community Server	8.3.7 and 8.3.11	Database server, replica set rs0, WiredTiger storage
mongosh	2.14	Running `ddl.js`, `queries.js` and `demo_queries.js`
Python	3.12	Application layer
PyMongo	4.x	Official driver: CRUD, aggregation, sessions, transactions, change streams
bcrypt	4.x	Salted, adaptive password hashing
mongomock	4.x	In-memory MongoDB for offline tests and `--mock` runs
pytest	8.x	Automated test runner
matplotlib, Graphviz	Latest	Charts and diagrams in this report
Git and GitHub	Latest	Version control and public repository

The repository is organised as a Python package plus scripts:

```
gym-membership-db/
  gym_db/config.py         connection, reference date, tier fees
  gym_db/schema.py         7 collections with $jsonSchema validators (DDL)
  gym_db/indexes.py        19 indexes and the explain() comparison
  gym_db/seed.py           reproducible sample data (1,016 documents)
  gym_db/crud.py           36 CRUD functions and integrity rules (DML)
```

```
gym_db/aggregations.py   reports A1 to A8, $graphLookup G1 and G2, view V1
gym_db/transactions.py   ACID transactions T1 and T2
gym_db/triggers.py       change-stream triggers TR1 and TR2
demo.py                  end-to-end demonstration (7 sections)
queries.py               query set Q1 to Q15 with SQL equivalents
benchmark.py             index benchmark at 200,000 documents
mongosh/ddl.js           complete DDL script for mongosh
mongosh/queries.js       query set for mongosh
data/*.json              exported sample data (Extended JSON)
tests/test_gym_db.py     6 automated tests
docs/                    review reports, final report, screenshots
```

5.5 Views and Indexes Created

View. One read-only view, `member_dashboard`, is defined on `members`. It joins the assigned trainer's name and the member's payments, totals unpaid bills and exposes only safe fields, so the member screen (FR3) can query a single object and the password hash can never leak:

```
MEMBER_DASHBOARD_PIPELINE = [
    {"$lookup": {"from": "trainers", "localField": "assigned_trainer_id", "foreignField": "_id",
        "as": "t"}},
    {"$lookup": {"from": "payments", "localField": "_id", "foreignField": "member_id",
        "as": "p"}},
    {"$project": {"_id": 0, "member_id": 1, "name": 1, "tier": "$membership.type",
                  "status": "$membership.status", "trainer": {"$first": "$t.name"},
                  "unpaid_amount": {"$sum": {"$map": {
                      "input": {"$filter": {"input": "$p", "as": "x",
                          "cond": {"$ne": ["$$x.status", "paid"]}}},
                      "as": "u", "in": "$$u.amount"}}}},
]


db.member_dashboard.findOne({ member_id: "MEM1005" })
{ member_id: 'MEM1005', name: 'Rahul Rao', tier: 'Premium', status: 'expired',
  trainer: 'Vivek Nair', unpaid_amount: 0 }
```

Indexes. The nineteen indexes of Table 4.12 are created by `gym_db/indexes.py` (Python) and `mongosh/ddl.js` (shell). Representative definitions:

```
db.members.createIndex({ email: 1 }, { name: "uq_email", unique: true });
db.classes.createIndex({ "enrolled_members.member_id": 1 },
    { name: "ix_enrolled_member" });   // multikey
db.classes.createIndex({ class_name: "text", category: "text" }, { name: "tx_class_search" });
db.attendance.createIndex({ member_id: 1, date: -1 }, { name: "ix_member_date" });
db.attendance.createIndex({ session_id: 1, member_id: 1 }, { name: "uq_session_member",
    unique: true });
db.payments.createIndex({ status: 1, due_date: 1 },
  { name: "px_unpaid_dues", partialFilterExpression: { status: { $in: ["pending",
      "overdue"] } } });
```

Validators. Each collection is created with a `$jsonSchema` validator (complete DDL in Appendix A). Inserting a member with the invalid tier "Gold" is rejected by the server with error code 121, Document failed validation (Figure F.1).

Transactions and triggers. Two multi-document transactions (T1 renew membership, T2 close a class session) and two change-stream triggers (TR1 audit trail, TR2 reactivate a membership when a bill is paid) complete the server-side logic. Their full code is in Appendix C and their results in Section 7.1.

5.7 Application Connectivity

The application connects to MongoDB through **PyMongo**, the official Python driver, which plays the role that JDBC or ODBC plays for a relational database [29]. The connection string is read from the `MONGO_URI` environment variable, so the same code runs against a laptop, a server or MongoDB Atlas:

```
"""Connection and shared settings for the Gym Membership Management System."""
import os
from datetime import datetime

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/?directConnection=true")
DB_NAME = os.getenv("GYM_DB_NAME", "gym_db")

# Fixed "today" so the seeded data and every report are reproducible during the demo.
REFERENCE_DATE = datetime(2026, 9, 24)

# Monthly membership fee per tier (INR)
TIER_FEES = {"Basic": 1500, "Premium": 2500, "VIP": 4000}

COLLECTIONS = ["members", "trainers", "workout_plans", "classes",
               "class_sessions", "attendance", "payments"]

_client = None
IS_MOCK = False


def get_client(mock: bool = False):
    """Return a MongoClient. mock=True uses mongomock (in-memory) for offline testing."""
    global _client, IS_MOCK
    if _client is None:
        if mock:
            import mongomock
            from . import mockpatch  # noqa: F401  ($round support for offline tests)
            _client = mongomock.MongoClient()
            IS_MOCK = True
        else:
            from pymongo import MongoClient
            _client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
            _client.admin.command("ping")
    return _client


def get_db(mock: bool = False):
    return get_client(mock)[DB_NAME]
```

Key aspects of the connectivity layer:

- **Connection pooling.** One `MongoClient` is created per process and reused; the driver maintains a pool of connections.
- **Sessions and transactions.** `client.start_session()` and `with_transaction()` provide ACID transactions with snapshot read concern, majority write concern and automatic retry on transient errors.
- **Change streams.** `db.watch()` opens a change stream used by the triggers.
- **No ORM.** Documents map directly to Python dictionaries, so an object-relational mapper is unnecessary; the `gym_db` functions act as a thin data-access layer.

- **Offline mode.** `get_db(mock=True)` swaps in mongomock so tests and the demonstration run without a server.

The planned web dashboard will call the same `gym_db` functions from a Python web framework (for example Flask), so no database code needs to change.

5.8 Version Control and Project Repository

All code, data and documents are version-controlled with Git and published at **https://github.com/malikabalaji/gym-membership-db**. The repository contains the source code, the seeded sample data in Extended JSON, mongosh scripts, automated tests, the outputs of the live runs (`demo_output.txt`, `query_output.txt`, `benchmark_output.txt`), the Review 1 and Review 2 reports, this final report and the terminal screenshots. The README explains installation and every command. Each review stage was committed separately, so the history records how the design evolved from Review 1 to the final submission.

5.9 Challenges Faced and Solutions Adopted

Table 5.2: Challenges and solutions

Challenge	Solution adopted
MongoDB has no foreign keys, so references can dangle	Ten referential rules in the application layer (Table 4.13): existence checks, RESTRICT, CASCADE, SET NULL and cascade update of copied names
Preventing overbooking when two members book the last place at the same time	Capacity and duplicate checks placed inside the update filter with `$expr` and `$size`, so the check and the write are one atomic operation
Transactions failed on a standalone server ("Transaction numbers are only allowed on a replica set member")	Ran MongoDB as a single-node replica set (`replSetName: rs0`, `rs.initiate()`)
Homebrew refused the MongoDB formula from an untrusted tap	Trusted the official tap with `brew trust mongodb/brew` before installing
Keeping denormalized names consistent after a rename	`update_member` propagates the new name to plans, attendance, payments and class rosters, including inside arrays with the positional operator
Testing without a running server	mongomock for offline tests, with a small shim adding the `$round` operator; features mongomock cannot emulate (validators, transactions, explain, change streams) were verified on a live server
Reports differed in row order between runs when sort keys tied	Added tie-breaker sort keys (member_id, class_name) so every report is deterministic
Making results reproducible for evaluators	Fixed random seed and fixed reference date (24 September 2026), and published all outputs
Index timings were all 0 ms on only 580 attendance rows	Built `benchmark.py` with 200,000 synthetic documents to measure real differences

CHAPTER 6
QUERY DESIGN AND PERFORMANCE TUNING

6.1 Representative Queries

The fifteen queries below cover every category in the course syllabus: selection, joins (inner, three-way and left outer), grouping and aggregation (including HAVING), subqueries (scalar, correlated and EXISTS) and set operations (UNION, INTERSECT and EXCEPT). Each is shown in MongoDB syntax with its SQL equivalent on the relational schema of Figure 4.5 and its actual output from the live MongoDB 8.3.7 replica set. All definitions are in `queries.py`, the mongosh version is `mongosh/queries.js` and the complete outputs are in Appendix D.

6.1.1 Selection

Q1. Active VIP members living in Chennai (equality on nested fields, projection, sort).

```
db.members.find(
  { "membership.type": "VIP", "membership.status": "active", "address.city": "Chennai" },
  { _id: 0, member_id: 1, name: 1, "membership.end_date": 1 }
).sort({ member_id: 1 })

-- SQL: SELECT member_id, name, end_date FROM members
--      WHERE tier = 'VIP' AND status = 'active' AND city = 'Chennai' ORDER BY member_id;
```

Table 6.1: Output of Q1

member_id	name	membership.end_date
MEM1014	Divya Das	2026-12-31
MEM1043	Gautham Nair	2027-05-31
MEM1046	Nandini Menon	2027-02-28

Q2. Trainers who teach Yoga or CrossFit with at least 6 years of experience (membership test on an array field, range condition).

```
db.trainers.find(
  { specialization: { $in: ["Yoga", "CrossFit"] }, experience_years: { $gte: 6 } },
  { _id: 0, trainer_id: 1, name: 1, specialization: 1, experience_years: 1 }
).sort({ experience_years: -1 })

-- SQL: SELECT DISTINCT t.trainer_id, t.name, t.experience_years
--      FROM trainers t JOIN trainer_specializations s ON s.trainer_id = t.trainer_id
--      WHERE s.skill IN ('Yoga', 'CrossFit') AND t.experience_years >= 6 ORDER BY 3 DESC;
```

Table 6.2: Output of Q2

trainer_id	name	specialization	experience_years
TRN03	Deepa Krishnan	Yoga, Pilates	9
TRN08	Vivek Nair	CrossFit, Olympic Lifting	8
TRN07	Kavitha Reddy	Yoga, Mobility	7
TRN01	Priya Menon	Strength Training, CrossFit	6

Q3. Unpaid bills already past their due date, oldest first (IN list, date range, multi-key sort, LIMIT). 17 bills match; the first 8 are shown.

```
db.payments.find(
  { status: { $in: ["pending", "overdue"] }, due_date: { $lt: ISODate("2026-09-24") } },
  { _id: 0, payment_id: 1, member_name: 1, amount: 1, status: 1, due_date: 1 }
).sort({ due_date: 1, payment_id: 1 }).limit(8)

-- SQL: SELECT payment_id, member_name, amount, status, due_date FROM payments
--      WHERE status IN ('pending','overdue') AND due_date < DATE '2026-09-24'
--      ORDER BY due_date, payment_id LIMIT 8;
```

Table 6.3: Output of Q3

payment_id	member_name	amount	status	due_date
PAY7054	Divya Menon	1500	overdue	2026-08-05
PAY7023	Isha Joshi	1500	pending	2026-09-05
PAY7027	Gautham Bhat	2500	pending	2026-09-05
PAY7047	Divya Das	4000	pending	2026-09-05
PAY7061	Aishwarya Das	4000	pending	2026-09-05
PAY7073	Karthik Sharma	2500	pending	2026-09-05
PAY7085	Ananya Das	2500	pending	2026-09-05
PAY7089	Vikram Kumar	1500	pending	2026-09-05

6.1.2 Joins

Q4. Inner join: active VIP members with their assigned trainer. `$lookup` followed by `$unwind` behaves as an inner join because `$unwind` drops members without a match.

```
db.members.aggregate([
  { $match: { "membership.type": "VIP", "membership.status": "active" } },
  { $lookup: { from: "trainers", localField: "assigned_trainer_id", foreignField: "_id",
      as: "t" } },
  { $unwind: "$t" },
  { $project: { _id: 0, member_id: 1, name: 1, trainer: "$t.name",
      certification: "$t.certification" } },
  { $sort: { member_id: 1 } }, { $limit: 8 }
])

-- SQL: SELECT m.member_id, m.name, t.name AS trainer, t.certification
--      FROM members m INNER JOIN trainers t ON t.trainer_pk = m.assigned_trainer_id
--      WHERE m.tier = 'VIP' AND m.status = 'active' ORDER BY m.member_id LIMIT 8;
```

Table 6.4: Output of Q4

member_id	name	trainer	certification
MEM1001	Harish Kumar	Priya Menon	ACE-CPT
MEM1012	Isha Das	Sanjay Pillai	ISSA-CPT
MEM1014	Divya Das	Rohit Sharma	NASM-CPT
MEM1018	Aishwarya Das	Kavitha Reddy	RYT-200
MEM1028	Lakshmi Krishnan	Rohit Sharma	NASM-CPT
MEM1033	Rahul Sharma	Vivek Nair	CF-L2
MEM1035	Dev Varma	Vivek Nair	CF-L2
MEM1038	Nandini Das	Farhan Ali	ACSM-EP

Q5. Three-way join: attendance history of MEM1001 with class name and session time (members, attendance, class_sessions and classes).

```
db.members.aggregate([
  { $match: { member_id: "MEM1001" } },
  { $lookup: { from: "attendance", localField: "_id", foreignField: "member_id", as: "a" } },
      { $unwind: "$a" },
  { $lookup: { from: "class_sessions", localField: "a.session_id", foreignField: "_id",
      as: "s" } }, { $unwind: "$s" },
  { $lookup: { from: "classes", localField: "a.class_id", foreignField: "_id", as: "c" } },
      { $unwind: "$c" },
  { $project: { _id: 0, date: "$s.date", class_name: "$c.class_name",
      start_time: "$s.start_time",
                status: "$a.status", check_in_time: "$a.check_in_time" } },
  { $sort: { date: -1 } }
])

-- SQL: SELECT s.date, c.class_name, s.start_time, a.status, a.check_in_time
--      FROM members m JOIN attendance a ON a.member_id = m.member_pk
--      JOIN class_sessions s ON s.session_pk = a.session_id JOIN classes c ON c.class_pk =
    a.class_id
--      WHERE m.member_id = 'MEM1001' ORDER BY s.date DESC;
```

Table 6.5: Output of Q5

date	class_name	start_time	status	check_in_time
2026-09-23	Evening Yoga	19:00	present	18:52
2026-09-19	Evening Yoga	17:00	present	16:51
2026-09-16	Evening Yoga	19:00	present	18:50
2026-09-12	Evening Yoga	17:00	present	16:57
2026-09-09	Evening Yoga	19:00	present	19:00

Q6. Left outer join: every trainer with the number of classes taught, including trainers who teach none. `$lookup` without `$unwind` keeps unmatched trainers with an empty array, exactly like a LEFT OUTER JOIN.

```
db.trainers.aggregate([
  { $lookup: { from: "classes", localField: "_id", foreignField: "trainer_id", as: "c" } },
  { $project: { _id: 0, trainer_id: 1, name: 1, classes: { $size: "$c" },
      class_names: "$c.class_name" } },
  { $sort: { classes: -1, trainer_id: 1 } }
```

```
])

-- SQL: SELECT t.trainer_id, t.name, COUNT(c.class_id) AS classes
--      FROM trainers t LEFT OUTER JOIN classes c ON c.trainer_id = t.trainer_pk
--      GROUP BY t.trainer_id, t.name ORDER BY classes DESC, t.trainer_id;
```

Table 6.6: Output of Q6

trainer_id	name	classes	class_names
TRN03	Deepa Krishnan	2	Morning Yoga, Pilates Core
TRN02	Rohit Sharma	1	HIIT Express
TRN04	Sanjay Pillai	1	Strength 101
TRN05	Meenakshi Iyer	1	Power Zumba
TRN06	Farhan Ali	1	Spin Cycle
TRN07	Kavitha Reddy	1	Evening Yoga
TRN08	Vivek Nair	1	CrossFit Burn
TRN01	Priya Menon	0	(none)

6.1.3 Grouping and Aggregation

Q7. Money collected by payment mode (COUNT, SUM, AVG).

```
db.payments.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$payment_mode", payments: { $sum: 1 }, total: { $sum: "$amount" },
      average: { $avg: "$amount" } } },
  { $project: { _id: 0, payment_mode: "$_id", payments: "$payments", total: "$total",
      average: { $round: ["$average", 0] } } },
  { $sort: { total: -1 } }
])

-- SQL: SELECT payment_mode, COUNT(*), SUM(amount), AVG(amount) FROM payments
--      WHERE status = 'paid' GROUP BY payment_mode ORDER BY SUM(amount) DESC;
```

Table 6.7: Output of Q7

payment_mode	payments	total (INR)	average (INR)
UPI	101	237,500	2,351
card	37	97,000	2,622
cash	15	37,500	2,500

UPI accounts for 66% of paid bills, and the three modes total INR 372,000, which matches the revenue report A1 (Figure 7.1).

Q8. Grouping with HAVING: cities with more than 10 members, split by tier (conditional counting with `$cond`, filter on the group with `$match` after `$group`).

```
db.members.aggregate([
  { $group: { _id: "$address.city", members: { $sum: 1 },
      basic:   { $sum: { $cond: [{ $eq: ["$membership.type", "Basic"] }, 1, 0] } },
      premium: { $sum: { $cond: [{ $eq: ["$membership.type", "Premium"] }, 1, 0] } },
      vip:     { $sum: { $cond: [{ $eq: ["$membership.type", "VIP"] }, 1, 0] } } } },
  { $match: { members: { $gt: 10 } } },                                  // HAVING
  { $project: { _id: 0, city: "$_id", members: "$members", basic: "$basic", premium: "$premium",
      vip: "$vip" } },
```

```
  { $sort: { members: -1 } }
])

-- SQL: SELECT city, COUNT(*), SUM(CASE WHEN tier='VIP' THEN 1 ELSE 0 END) AS vip, ...
--      FROM members GROUP BY city HAVING COUNT(*) > 10 ORDER BY 2 DESC;
```

Table 6.8: Output of Q8

city	members	basic	premium	vip
Chennai	16	6	6	4
Bengaluru	13	6	5	2
Hosur	12	4	2	6

Q9. Attendance outcome by class category (join, then grouping with conditional sums).

```
db.attendance.aggregate([
  { $lookup: { from: "classes", localField: "class_id", foreignField: "_id", as: "c" } },
      { $unwind: "$c" },
  { $group: { _id: "$c.category", marked: { $sum: 1 },
      present: { $sum: { $cond: [{ $eq: ["$status", "present"] }, 1, 0] } },
      late:    { $sum: { $cond: [{ $eq: ["$status", "late"] }, 1, 0] } },
      absent:  { $sum: { $cond: [{ $eq: ["$status", "absent"] }, 1, 0] } } } },
  { $project: { _id: 0, category: "$_id", marked: "$marked", present: "$present", late: "$late",
      absent: "$absent" } },
  { $sort: { marked: -1 } }
])

-- SQL: SELECT c.category, COUNT(*), SUM(CASE WHEN a.status='present' THEN 1 ELSE 0 END), ...
--      FROM attendance a JOIN classes c ON c.class_pk = a.class_id GROUP BY c.category ORDER
    BY 2 DESC;
```

Table 6.9: Output of Q9

category	marked	present	late	absent
Yoga	160	103	8	49
Zumba	95	69	8	18
CrossFit	75	51	10	14
Strength	70	46	7	17
HIIT	68	46	8	14
Pilates	60	46	5	9
Cardio	52	36	4	12

Yoga has the most bookings but also the most absences (31% absent), a finding examined further in Section 7.4.

6.1.4 Subqueries

Q10. Scalar subquery: members whose attendance rate is below the gym-wide average. `$facet` computes the overall average (the inner query) and the per-member rates in one pass, and `$expr` compares them.

```
db.attendance.aggregate([
  { $addFields: { attended: { $cond: [{ $eq: ["$status", "absent"] }, 0, 100] } } },
  { $facet: { overall:    [ { $group: { _id: null, avg: { $avg: "$attended" } } } ],
              per_member: [ { $group: { _id: "$member_name", rate: { $avg: "$attended" },
```

```
                marked: { $sum: 1 } } } ] } },
  { $unwind: "$overall" }, { $unwind: "$per_member" },
  { $match: { $expr: { $lt: ["$per_member.rate", "$overall.avg"] } } },
  { $project: { _id: 0, name: "$per_member._id", marked: "$per_member.marked",
                rate_pct: { $round: ["$per_member.rate", 1] },
                    gym_average_pct: { $round: ["$overall.avg", 1] } } },
  { $sort: { rate_pct: 1, name: 1 } }, { $limit: 8 }
])

-- SQL: SELECT member_id, AVG(CASE WHEN status <> 'absent' THEN 100 ELSE 0 END) AS rate
--      FROM attendance GROUP BY member_id
--      HAVING rate < (SELECT AVG(CASE WHEN status <> 'absent' THEN 100 ELSE 0 END) FROM
    attendance)
--      ORDER BY rate LIMIT 8;
```

Table 6.10: Output of Q10 (gym-wide average 77.1%)

name	marked	rate_pct	gym_average_pct
Isha Nair	5	20	77.1
Keerthana Iyer	10	40	77.1
Meera Varma	24	45.8	77.1
Gautham Joshi	14	50	77.1
Isha Joshi	10	50	77.1
Karthik Krishnan	8	50	77.1
Siddharth Varma	24	50	77.1
Nandini Menon	10	60	77.1

Q11. Correlated subquery: members with more than one unpaid bill. `$lookup` with `let` and an inner `pipeline` runs a sub-query for each member, correlated through the variable `$$mid`.

```
db.members.aggregate([
  { $lookup: { from: "payments", let: { mid: "$_id" }, as: "unpaid",
      pipeline: [ { $match: { $expr: { $and: [ { $eq: ["$member_id", "$$mid"] },
                                               { $in: ["$status", ["pending",
                                          "overdue"]] } ] } } } ] } },
  { $match: { $expr: { $gt: [{ $size: "$unpaid" }, 1] } } },
  { $project: { _id: 0, member_id: 1, name: 1, tier: "$membership.type",
                unpaid_bills: { $size: "$unpaid" }, amount_due: { $sum: "$unpaid.amount" } } },
  { $sort: { member_id: 1 } }
])

-- SQL: SELECT m.member_id, m.name FROM members m
--      WHERE (SELECT COUNT(*) FROM payments p WHERE p.member_id = m.member_pk
--             AND p.status IN ('pending','overdue')) > 1 ORDER BY m.member_id;
```

The query returns 17 members, each with two unpaid bills (August or September plus October). The first eight are shown; the full list is in Appendix D.

Table 6.11: Output of Q11 (first 8 of 17)

member_id	name	tier	unpaid_bills	amount_due
MEM1008	Isha Joshi	Basic	2	3000
MEM1009	Gautham Bhat	Premium	2	5000
MEM1014	Divya Das	VIP	2	8000
MEM1016	Divya Menon	Basic	2	3000
MEM1018	Aishwarya Das	VIP	2	8000
MEM1021	Karthik Sharma	Premium	2	5000
MEM1024	Ananya Das	Premium	2	5000
MEM1025	Vikram Kumar	Basic	2	3000

Q12. EXISTS subquery: classes taught by trainers who coach more than 8 members.

```
db.classes.aggregate([
  { $lookup: { from: "members", localField: "trainer_id", foreignField: "assigned_trainer_id",
      as: "m" } },
  { $match: { $expr: { $gt: [{ $size: "$m" }, 8] } } },
  { $project: { _id: 0, class_name: 1, trainer_name: 1, trainer_members: { $size: "$m" } } },
  { $sort: { class_name: 1 } }
])

-- SQL: SELECT c.class_name, c.trainer_name FROM classes c
--      WHERE EXISTS (SELECT 1 FROM members m WHERE m.assigned_trainer_id = c.trainer_id
--                    GROUP BY m.assigned_trainer_id HAVING COUNT(*) > 8);
```

Table 6.12: Output of Q12

class_name	trainer_name	trainer_members
Morning Yoga	Deepa Krishnan	12
Pilates Core	Deepa Krishnan	12
Spin Cycle	Farhan Ali	10
Strength 101	Sanjay Pillai	10

6.1.5 Set Operations

MongoDB implements set algebra on arrays with `$setUnion`, `$setIntersection` and `$setDifference`. The class rosters are arrays, so each set operation is evaluated over the names in two rosters, with `$facet` producing the two operand sets.

Q13. UNION: members enrolled in Morning Yoga or Evening Yoga, each counted once.

```
db.classes.aggregate([
  { $facet: { a: [ { $match: { class_name: "Morning Yoga" } },
      { $project: { n: "$enrolled_members.member_name" } } ],
              b: [ { $match: { class_name: "Evening Yoga" } },
                  { $project: { n: "$enrolled_members.member_name" } } ] } },
  { $project: { result: { $setUnion: [ { $arrayElemAt: ["$a.n", 0] }, { $arrayElemAt: ["$b.n",
      0] } ] } } }
])

-- SQL: SELECT member_name FROM enrollments WHERE class_name = 'Morning Yoga'
--      UNION
--      SELECT member_name FROM enrollments WHERE class_name = 'Evening Yoga';
```

Result: Morning Yoga has 13 members and Evening Yoga 19; the union has **27** distinct members, so 5 members attend both yoga classes.

Q14. INTERSECT: members enrolled in both CrossFit Burn and Strength 101 (same pattern with `$setIntersection`). Result: **5** members: Dev Varma, Divya Menon, Meera Varma, Rahul Reddy and Siddharth Sharma.

Q15. EXCEPT: active members not enrolled in any class (`$setDifference` of the set of active members and the set of members found in any roster). Result: of 50 active members, 47 are enrolled somewhere, so **3** are not: Ashwin Krishnan, Rahul Sharma and Swathi Varma. These are natural targets for a class promotion.

```
-- SQL: SELECT name FROM members WHERE status = 'active'
--      EXCEPT
--      SELECT member_name FROM enrollments;
```

6.1.6 Analytical Reports and Graph Query

In addition to Q1 to Q15, the application has eight production report pipelines (A1 to A8) and a recursive graph query, summarised in Table 6.13. Their outputs are discussed in Chapter 7.

Table 6.13: Report pipelines and graph queries

ID	Report	Main stages and operators
A1	Monthly revenue by tier	$match, $lookup, $unwind, two $group stages, $dateToString, $push
A2	Class popularity	$group with $cond and $addToSet, $lookup, $size, $divide, $round
A3	Members at risk (attendance below 60%)	$group, $addFields, $match, $lookup
A4	Outstanding dues with priority	Date subtraction, $floor, $max, $group, nested $cond
A5	Trainer workload	Three $lookup joins, $filter, $size
A6	Membership overview in one query	$facet with four sub-pipelines, $bucket on computed age
A7	Most prescribed exercises	$unwind on embedded array, $group, $avg
A8	Peak check-in hours	$substr, $concat, $group
G1, G2	Referral network and top referrers	Recursive $graphLookup with depthField and maxDepth

6.2 Performance Tuning

6.2.1 Execution Plans with explain()

Every index was verified with `explain("executionStats")` [16]. Table 6.14 compares the member attendance history query (the most frequent dashboard query) with a forced collection scan (`hint({ $natural: 1 })`) and with the index `ix_member_date` on the 580-document sample data.

Table 6.14: Execution plans for the member attendance history query (sample data)

Plan	Winning stage	Documents examined	Keys examined	Returned	In-memory sort
Without index	COLLSCAN	580	0	5	Yes
With ix_member_date	IXSCAN	5	5	5	No

The index reduced documents examined from 580 to 5, and because the index stores `date` in descending order, results arrive already sorted, so the blocking in-memory sort stage disappears.

6.2.2 Benchmark at Scale

Timings on 580 documents are all below 1 ms, so `benchmark.py` builds a separate database with **200,000 attendance** and **60,000 payment** documents of the same shape and measures three report queries (median of 5 runs on MongoDB 8.3.7).

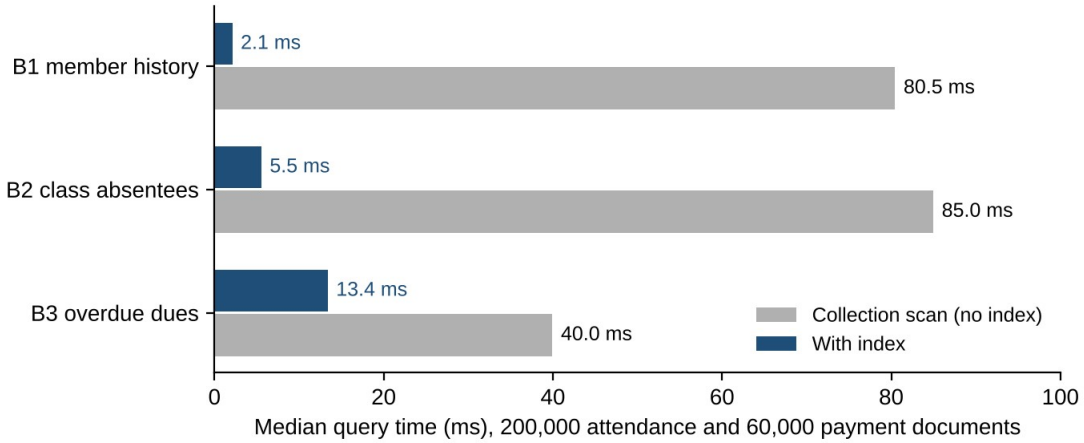


Figure 6.1: Median query time with and without indexes at 200,000 documents

Table 6.15: Benchmark results

Query	Plan	Documents examined	Keys examined	Returned	Median time
B1 member attendance history	COLLSCAN	200,000	0	105	80.46 ms
B1 member attendance history	ix_member_date	105	105	105	2.10 ms
B2 absentees of one class	COLLSCAN	200,000	0	998	84.97 ms
B2 absentees of one class	ix_class_status	998	998	998	5.53 ms
B3 overdue dues report	COLLSCAN	60,000	0	2,965	39.95 ms
B3 overdue dues report	px_unpaid_dues (partial)	2,965	2,966	2,965	13.38 ms
B3 overdue dues report	Full index on same keys	2,965	2,967	2,965	12.78 ms

Findings.

- The compound index `ix_member_date` made the member history **38 times faster** (80.46 ms to 2.10 ms) and reduced documents examined by a factor of about 1,900.
- `ix_class_status` made the class absentee query **15 times faster**.
- The partial index `px_unpaid_dues` was as fast as a full index on the same keys (13.38 ms against 12.78 ms) but occupied **60 KB instead of 604 KB**, ten times less, because it indexes only the 5% of bills that are unpaid. As paid history grows, the full index keeps growing while the partial index stays small.

6.2.3 Other Tuning Measures

- **Covered and projected reads:** queries project only the fields the screen needs, and the password hash is always excluded.
- **Pipeline ordering:** every pipeline starts with `$match` so filtering happens before joins and grouping, letting the first stage use an index.
- **Denormalized names:** copying member, trainer and class names into referencing documents removes a `$lookup` from most list screens.
- **Pagination:** member lists use sort, skip and limit with page size 10.
- **Atomic updates instead of read-modify-write:** enrolment, attendance marking and the overdue sweep are single server-side updates, which avoids round trips and race conditions.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 Output Screenshots and Sample Reports

All results below come from running the code in the repository against a live MongoDB replica set: the terminal screenshots were captured on macOS with MongoDB 8.3.11, and the query, benchmark and trigger outputs were produced with MongoDB 8.3.7. The full set of screenshots is in Appendix F.

7.1.1 Sample Data

Table 7.1: Sample data loaded

Collection	Documents	Contents
trainers	8	Specialisations, certifications, weekly availability
members	60	24 Basic, 22 Premium, 14 VIP; 50 active, 6 expired, 4 frozen; 5 cities; referral links
workout_plans	76	60 active and 16 completed plans with 4 exercises each
classes	8	Yoga, Zumba, CrossFit, HIIT, Cardio, Pilates, Strength; 2 weekly slots each
class_sessions	64	Four weeks of sessions, 7 September to 3 October 2026
attendance	580	Present, late or absent for every enrolled member in every completed session
payments	220	July to October 2026 bills: paid, pending and overdue
Total	**1,016**	All 1,016 documents pass the seven validators

7.1.2 Revenue Report (A1)

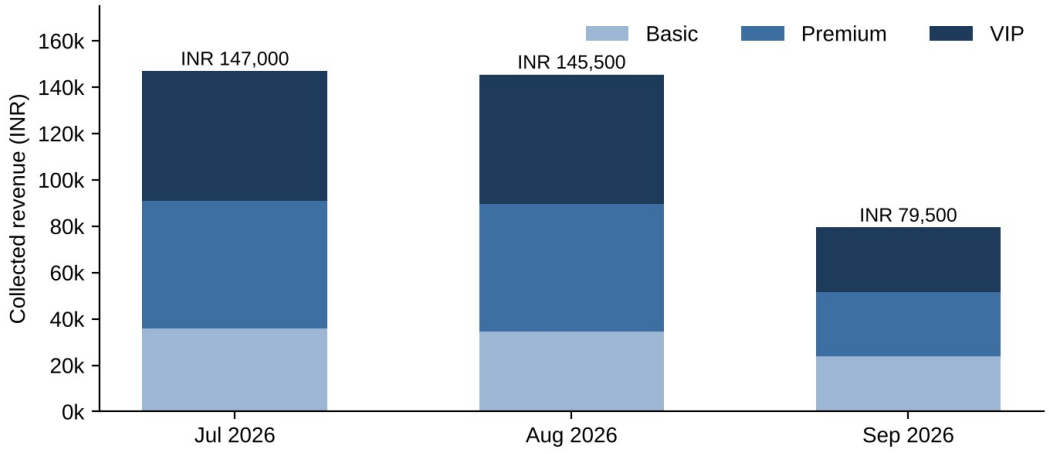


Figure 7.1: Collected revenue by month and membership tier (pipeline A1)

Revenue was INR 147,000 in July and INR 145,500 in August, then fell to INR 79,500 in September. The overview report A6 and the dues report A4 explain the drop: six memberships

expired on 31 August, four are frozen and sixteen September bills were unpaid at the reference date. VIP members are 14 of 60 members but produced INR 56,000 of July revenue, 38% of the total.

7.1.3 Class Popularity (A2)

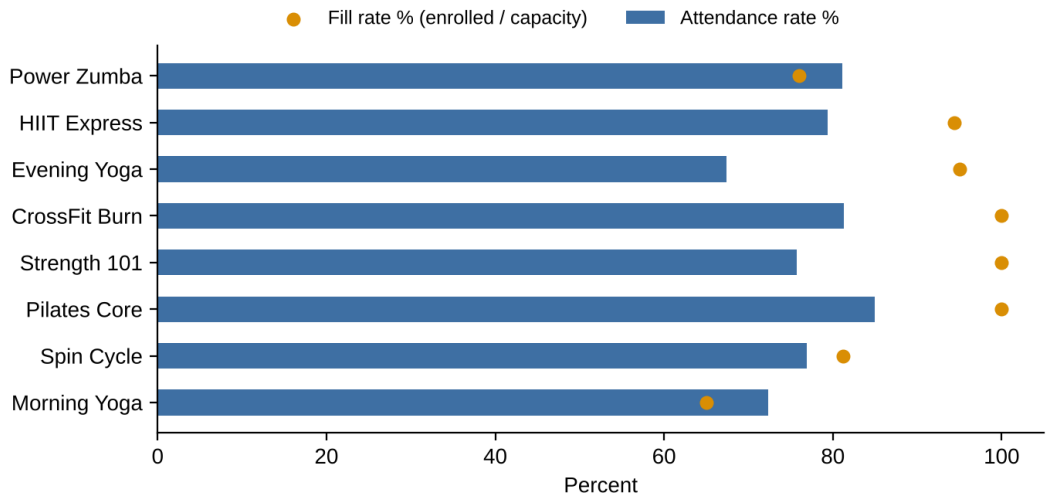


Figure 7.2: Attendance rate and fill rate by class (pipeline A2)

Table 7.2: Class popularity report (A2)

Class	Trainer	Enrolled / capacity	Sessions held	Average turnout	Attendance %
Power Zumba	Meenakshi Iyer	19 / 25	5	15.4	81.1
HIIT Express	Rohit Sharma	17 / 18	4	13.5	79.4
Evening Yoga	Kavitha Reddy	19 / 20	5	12.8	67.4
CrossFit Burn	Vivek Nair	15 / 15	5	12.2	81.3
Strength 101	Sanjay Pillai	14 / 14	5	10.6	75.7
Pilates Core	Deepa Krishnan	12 / 12	5	10.2	85.0
Spin Cycle	Farhan Ali	13 / 16	4	10.0	76.9
Morning Yoga	Deepa Krishnan	13 / 20	5	9.4	72.3

7.1.4 Retention and Collections (A3, A4)

Table 7.3: Members at risk, attendance below 60% (A3)

member_id	Name	Tier	Sessions marked	Attended	Rate %
MEM1052	Isha Nair	Premium	5	1	20.0
MEM1020	Keerthana Iyer	Premium	10	4	40.0
MEM1030	Meera Varma	Basic	24	11	45.8
MEM1008	Isha Joshi	Basic	10	5	50.0
MEM1019	Siddharth Varma	Basic	24	12	50.0
MEM1039	Gautham Joshi	Premium	14	7	50.0
MEM1053	Karthik Krishnan	Basic	8	4	50.0

Table 7.4: Outstanding dues, first six rows of the collections worklist (A4)

member_id	Name	Tier	Bills	Amount due	Days overdue	Priority
MEM1016	Divya Menon	Basic	2	3,000	50	HIGH
MEM1014	Divya Das	VIP	2	8,000	19	HIGH
MEM1018	Aishwarya Das	VIP	2	8,000	19	HIGH
MEM1038	Nandini Das	VIP	2	8,000	19	HIGH
MEM1046	Nandini Menon	VIP	2	8,000	19	HIGH
MEM1009	Gautham Bhat	Premium	2	5,000	19	HIGH

7.1.5 Workload, Overview and Referral Network (A5, A6, G1, G2)

Table 7.5: Trainer workload (A5)

trainer_id	Name	Members	Active plans	Classes
TRN03	Deepa Krishnan	12	12	2
TRN04	Sanjay Pillai	10	10	1
TRN06	Farhan Ali	10	10	1
TRN08	Vivek Nair	8	8	1
TRN07	Kavitha Reddy	7	7	1
TRN01	Priya Menon	6	6	0
TRN02	Rohit Sharma	5	5	1
TRN05	Meenakshi Iyer	2	2	1
TRN09	Nisha Varghese (created in the CRUD demo)	0	0	1

The membership overview (A6) returns four summaries in one query: tiers (Basic 24, Premium 22, VIP 14), statuses (active 50, expired 6, frozen 4), age bands (18 to 24: 12, 25 to 34: 15, 35 to 44: 22, 45 to 59: 11) and cities (Chennai 16, Bengaluru 13, Hosur 12, Katpadi 10, Vellore 9). The most prescribed exercises (A7) are Plank (23 plans), Rowing Machine (20) and Squat (19,

average 67.6 kg), and the busiest check-in hours (A8) are 18:00 (115 check-ins), 06:00 (98) and 17:00 (82).

The referral query G2 ranks members by the size of their whole downstream network. Lakshmi Bhat (MEM1036) referred only one member directly, yet five members across three levels joined through that referral (G1): Arjun Rao (level 1), Nandini Menon and Varun Das (level 2), and Ashwin Krishnan and Aishwarya Menon (level 3). A count of direct referrals alone would miss this.

7.1.6 Integrity, Transactions and Triggers

Table 7.6: Results of integrity, transaction and trigger tests

Test	Result on the live server
Insert a member with tier "Gold"	Rejected by validator, error 121 Document failed validation
Insert a member with a duplicate email	Rejected, DuplicateKeyError on uq_email
Create a member for non-existent trainer TRN77	Rejected, IntegrityError
Enrol three members in a class of capacity 2	Third enrolment blocked atomically
Rename a member	New name in members, workout_plans, payments and class roster
Mark attendance late, then present	Still exactly one attendance row (upsert)
Delete a member with unpaid bills	Blocked (RESTRICT)
Delete a member without dues	Member and 1 plan removed (roster entry already withdrawn); 0 orphans
T1 renew membership with simulated failure after the payment insert	Aborted; payments count 221 to 221 and end date unchanged (rollback)
T1 renew MEM1010 for 3 months	Committed: bill of INR 4,500; end date 31 Mar 2027 to 30 Jun 2027
T2 close session SES9006 with 3 roll-call marks	Committed: 3 attendance rows, session completed
T2 with a member not on the roster	Aborted after earlier writes; attendance 583 to 583; session still scheduled
TR2 a bill paid by expired member MEM1004	Trigger set membership status to active
TR1 audit trail	Insert and update of the bill, the reactivation and a phone change all logged
Automated tests (pytest)	6 passed

```
======================================================================
3d. MULTI-DOCUMENT ACID TRANSACTIONS
======================================================================

--- T1 renew_membership with a simulated failure -> rollback
aborted: Simulated failure after payment insert
payments count 220 -> 220 (unchanged) | end_date still 2027-03-31

--- T1 renew_membership, 3 months -> commit
{'payment_id': 'PAY7221', 'amount': 4500, 'new_end_date': '2027-06-30'}
end_date 2027-03-31 -> 2027-06-30 | payments 220 -> 221

--- T2 close_session: attendance rows + session status in one transaction
SES9006 Morning Yoga 2026-09-24: {'session_id': 'SES9006', 'attendance_rows': 3}
session status now: completed

--- T2 with a member not on the roster -> whole transaction rolled back
aborted: MEM1001 is not enrolled in Pilates Core
attendance rows 583 -> 583 | SES9046 status: scheduled

Demo complete.
```

Figure 7.3: Live run, transactions T1 and T2 with commit and rollback

```
(.venv) gym-membership-db % python demo.py --section trig
================================================================================
3e. TRIGGERS: change streams on members and payments
================================================================================

--- TR2: MEM1004 was expired; after paying PAY7223 the trigger set status to active

--- TR1: audit_log written by the trigger
collection  operation           key      changed_fields
----------  ------------------  -------  ---------------------------------
payments    insert              PAY7223
payments    update              PAY7223  payment_date, payment_mode, status
members     trigger:reactivate  MEM1004  membership.status
members     update              MEM1002  phone
members     update              MEM1004  membership.status

Demo complete.
```

Figure 7.4: Live run, change-stream triggers TR1 and TR2

7.2 Evaluation Against Objectives

Table 7.7: Evaluation against the objectives of Section 1.4

ID	Objective	Evidence	Status
O1	Document schema derived from a formal EER model	EER (Figure 4.4), 3NF schema (Figure 4.5), document model (Figure 4.6), Tables 4.3 to 4.11	Met
O2	Full CRUD with referential integrity	36 functions; ten referential rules; CRUD results in Table 7.6 and Figures F.3, F.4	Met
O3	Database-level data quality	7 validators, unique indexes; validator rejection (Figure F.1); 1,016 documents validate	Met
O4	Analytical insight through aggregation	Q7 to Q15, reports A1 to A8, G1 and G2 (Sections 6.1, 7.1)	Met
O5	Authentication and role-based access	bcrypt login works for correct password and rejects wrong one; roles in Table 3.2; member_dashboard view	Met (web screens planned)
O6	Performance through indexing	19 indexes; explain plans; 38 times faster member history; partial index 10 times smaller (Section 6.2)	Met
O7	Consistency through transactions and triggers	T1, T2 commit and rollback; TR1, TR2 triggers (Table 7.6)	Met
O8	Reproducible, version-controlled implementation	Public GitHub repository, seeded data, 6 passing tests, README, user manual	Met

7.3 Comparison with the Existing System

Table 7.8: Existing manual system compared with the proposed system

Aspect	Existing (register and spreadsheet)	Proposed (MongoDB system)
Data integrity	None; duplicates and typos common	Validators, unique keys, 10 referential rules
Overbooking	Possible	Impossible: atomic capacity check
Dues tracking	Manual, often late	Automatic overdue sweep and prioritised worklist (A4)
Revenue report	Hours of manual work	One query, milliseconds (A1)
Retention	Inactive members noticed after they leave	At-risk report (A3) flags members below 60% attendance
Concurrency	Last edit wins	Atomic updates, transactions with snapshot isolation
Audit trail	None	TR1 logs every change to members and payments
Security	Open register, shared sheets	bcrypt hashes, role-based access, hashes never returned
Backup	Single copy	Journal, replica set, mongodump, JSON export
Cost	Free	Free (open source)

7.4 Discussion of Findings

- **The document model fitted the domain.** The three most frequent screens each need one document read: a workout plan with its exercises, a class with its timetable and roster, and the member dashboard view. In the 3NF relational design these needed joins across two to four tables.
- **Integrity on a NoSQL database is achievable but must be designed.** Validators covered NOT NULL and CHECK constraints well. Foreign keys had to be enforced in code, which worked because every write goes through one function per operation, and transactions covered the cases where several documents change together. The most effective technique was moving the capacity check into the update filter, which removes the race condition without any locking.
- **Analytics revealed actionable patterns.** Pilates Core is full and has the best attendance (85.0%), while Morning Yoga is only 65% full and has the lowest turnout per session (9.4), so it is a candidate for a new time slot. Yoga as a category has the highest absence rate (31%), and seven members attend fewer than 60% of their sessions. Two of them, Isha Nair and Isha Joshi, also have an overdue September bill, which suggests that disengagement and non-payment go together and that the at-risk and dues reports should be read side by side.
- **Revenue concentration.** UPI is the dominant payment mode (66% of paid bills), and VIP members generate a disproportionate share of revenue, so retaining them matters most.

- **Indexes matter more as data grows.** At 580 documents, indexes made no measurable time difference; at 200,000 they made the key queries 15 to 38 times faster. A partial index gave the same speed as a full index at a tenth of the size.
- **Graph traversal adds business value cheaply.** `$graphLookup` answered the referral question inside the same database, with no separate graph store.

7.5 Limitations Observed

- Referential integrity depends on all writes going through the application layer; a direct write from mongosh could bypass the rules (validators still apply).
- Denormalized copies (names) must be kept in sync; a missed update path would leave stale names.
- `$lookup` joins are slower than relational joins on very large collections, so heavy ad hoc joins would need careful indexing or pre-aggregation.
- Transactions and change streams require a replica set, which adds setup steps for new users.
- The dataset is synthetic and small; behaviour with real gym data and concurrent users was not measured.
- Role-based access is enforced in the application, not with MongoDB's built-in role-based access control (RBAC) users, and the web interface is not yet built.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 Summary of the Work

This project designed and implemented a Gym Membership Management System on MongoDB by following the full database life cycle. Requirements were captured as eight functional and five non-functional requirements and twelve business rules. A conceptual EER model with nine entities, eleven relationships and a USER generalization was mapped to a relational schema of twelve relations in third normal form and then to seven MongoDB collections through justified denormalization. The physical design added BSON types, nineteen indexes, validators and a replica set. The implementation provides 36 CRUD functions with ten referential integrity rules, eight analytical pipelines, a recursive referral query, a view, two ACID transactions and two change-stream triggers, loaded with 1,016 reproducible documents. Fifteen representative queries, explain plans and a 200,000-document benchmark demonstrated correctness and performance, and all objectives were met.

8.2 Key Contributions

- A complete, documented design path from EER to 3NF to a document model, with every denormalization justified and protected.
- A demonstration of relational-grade integrity on MongoDB: validators as constraints, unique indexes as keys, application-level referential actions, atomic conditional updates, transactions and triggers.
- A reusable set of fifteen queries that maps every SQL query category to its MongoDB equivalent, including set operations.
- Measured evidence for indexing decisions, including the advantage of a partial index.
- An open, reproducible repository that any evaluator can run to obtain the same results.

8.3 Future Enhancements

- **Web dashboard** for administrators, trainers and members, using Flask or React on top of the existing `gym_db` layer.
- **Database-level security** with MongoDB role-based access control users per role, field-level encryption for phone numbers and TLS.
- **Online payments** through a UPI or card gateway, with webhook events written to `payments` in a transaction.
- **Notifications** (SMS, WhatsApp or email) for overdue bills and at-risk members, driven by change streams.
- **Multi-branch support** with a `branch_id` field and sharding of attendance and payments on a hashed member key.

- **Predictive analytics:** a churn prediction model trained on attendance and payment history, and demand forecasting for class timetables.
- **Schema versioning** with a `schema_version` field and migration scripts.

8.4 Concluding Remarks

The project showed that the choice between relational and NoSQL databases is not a choice between integrity and flexibility. By starting from a rigorous relational design and denormalizing only where the queries justify it, a document database can give a gym consistent, secure and fast data management with built-in analytics. The experience of carrying one design through every stage of the life cycle, and of measuring rather than assuming performance, is the main learning outcome of this work.

PART C: REFERENCES AND APPENDICES

References

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377 to 387, 1970.

[2] P. P. Chen, "The entity-relationship model: Toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9 to 36, 1976.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[4] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12 to 27, 2011.

[6] A. Pavlo and M. Aslett, "What's really new with NewSQL?," *ACM SIGMOD Record*, vol. 45, no. 2, pp. 45 to 55, 2016.

[7] E. Brewer, "CAP twelve years later: How the 'rules' have changed," *Computer*, vol. 45, no. 2, pp. 23 to 29, 2012.

[8] M. Stonebraker, "SQL databases v. NoSQL databases," *Communications of the ACM*, vol. 53, no. 4, pp. 10 to 11, 2010.

[9] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[10] MongoDB, Inc., "Data modeling," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/data-modeling/

[11] MongoDB, Inc., "Schema validation," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/schema-validation/

[12] MongoDB, Inc., "Aggregation operations," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/aggregation/

[13] MongoDB, Inc., "Transactions," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[14] MongoDB, Inc., "Partial indexes," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/index-partial/

[15] MongoDB, Inc., "$graphLookup (aggregation stage)," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/reference/operator/aggregation/graphLookup/

[16] MongoDB, Inc., "Explain results," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/reference/explain-results/

[17] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81 to 91.

[18] OWASP Foundation, "Password storage cheat sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[19] P. A. Grassi et al., "Digital identity guidelines: Authentication and lifecycle management," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST SP 800-63B, 2017.

[20] *Information technology: Database languages SQL, Part 1: Framework (SQL/Framework)*, ISO/IEC 9075-1:2023, International Organization for Standardization, 2023.

[21a] Mindbody, "Fitness studio and gym software." [Online]. Available: https://www.mindbodyonline.com

[21b] Glofox, "Gym management software." [Online]. Available: https://www.glofox.com

[21c] Zen Planner, "Gym management software." [Online]. Available: https://zenplanner.com

[21d] Gymdesk, "Gym management software." [Online]. Available: https://gymdesk.com

[21e] Wodify, "Best fitness studio management software for 2026: 7 platforms compared." [Online]. Available: https://www.wodify.com/blog/best-fitness-studio-software

[22] solid IT GmbH, "DB-Engines ranking." [Online]. Available: https://db-engines.com/en/ranking

[23] Government of India, *The Digital Personal Data Protection Act, 2023*, Act No. 22 of 2023, Ministry of Law and Justice, 2023.

[24] The PostgreSQL Global Development Group, "JSON types," *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/datatype-json.html

[25] K. Banker, P. Bakkum, S. Verch, D. Garrett and T. Hawkins, *MongoDB in Action*, 2nd ed. Shelter Island, NY, USA: Manning, 2016.

[26] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35 to 40, 2010.

[27] R. Taft et al., "CockroachDB: The resilient geo-distributed SQL database," in *Proc. ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1493 to 1509.

[28] MongoDB, Inc., "Change streams," *MongoDB Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/changeStreams/

[29] MongoDB, Inc., "PyMongo documentation." [Online]. Available: https://pymongo.readthedocs.io/

Appendix A: Complete DDL Scripts

MongoDB has no CREATE TABLE statement; the equivalent data definition is `createCollection` with a `$jsonSchema` validator, `createIndex` and `createView`. The script below is `mongosh/ddl.js`, generated from the same definitions the Python code uses, so the two can never disagree. It creates the seven collections, the nineteen indexes and the view. The Python equivalent is `gym_db/schema.py` and `gym_db/indexes.py`.

```
// Generated by queries.py --export-js. Creates EMPTY collections (drops existing ones), then indexes.
// Load data afterwards with mongoimport (data/*.json) or use python demo.py, which does all of this.
//   mongosh "mongodb://localhost:27017/?directConnection=true" mongosh/ddl.js

// Data definition for gym_db: 7 collections with $jsonSchema validators, then 19 indexes.
db = db.getSiblingDB("gym_db");

db.getCollection("members").drop();
db.createCollection("members", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["member_id", "name", "email", "password_hash", "role", "membership"],
      properties: {
        member_id: { bsonType: "string", pattern: "^MEM[0-9]{4}$" },
        name: { bsonType: "string", minLength: 2 },
        email: { bsonType: "string", pattern: "^.+@.+\\..+$" },
        phone: { bsonType: "string", pattern: "^[0-9]{10}$" },
        gender: { enum: ["Male", "Female", "Other"] },
        role: { enum: ["member"] },
        membership: {
          bsonType: "object",
          required: ["type", "status", "start_date", "end_date"],
          properties: {
            type: { enum: ["Basic", "Premium", "VIP"] },
            status: { enum: ["active", "expired", "frozen"] },
            start_date: { bsonType: "date" },
            end_date: { bsonType: "date" }
          }
        },
        assigned_trainer_id: { bsonType: ["objectId", "null"] },
        referred_by: { bsonType: ["objectId", "null"] }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("trainers").drop();
db.createCollection("trainers", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["trainer_id", "name", "email", "password_hash", "role"],
      properties: {
        trainer_id: { bsonType: "string", pattern: "^TRN[0-9]{2}$" },
        role: { enum: ["trainer"] },
        specialization: { bsonType: "array", items: { bsonType: "string" } },
        experience_years: { bsonType: ["int", "long", "double"], minimum: 0 },
        availability: {
          bsonType: "array",
          items: { bsonType: "object", required: ["day", "start_time", "end_time"] }
```

```
        }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("workout_plans").drop();
db.createCollection("workout_plans", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["plan_id", "member_id", "trainer_id", "goal", "status", "exercises"],
      properties: {
        member_id: { bsonType: "objectId" },
        trainer_id: { bsonType: "objectId" },
        duration_weeks: { bsonType: ["int", "long"], minimum: 1, maximum: 52 },
        status: { enum: ["active", "completed"] },
        exercises: {
          bsonType: "array",
          minItems: 1,
          items: {
            bsonType: "object",
            required: ["day", "exercise_name", "sets", "reps"],
            properties: {
              sets: { bsonType: ["int", "long"], minimum: 1 },
              reps: { bsonType: ["int", "long"], minimum: 1 }
            }
          }
        }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("classes").drop();
db.createCollection("classes", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["class_id", "class_name", "category", "trainer_id", "capacity"],
      properties: {
        trainer_id: { bsonType: "objectId" },
        capacity: { bsonType: ["int", "long"], minimum: 1, maximum: 50 },
        schedule: { bsonType: "array" },
        enrolled_members: { bsonType: "array" }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("class_sessions").drop();
db.createCollection("class_sessions", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["session_id", "class_id", "date", "status"],
      properties: {
        class_id: { bsonType: "objectId" },
        date: { bsonType: "date" },
        status: { enum: ["scheduled", "completed", "cancelled"] }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
```

```
});

db.getCollection("attendance").drop();
db.createCollection("attendance", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["session_id", "class_id", "member_id", "date", "status"],
      properties: {
        session_id: { bsonType: "objectId" },
        member_id: { bsonType: "objectId" },
        status: { enum: ["present", "absent", "late"] }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.getCollection("payments").drop();
db.createCollection("payments", {
  validator: { $jsonSchema: {
      bsonType: "object",
      required: ["payment_id", "member_id", "amount", "status", "due_date"],
      properties: {
        member_id: { bsonType: "objectId" },
        amount: { bsonType: ["int", "long", "double"], minimum: 0 },
        currency: { enum: ["INR"] },
        payment_mode: { enum: ["cash", "card", "UPI", null] },
        status: { enum: ["paid", "pending", "overdue"] },
        due_date: { bsonType: "date" }
      }
    } },
  validationLevel: "strict",
  validationAction: "error"
});

db.members.createIndex({ email: 1 }, { unique: true, name: "uq_email" });
db.members.createIndex({ member_id: 1 }, { unique: true, name: "uq_member_id" });
db.members.createIndex({ assigned_trainer_id: 1 }, { name: "ix_trainer" });
db.members.createIndex({ "membership.type": 1, "membership.status": 1 }, { name: "ix_tier_status" });
db.members.createIndex({ referred_by: 1 }, { name: "ix_referred_by" });
db.trainers.createIndex({ email: 1 }, { unique: true, name: "uq_email" });
db.trainers.createIndex({ trainer_id: 1 }, { unique: true, name: "uq_trainer_id" });
db.workout_plans.createIndex({ member_id: 1, status: 1 }, { name: "ix_member_status" });
db.workout_plans.createIndex({ trainer_id: 1 }, { name: "ix_trainer" });
db.classes.createIndex({ trainer_id: 1 }, { name: "ix_trainer" });
db.classes.createIndex({ "enrolled_members.member_id": 1 }, { name: "ix_enrolled_member" });
db.classes.createIndex({ class_name: "text", category: "text" }, { name: "tx_class_search" });
db.class_sessions.createIndex({ class_id: 1, date: 1 }, { name: "ix_class_date" });
db.attendance.createIndex({ member_id: 1, date: -1 }, { name: "ix_member_date" });
db.attendance.createIndex({ session_id: 1, member_id: 1 }, { unique: true, name: "uq_session_member" });
db.attendance.createIndex({ class_id: 1, status: 1 }, { name: "ix_class_status" });
db.payments.createIndex({ member_id: 1, status: 1 }, { name: "ix_member_status" });
db.payments.createIndex({ due_date: 1 }, { name: "ix_due_date" });
db.payments.createIndex({ status: 1, due_date: 1 }, {
  name: "px_unpaid_dues",
  partialFilterExpression: { status: { "$in": ["pending", "overdue"] } }
});

// Read-only view used by the member dashboard (hides password_hash).
db.getCollection("member_dashboard").drop();
db.createView("member_dashboard", "members", [
  {
    "$lookup": { from: "trainers", localField: "assigned_trainer_id", foreignField: "_id", as: "t" }
  },
```

```
  { "$lookup": { from: "payments", localField: "_id", foreignField: "member_id", as: "p" } },
  {
    "$project": {
      _id: 0,
      member_id: 1,
      name: 1,
      tier: "$membership.type",
      status: "$membership.status",
      trainer: { "$first": "$t.name" },
      unpaid_amount: {
        "$sum": {
          "$map": {
            input: { "$filter": { input: "$p", as: "x", cond: { "$ne": ["$$x.status", "paid"] } } },
            as: "u",
            in: "$$u.amount"
          }
        }
      }
    }
  }
]);

print("gym_db created: " + db.getCollectionNames().sort().join(", "));
```

Appendix B: Sample Data and DML Scripts

B.1 Sample Documents

One document from each collection as stored in MongoDB (Extended JSON). The complete dataset of 1,016 documents is in `data/*.json` in the repository and can be loaded with `mongoimport --jsonArray`.

members


```
{
  "_id": {
    "$oid": "6ae07293995dced28d294f9c"
  },
  "member_id": "MEM1001",
  "name": "Harish Kumar",
  "email": "harish.kumar1@example.com",
  "phone": "9952304698",
  "dob": {
    "$date": "1987-04-04T00:00:00Z"
  },
  "gender": "Male",
  "password_hash": "$2b$04$/zukbpdrHdQvDWSJSJ7LB.h77zHVCrIdcDUy8dQziLg22wwXkUJkm",
  "role": "member",
  "membership": {
    "type": "VIP",
    "status": "active",
    "start_date": {
      "$date": "2026-01-01T00:00:00Z"
    },
    "end_date": {
      "$date": "2026-12-31T00:00:00Z"
    }
  },
  "assigned_trainer_id": {
    "$oid": "6ae07293995dced28d294f94"
  },
  "referred_by": null,
  "address": {
    "line1": "193 Katpadi Main Road",
    "city": "Bengaluru",
    "pincode": "560034"
  },
  "emergency_contact": {
    "name": "Aishwarya Kumar",
    "phone": "9817771848"
  },
  "created_at": {
    "$date": "2025-12-19T00:00:00Z"
  }
}
```


trainers


```
{
  "_id": {
    "$oid": "6ae07293995dced28d294f94"
  },
  "trainer_id": "TRN01",
```

```
  "name": "Priya Menon",
  "email": "priya.menon@gym.com",
  "phone": "9845012341",
  "password_hash": "$2b$04$/zukbpdrHdQvDWSJSJ7LB.Wet2/x46WG15aGw/Saq2HHRyaw1Fa.O",
  "role": "trainer",
  "specialization": [
    "Strength Training",
    "CrossFit"
  ],
  "certification": "ACE-CPT",
  "experience_years": 6,
  "availability": [
    {
      "day": "Mon",
      "start_time": "06:00",
      "end_time": "14:00"
    },
    {
      "day": "Wed",
      "start_time": "06:00",
      "end_time": "14:00"
    },
    {
      "day": "Sat",
      "start_time": "06:00",
      "end_time": "14:00"
    },
    {
      "day": "Thu",
      "start_time": "06:00",
      "end_time": "14:00"
    }
  ],
  "created_at": {
    "$date": "2024-01-31T00:00:00Z"
  }
}
```


workout_plans


```
{
  "_id": {
    "$oid": "6ae07293995dced28d294fd8"
  },
  "plan_id": "WP5001",
  "member_id": {
    "$oid": "6ae07293995dced28d294f9c"
  },
  "member_name": "Harish Kumar",
  "trainer_id": {
    "$oid": "6ae07293995dced28d294f94"
  },
  "trainer_name": "Priya Menon",
  "goal": "Flexibility",
  "duration_weeks": 10,
  "status": "active",
  "exercises": [
    {
      "day": "Mon",
      "exercise_name": "Plank",
      "sets": 4,
      "reps": 8,
      "weight_kg": 0,
      "notes": ""
```

```
    },
    {
      "day": "Tue",
      "exercise_name": "Sun Salutation",
      "sets": 4,
      "reps": 10,
      "weight_kg": 0,
      "notes": ""
    },
    {
      "day": "Wed",
      "exercise_name": "Cat-Cow Stretch",
      "sets": 3,
      "reps": 10,
      "weight_kg": 0,
      "notes": ""
    },
    {
      "day": "Fri",
      "exercise_name": "Hip Opener Flow",
      "sets": 5,
      "reps": 8,
      "weight_kg": 0,
      "notes": ""
    }
  ],
  "created_at": {
    "$date": "2026-01-01T00:00:00Z"
  },
  "updated_at": {
    "$date": "2026-01-01T00:00:00Z"
  }
}
```


classes (roster shortened)


```
{
  "_id": {
    "$oid": "6ae07293995dced28d295029"
  },
  "class_id": "CLS06",
  "class_name": "Pilates Core",
  "category": "Pilates",
  "trainer_id": {
    "$oid": "6ae07293995dced28d294f96"
  },
  "trainer_name": "Deepa Krishnan",
  "capacity": 12,
  "schedule": [
    {
      "day_of_week": "Tue",
      "start_time": "10:00",
      "end_time": "11:00",
      "room": "A2"
    },
    {
      "day_of_week": "Thu",
      "start_time": "10:00",
      "end_time": "11:00",
      "room": "A2"
    }
  ],
  "enrolled_members": [
    {
```

```
      "member_id": {
        "$oid": "6ae07293995dced28d294fa4"
      },
      "member_name": "Gautham Bhat",
      "enrolled_on": {
        "$date": "2026-07-11T00:00:00Z"
      }
    },
    {
      "member_id": {
        "$oid": "6ae07293995dced28d294fa3"
      },
      "member_name": "Isha Joshi",
      "enrolled_on": {
        "$date": "2026-07-12T00:00:00Z"
      }
    },
    {
      "...": "10 more"
    }
  ]
}
```


class_sessions


```
{
  "_id": {
    "$oid": "6ae07293995dced28d29502c"
  },
  "session_id": "SES9001",
  "class_id": {
    "$oid": "6ae07293995dced28d295024"
  },
  "class_name": "Morning Yoga",
  "trainer_id": {
    "$oid": "6ae07293995dced28d294f96"
  },
  "date": {
    "$date": "2026-09-07T00:00:00Z"
  },
  "start_time": "07:00",
  "end_time": "08:00",
  "status": "completed"
}
```


attendance


```
{
  "_id": {
    "$oid": "6ae07293995dced28d29506c"
  },
  "session_id": {
    "$oid": "6ae07293995dced28d29502c"
  },
  "class_id": {
    "$oid": "6ae07293995dced28d295024"
  },
  "member_id": {
    "$oid": "6ae07293995dced28d294fa8"
  },
  "member_name": "Manoj Shetty",
  "date": {
    "$date": "2026-09-07T00:00:00Z"
  },
```

```
  "status": "present",
  "check_in_time": "06:50"
}
```

payments

```
{
  "_id": {
    "$oid": "6ae07293995dced28d2952b0"
  },
  "payment_id": "PAY7001",
  "member_id": {
    "$oid": "6ae07293995dced28d294f9c"
  },
  "member_name": "Harish Kumar",
  "amount": 4000,
  "currency": "INR",
  "payment_mode": "card",
  "status": "paid",
  "billing_period_start": {
    "$date": "2026-07-01T00:00:00Z"
  },
  "billing_period_end": {
    "$date": "2026-07-31T00:00:00Z"
  },
  "payment_date": {
    "$date": "2026-07-08T00:00:00Z"
  },
  "due_date": {
    "$date": "2026-07-05T00:00:00Z"
  }
}
```

B.2 Sample Data Generator

`gym_db/seed.py` generates the sample data with a fixed random seed, so every run produces the same 1,016 documents and the same report figures.

```
"""Step 2: load a realistic, reproducible sample dataset.

Volumes (well above the 50-record minimum):
  trainers 8 | members 60 | workout_plans 75 | classes 8
  class_sessions 64 | attendance ~600 | payments ~200
Random choices use a fixed seed, so every run produces identical data.
"""
import random
from datetime import datetime, timedelta

import bcrypt
from bson import ObjectId

from .config import REFERENCE_DATE, TIER_FEES

rng = random.Random(2026)
# Cheap work factor for bulk seeding; the application uses the default (12) for real sign-ups.
_SALT = bcrypt.gensalt(rounds=4)


def _hash(pw: str) -> str:
    return bcrypt.hashpw(pw.encode(), _SALT).decode()


FIRST_M = ["Arjun", "Rahul", "Karthik", "Vikram", "Siddharth", "Aditya", "Rohan", "Nikhil",
```

```
          "Pranav", "Harish", "Gautham", "Varun", "Manoj", "Ashwin", "Dev"]
FIRST_F = ["Ananya", "Divya", "Kavya", "Meera", "Sneha", "Pooja", "Lakshmi", "Nandini",
          "Aishwarya", "Ritika", "Swathi", "Keerthana", "Shreya", "Isha", "Tara"]
LAST = ["Rao", "Iyer", "Nair", "Menon", "Reddy", "Sharma", "Krishnan", "Pillai",
        "Kumar", "Shetty", "Das", "Varma", "Joshi", "Bhat", "Subramanian"]
CITIES = [("Vellore", "632014"), ("Katpadi", "632007"), ("Chennai", "600040"),
          ("Hosur", "635109"), ("Bengaluru", "560034")]
DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]

TRAINERS = [
    ("Priya Menon", ["Strength Training", "CrossFit"], "ACE-CPT", 6),
    ("Rohit Sharma", ["HIIT", "Functional Training"], "NASM-CPT", 4),
    ("Deepa Krishnan", ["Yoga", "Pilates"], "RYT-500", 9),
    ("Sanjay Pillai", ["Bodybuilding", "Strength Training"], "ISSA-CPT", 11),
    ("Meenakshi Iyer", ["Zumba", "Aerobics"], "ZIN Licensed", 5),
    ("Farhan Ali", ["Spin", "Cardio"], "ACSM-EP", 3),
    ("Kavitha Reddy", ["Yoga", "Mobility"], "RYT-200", 7),
    ("Vivek Nair", ["CrossFit", "Olympic Lifting"], "CF-L2", 8),
]

CLASSES = [  # name, category, trainer index, capacity, [(day, start, end, room)]
    ("Morning Yoga", "Yoga", 2, 20, [("Mon", "07:00", "08:00", "A1"), ("Thu", "07:00", "08:00", "A1")]),
    ("Power Zumba", "Zumba", 4, 25, [("Tue", "18:00", "19:00", "B1"), ("Fri", "18:00", "19:00", "B1")]),
    ("CrossFit Burn", "CrossFit", 7, 15, [("Mon", "18:30", "19:30", "C1"), ("Wed", "18:30", "19:30", "C1")]),
    ("HIIT Express", "HIIT", 1, 18, [("Tue", "06:30", "07:15", "B2"), ("Sat", "08:00", "08:45", "B2")]),
    ("Spin Cycle", "Cardio", 5, 16, [("Wed", "07:00", "07:45", "S1"), ("Sat", "09:00", "09:45", "S1")]),
    ("Pilates Core", "Pilates", 2, 12, [("Tue", "10:00", "11:00", "A2"), ("Thu", "10:00", "11:00", "A2")]),
    ("Strength 101", "Strength", 3, 14, [("Mon", "17:00", "18:00", "G1"), ("Fri", "17:00", "18:00", "G1")]),
    ("Evening Yoga", "Yoga", 6, 20, [("Wed", "19:00", "20:00", "A1"), ("Sat", "17:00", "18:00", "A1")]),
]

GOALS = {
    "Muscle Gain": [("Bench Press", 40), ("Deadlift", 60), ("Squat", 50), ("Overhead Press", 25),
                    ("Barbell Row", 35), ("Bicep Curl", 10)],
    "Weight Loss": [("Kettlebell Swing", 12), ("Burpees", 0), ("Treadmill Intervals", 0),
                    ("Jump Squats", 0), ("Rowing Machine", 0), ("Mountain Climbers", 0)],
    "Endurance": [("Cycling", 0), ("Rowing Machine", 0), ("Lunges", 8), ("Plank", 0),
                  ("Box Jumps", 0), ("Treadmill Intervals", 0)],
    "Flexibility": [("Sun Salutation", 0), ("Hamstring Stretch", 0), ("Hip Opener Flow", 0),
                    ("Plank", 0), ("Cat-Cow Stretch", 0)],
    "Strength": [("Deadlift", 80), ("Squat", 70), ("Bench Press", 55), ("Pull Ups", 0),
                 ("Farmer Walk", 24)],
}
DAY_INDEX = {d: i for i, d in enumerate(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])}


def _trainers():
    docs = []
    for i, (name, spec, cert, yrs) in enumerate(TRAINERS, start=1):
        first = name.split()[0].lower()
        docs.append({
            "_id": ObjectId(),
            "trainer_id": f"TRN{i:02d}",
            "name": name,
            "email": f"{first}.{name.split()[1].lower()}@gym.com",
            "phone": f"98450{12340 + i:05d}",
            "password_hash": _hash(f"Trainer@{i:02d}"),
            "role": "trainer",
            "specialization": spec,
            "certification": cert,
            "experience_years": yrs,
            "availability": [{"day": d, "start_time": "06:00" if i % 2 else "12:00",
                              "end_time": "14:00" if i % 2 else "21:00"}
                             for d in rng.sample(DAYS, 4)],
```

```
            "created_at": datetime(2024, 1, 1) + timedelta(days=30 * i),
        })
    return docs


def _members(trainers):
    docs, used = [], set()
    # Target mix: 24 Basic, 22 Premium, 14 VIP; 50 active, 6 expired, 4 frozen
    tiers = ["Basic"] * 24 + ["Premium"] * 22 + ["VIP"] * 14
    statuses = ["active"] * 50 + ["expired"] * 6 + ["frozen"] * 4
    rng.shuffle(tiers)
    rng.shuffle(statuses)
    for i in range(60):
        gender = "Male" if i % 2 == 0 else "Female"
        while True:
            first = rng.choice(FIRST_M if gender == "Male" else FIRST_F)
            last = rng.choice(LAST)
            if (first, last) not in used:
                used.add((first, last))
                break
        start = datetime(2026, rng.randint(1, 6), 1)
        status = statuses[i]
        end = datetime(2026, 8, 31) if status == "expired" else datetime(start.year + 1, start.month,
            1) - timedelta(days=1)
        city, pin = rng.choice(CITIES)
        docs.append({
            "_id": ObjectId(),
            "member_id": f"MEM{1001 + i}",
            "name": f"{first} {last}",
            "email": f"{first.lower()}.{last.lower()}{i + 1}@example.com",
            "phone": f"9{rng.randint(100000000, 999999999)}",
            "dob": datetime(rng.randint(1975, 2006), rng.randint(1, 12), rng.randint(1, 28)),
            "gender": gender,
            "password_hash": _hash(f"Member@{1001 + i}"),
            "role": "member",
            "membership": {"type": tiers[i], "status": status, "start_date": start, "end_date": end},
            "assigned_trainer_id": rng.choice(trainers)["_id"],
            "referred_by": None,
            "address": {"line1": f"{rng.randint(1, 250)} {rng.choice(['MG Road', 'Gandhi Nagar', 'Anna Salai',
                'Brigade Road', 'Katpadi Main Road'])}",
                        "city": city, "pincode": pin},
            "emergency_contact": {"name": f"{rng.choice(FIRST_F + FIRST_M)} {last}",
                                  "phone": f"9{rng.randint(100000000, 999999999)}"},
            "created_at": start - timedelta(days=rng.randint(1, 15)),
        })
    # Referral network (used by the $graphLookup demo): some members were referred by earlier members
    # Referrers are usually recent joiners, which produces multi-level chains.
    for i in range(4, 60):
        if rng.random() < 0.55:
            docs[i]["referred_by"] = docs[rng.randint(max(0, i - 6), i - 1)]["_id"]
    return docs


def _workout_plans(members, trainers):
    tmap = {t["_id"]: t for t in trainers}
    docs, n = [], 5001
    for m in members:
        plans = [("active", m["membership"]["start_date"])]
        if rng.random() < 0.25:  # some members also have an earlier completed plan
            plans.insert(0, ("completed", m["membership"]["start_date"] - timedelta(days=70)))
        for status, created in plans:
            goal = rng.choice(list(GOALS))
            picks = rng.sample(GOALS[goal], k=min(4, len(GOALS[goal])))
            days = sorted(rng.sample(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"], k=len(picks)),
```

```
                key=DAY_INDEX.get)
            t = tmap[m["assigned_trainer_id"]]
            docs.append({
                "_id": ObjectId(),
                "plan_id": f"WP{n}",
                "member_id": m["_id"], "member_name": m["name"],
                "trainer_id": t["_id"], "trainer_name": t["name"],
                "goal": goal,
                "duration_weeks": rng.choice([6, 8, 10, 12]),
                "status": status,
                "exercises": [{"day": d, "exercise_name": ex, "sets": rng.choice([3, 4, 5]),
                               "reps": rng.choice([8, 10, 12, 15]),
                               "weight_kg": w + rng.choice([0, 5, 10]) if w else 0,
                               "notes": ""} for d, (ex, w) in zip(days, picks)],
                "created_at": created, "updated_at": created,
            })
            n += 1
    return docs


def _classes(trainers, members):
    active = [m for m in members if m["membership"]["status"] == "active"]
    docs = []
    for i, (name, cat, t_idx, cap, sched) in enumerate(CLASSES, start=1):
        t = trainers[t_idx]
        fill = rng.randint(int(cap * 0.55), cap)  # Morning Yoga ends up busier than Pilates, etc.
        enrolled = rng.sample(active, k=min(fill, len(active)))
        docs.append({
            "_id": ObjectId(),
            "class_id": f"CLS{i:02d}", "class_name": name, "category": cat,
            "trainer_id": t["_id"], "trainer_name": t["name"], "capacity": cap,
            "schedule": [{"day_of_week": d, "start_time": s, "end_time": e, "room": r} for d, s, e,
                r in sched],
            "enrolled_members": [{"member_id": m["_id"], "member_name": m["name"],
                                  "enrolled_on": datetime(2026, 7, rng.randint(1, 28))} for m in enrolled],
        })
    return docs


def _sessions(classes):
    """One dated session per schedule slot for 4 weeks: 7 Sep to 3 Oct 2026."""
    docs, n = [], 9001
    week0 = datetime(2026, 9, 7)  # a Monday
    for c in classes:
        for w in range(4):
            for slot in c["schedule"]:
                date = week0 + timedelta(days=7 * w + DAY_INDEX[slot["day_of_week"]])
                if date < REFERENCE_DATE:
                    status = "cancelled" if rng.random() < 0.06 else "completed"
                else:
                    status = "scheduled"
                docs.append({
                    "_id": ObjectId(), "session_id": f"SES{n}",
                    "class_id": c["_id"], "class_name": c["class_name"], "trainer_id": c["trainer_id"],
                    "date": date, "start_time": slot["start_time"], "end_time": slot["end_time"],
                    "status": status,
                })
                n += 1
    return docs


def _attendance(sessions, classes):
    cmap = {c["_id"]: c for c in classes}
    reliability = {}  # each member gets a personal attendance tendency
```

```
    docs = []
    for s in sessions:
        if s["status"] != "completed":
            continue
        h, mi = map(int, s["start_time"].split(":"))
        for e in cmap[s["class_id"]]["enrolled_members"]:
            p = reliability.setdefault(e["member_id"], rng.uniform(0.45, 0.97))
            r = rng.random()
            if r < p:
                status, offset = "present", rng.randint(-10, 2)
            elif r < p + 0.08:
                status, offset = "late", rng.randint(5, 15)
            else:
                status, offset = "absent", None
            check_in = None
            if offset is not None:
                t = s["date"].replace(hour=h, minute=mi) + timedelta(minutes=offset)
                check_in = t.strftime("%H:%M")
            docs.append({
                "_id": ObjectId(), "session_id": s["_id"], "class_id": s["class_id"],
                "member_id": e["member_id"], "member_name": e["member_name"],
                "date": s["date"], "status": status, "check_in_time": check_in,
            })
    return docs


def _payments(members):
    docs, n = [], 7001
    months = [datetime(2026, 7, 1), datetime(2026, 8, 1), datetime(2026, 9, 1), datetime(2026, 10, 1)]
    for m in members:
        fee = TIER_FEES[m["membership"]["type"]]
        for start in months:
            ms = m["membership"]
            if start < ms["start_date"].replace(day=1) or start > ms["end_date"]:
                continue
            if ms["status"] == "frozen" and start >= datetime(2026, 9, 1):
                continue
            end = (start.replace(day=28) + timedelta(days=4)).replace(day=1) - timedelta(days=1)
            due = start.replace(day=5)
            if due > REFERENCE_DATE:
                status, paid_on, mode = "pending", None, None
            elif start.month == 9 and rng.random() < 0.18:
                status, paid_on, mode = "pending", None, None  # past due; the overdue sweep will flag it
            elif start.month == 8 and rng.random() < 0.05:
                status, paid_on, mode = "overdue", None, None
            else:
                status = "paid"
                paid_on = due - timedelta(days=rng.randint(-3, 4))
                mode = rng.choices(["UPI", "card", "cash"], weights=[6, 3, 1])[0]
            docs.append({
                "_id": ObjectId(), "payment_id": f"PAY{n}",
                "member_id": m["_id"], "member_name": m["name"],
                "amount": fee, "currency": "INR", "payment_mode": mode, "status": status,
                "billing_period_start": start, "billing_period_end": end,
                "payment_date": paid_on, "due_date": due,
            })
            n += 1
    return docs


def seed(db):
    trainers = _trainers()
    members = _members(trainers)
    plans = _workout_plans(members, trainers)
```

```
    classes = _classes(trainers, members)
    sessions = _sessions(classes)
    attendance = _attendance(sessions, classes)
    payments = _payments(members)
    data = {"trainers": trainers, "members": members, "workout_plans": plans, "classes": classes,
            "class_sessions": sessions, "attendance": attendance, "payments": payments}
    for name, docs in data.items():
        db[name].insert_many(docs)
    return {k: len(v) for k, v in data.items()}
```

B.3 DML: CRUD Operations

`gym_db/crud.py` contains every insert, read, update and delete used by the system, with the referential integrity rules.

```
"""Step 4: CRUD operations for every collection (FR1 to FR6, FR8).

MongoDB has no foreign keys, so referential integrity is enforced here (NFR5):
  * references are checked before insert/update,
  * denormalized names are propagated when the source changes,
  * deletes either cascade or are blocked when dependent data exists.
"""
from datetime import datetime

import bcrypt
from bson import ObjectId
from pymongo import ReturnDocument

from .config import REFERENCE_DATE, TIER_FEES

SAFE = {"password_hash": 0}  # never return password hashes to callers


class IntegrityError(Exception):
    pass


def _next_code(coll, field, prefix, width):
    last = coll.find_one({field: {"$regex": f"^{prefix}"}}, sort=[(field, -1)], projection={field: 1})
    n = int(last[field][len(prefix):]) + 1 if last else 1
    return f"{prefix}{n:0{width}d}"


def _require(db, coll, _id, label):
    doc = db[coll].find_one({"_id": _id})
    if not doc:
        raise IntegrityError(f"{label} {_id} does not exist")
    return doc


def _member_oid(db, member_id):
    doc = db.members.find_one({"member_id": member_id}, {"_id": 1})
    if not doc:
        raise IntegrityError(f"Member {member_id} not found")
    return doc["_id"]


# ---------------------------------------------------------------- MEMBERS
def create_member(db, name, email, phone, dob, gender, password, tier="Basic",
                  trainer_id=None, referred_by=None, city="Vellore", pincode="632014"):
    trainer_oid = None
    if trainer_id:
        trainer_oid = db.trainers.find_one({"trainer_id": trainer_id}, {"_id": 1})
```

```
        if not trainer_oid:
            raise IntegrityError(f"Trainer {trainer_id} not found")
        trainer_oid = trainer_oid["_id"]
    ref_oid = _member_oid(db, referred_by) if referred_by else None
    start = REFERENCE_DATE.replace(day=1)
    doc = {
        "member_id": _next_code(db.members, "member_id", "MEM", 4),
        "name": name, "email": email.lower(), "phone": phone, "dob": dob, "gender": gender,
        "password_hash": bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode(),
        "role": "member",
        "membership": {"type": tier, "status": "active", "start_date": start,
                       "end_date": datetime(start.year + 1, start.month, 1)},
        "assigned_trainer_id": trainer_oid, "referred_by": ref_oid,
        "address": {"line1": "", "city": city, "pincode": pincode},
        "emergency_contact": {"name": "", "phone": ""},
        "created_at": REFERENCE_DATE,
    }
    db.members.insert_one(doc)
    return doc["member_id"]


def get_member(db, member_id):
    return db.members.find_one({"member_id": member_id}, SAFE)


def list_members(db, tier=None, status=None, city=None, page=1, page_size=10):
    q = {}
    if tier:
        q["membership.type"] = tier
    if status:
        q["membership.status"] = status
    if city:
        q["address.city"] = city
    cur = (db.members.find(q, {"member_id": 1, "name": 1, "membership": 1, "address.city": 1, "_id": 0})
           .sort("member_id", 1).skip((page - 1) * page_size).limit(page_size))
    return list(cur), db.members.count_documents(q)


def update_member(db, member_id, **fields):
    """Update profile fields. A name change is propagated to every denormalized copy."""
    allowed = {"name", "phone", "email", "address", "emergency_contact"}
    bad = set(fields) - allowed
    if bad:
        raise ValueError(f"Cannot update {bad} here")
    doc = db.members.find_one_and_update({"member_id": member_id}, {"$set": fields},
                                         projection=SAFE, return_document=ReturnDocument.AFTER)
    if doc and "name" in fields:
        oid, name = doc["_id"], fields["name"]
        for coll in ("workout_plans", "attendance", "payments"):
            db[coll].update_many({"member_id": oid}, {"$set": {"member_name": name}})
        db.classes.update_many({"enrolled_members.member_id": oid},
                               {"$set": {"enrolled_members.$.member_name": name}})  # positional $
    return doc


def change_membership(db, member_id, tier=None, status=None):
    s = {}
    if tier:
        if tier not in TIER_FEES:
            raise ValueError("tier must be Basic, Premium or VIP")
        s["membership.type"] = tier
    if status:
        s["membership.status"] = status
    return db.members.find_one_and_update({"member_id": member_id}, {"$set": s}, projection=SAFE,
```

```
                                        return_document=ReturnDocument.AFTER)


def assign_trainer(db, member_id, trainer_id):
    t = db.trainers.find_one({"trainer_id": trainer_id}, {"_id": 1})
    if not t:
        raise IntegrityError(f"Trainer {trainer_id} not found")
    return db.members.update_one({"member_id": member_id},
                                 {"$set": {"assigned_trainer_id": t["_id"]}}).modified_count


def delete_member(db, member_id):
    """Blocked if dues are outstanding; otherwise cascades to plans and class rosters.
    Attendance and payment history are kept as the gym's audit trail."""
    oid = _member_oid(db, member_id)
    dues = db.payments.count_documents({"member_id": oid, "status": {"$in": ["pending", "overdue"]}})
    if dues:
        raise IntegrityError(f"{member_id} has {dues} unpaid bill(s); settle before deleting")
    plans = db.workout_plans.delete_many({"member_id": oid}).deleted_count
    rosters = db.classes.update_many({}, {"$pull": {"enrolled_members": {"member_id": oid}}}).modified_count
    db.members.update_many({"referred_by": oid}, {"$set": {"referred_by": None}})
    db.members.delete_one({"_id": oid})
    return {"plans_deleted": plans, "classes_updated": rosters}


# ---------------------------------------------------------------- TRAINERS
def create_trainer(db, name, email, phone, password, specialization, certification, experience_years):
    doc = {"trainer_id": _next_code(db.trainers, "trainer_id", "TRN", 2), "name": name,
           "email": email.lower(), "phone": phone,
           "password_hash": bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode(),
           "role": "trainer", "specialization": specialization, "certification": certification,
           "experience_years": experience_years, "availability": [], "created_at": REFERENCE_DATE}
    db.trainers.insert_one(doc)
    return doc["trainer_id"]


def get_trainer(db, trainer_id):
    return db.trainers.find_one({"trainer_id": trainer_id}, SAFE)


def find_trainers_by_skill(db, skill):
    return list(db.trainers.find({"specialization": skill}, {"_id": 0, "trainer_id": 1, "name": 1,
                                                             "specialization": 1}))


def add_specialization(db, trainer_id, skill):
    # $addToSet: no duplicates in the embedded array
    return db.trainers.update_one({"trainer_id": trainer_id},
                                  {"$addToSet": {"specialization": skill}}).modified_count


def set_availability(db, trainer_id, day, start_time, end_time):
    db.trainers.update_one({"trainer_id": trainer_id}, {"$pull": {"availability": {"day": day}}})
    return db.trainers.update_one({"trainer_id": trainer_id}, {"$push": {"availability": {
        "day": day, "start_time": start_time, "end_time": end_time}}}).modified_count


def delete_trainer(db, trainer_id):
    t = db.trainers.find_one({"trainer_id": trainer_id}, {"_id": 1})
    if not t:
        raise IntegrityError(f"Trainer {trainer_id} not found")
    members = db.members.count_documents({"assigned_trainer_id": t["_id"]})
    classes = db.classes.count_documents({"trainer_id": t["_id"]})
    if members or classes:
```

```
        raise IntegrityError(f"{trainer_id} still has {members} members and {classes} classes; reassign
            first")
    return db.trainers.delete_one({"_id": t["_id"]}).deleted_count


# ---------------------------------------------------------------- WORKOUT PLANS
def create_workout_plan(db, member_id, goal, duration_weeks, exercises):
    m = db.members.find_one({"member_id": member_id})
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    t = _require(db, "trainers", m["assigned_trainer_id"], "Trainer")
    # business rule: only one active plan per member
    db.workout_plans.update_many({"member_id": m["_id"], "status": "active"},
                                 {"$set": {"status": "completed", "updated_at": REFERENCE_DATE}})
    doc = {"plan_id": _next_code(db.workout_plans, "plan_id", "WP", 4),
           "member_id": m["_id"], "member_name": m["name"],
           "trainer_id": t["_id"], "trainer_name": t["name"],
           "goal": goal, "duration_weeks": duration_weeks, "status": "active",
           "exercises": exercises, "created_at": REFERENCE_DATE, "updated_at": REFERENCE_DATE}
    db.workout_plans.insert_one(doc)
    return doc["plan_id"]


def get_active_plan(db, member_id):
    return db.workout_plans.find_one({"member_id": _member_oid(db, member_id), "status": "active"},
                                     {"_id": 0, "member_id": 0, "trainer_id": 0})


def add_exercise(db, plan_id, exercise):
    return db.workout_plans.update_one({"plan_id": plan_id},
                                       {"$push": {"exercises": exercise},
                                        "$set": {"updated_at": REFERENCE_DATE}}).modified_count


def progress_exercise(db, plan_id, exercise_name, add_kg):
    """Progressive overload: increase weight of one embedded exercise (positional $ operator)."""
    return db.workout_plans.update_one(
        {"plan_id": plan_id, "exercises.exercise_name": exercise_name},
        {"$inc": {"exercises.$.weight_kg": add_kg}, "$set": {"updated_at": REFERENCE_DATE}}).modified_count


def remove_exercise(db, plan_id, exercise_name):
    return db.workout_plans.update_one({"plan_id": plan_id},
                                       {"$pull": {"exercises": {"exercise_name":
                                        exercise_name}}}).modified_count


def delete_workout_plan(db, plan_id):
    return db.workout_plans.delete_one({"plan_id": plan_id}).deleted_count


# ---------------------------------------------------------------- CLASSES
def create_class(db, class_name, category, trainer_id, capacity, schedule):
    t = db.trainers.find_one({"trainer_id": trainer_id})
    if not t:
        raise IntegrityError(f"Trainer {trainer_id} not found")
    doc = {"class_id": _next_code(db.classes, "class_id", "CLS", 2), "class_name": class_name,
           "category": category, "trainer_id": t["_id"], "trainer_name": t["name"],
           "capacity": capacity, "schedule": schedule, "enrolled_members": []}
    db.classes.insert_one(doc)
    return doc["class_id"]


def get_class(db, class_id):
```

```
    return db.classes.find_one({"class_id": class_id}, {"_id": 0, "trainer_id": 0})


def enroll_member(db, class_id, member_id):
    """FR4. Atomic single-document update: the filter itself checks capacity and duplicates,
    so two simultaneous requests can never overbook the class."""
    m = db.members.find_one({"member_id": member_id, "membership.status": "active"})
    if not m:
        raise IntegrityError(f"{member_id} not found or membership not active")
    res = db.classes.update_one(
        {"class_id": class_id,
         "enrolled_members.member_id": {"$ne": m["_id"]},
         "$expr": {"$lt": [{"$size": "$enrolled_members"}, "$capacity"]}},
        {"$push": {"enrolled_members": {"member_id": m["_id"], "member_name": m["name"],
                                        "enrolled_on": REFERENCE_DATE}}})
    if res.modified_count == 0:
        raise IntegrityError(f"Cannot enroll {member_id} in {class_id}: class full or already enrolled")
    return True


def withdraw_member(db, class_id, member_id):
    oid = _member_oid(db, member_id)
    return db.classes.update_one({"class_id": class_id},
                                 {"$pull": {"enrolled_members": {"member_id": oid}}}).modified_count


def member_classes(db, member_id):
    oid = _member_oid(db, member_id)
    return list(db.classes.find({"enrolled_members.member_id": oid},
                                {"_id": 0, "class_id": 1, "class_name": 1, "schedule": 1, "trainer_name": 1}))


def delete_class(db, class_id):
    c = db.classes.find_one({"class_id": class_id}, {"_id": 1})
    if not c:
        raise IntegrityError(f"Class {class_id} not found")
    future = db.class_sessions.delete_many({"class_id": c["_id"], "status": "scheduled"}).deleted_count
    db.classes.delete_one({"_id": c["_id"]})
    return {"future_sessions_removed": future}


# ---------------------------------------------------------------- SESSIONS & ATTENDANCE
def create_session(db, class_id, date, start_time, end_time):
    c = db.classes.find_one({"class_id": class_id})
    if not c:
        raise IntegrityError(f"Class {class_id} not found")
    doc = {"session_id": _next_code(db.class_sessions, "session_id", "SES", 4), "class_id": c["_id"],
           "class_name": c["class_name"], "trainer_id": c["trainer_id"], "date": date,
           "start_time": start_time, "end_time": end_time, "status": "scheduled"}
    db.class_sessions.insert_one(doc)
    return doc["session_id"]


def cancel_session(db, session_id):
    return db.class_sessions.update_one({"session_id": session_id, "status": "scheduled"},
                                        {"$set": {"status": "cancelled"}}).modified_count


def mark_attendance(db, session_id, member_id, status, check_in_time=None):
    """FR5. Upsert keyed on (session, member): re-marking corrects the record, never duplicates it."""
    s = db.class_sessions.find_one({"session_id": session_id})
    if not s:
        raise IntegrityError(f"Session {session_id} not found")
    m = db.members.find_one({"member_id": member_id})
```

```
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    return db.attendance.update_one(
        {"session_id": s["_id"], "member_id": m["_id"]},
        {"$set": {"status": status, "check_in_time": check_in_time},
         "$setOnInsert": {"class_id": s["class_id"], "member_name": m["name"], "date": s["date"]}},
        upsert=True)


def attendance_history(db, member_id, limit=5):
    oid = _member_oid(db, member_id)
    return list(db.attendance.find({"member_id": oid}, {"_id": 0, "date": 1, "status": 1,
                                                        "check_in_time": 1, "class_id": 1})
                .sort("date", -1).limit(limit))


def delete_attendance(db, session_id, member_id):
    s = db.class_sessions.find_one({"session_id": session_id}, {"_id": 1})
    return db.attendance.delete_one({"session_id": s["_id"], "member_id": _member_oid(db,
        member_id)}).deleted_count


# ---------------------------------------------------------------- PAYMENTS
def create_invoice(db, member_id, period_start, period_end, due_date):
    m = db.members.find_one({"member_id": member_id})
    if not m:
        raise IntegrityError(f"Member {member_id} not found")
    doc = {"payment_id": _next_code(db.payments, "payment_id", "PAY", 4), "member_id": m["_id"],
           "member_name": m["name"], "amount": TIER_FEES[m["membership"]["type"]], "currency": "INR",
           "payment_mode": None, "status": "pending", "billing_period_start": period_start,
           "billing_period_end": period_end, "payment_date": None, "due_date": due_date}
    db.payments.insert_one(doc)
    return doc["payment_id"]


def record_payment(db, payment_id, mode, paid_on=None):
    res = db.payments.update_one(
        {"payment_id": payment_id, "status": {"$in": ["pending", "overdue"]}},  # paid bills are immutable
        {"$set": {"status": "paid", "payment_mode": mode, "payment_date": paid_on or REFERENCE_DATE}})
    if not res.modified_count:
        return None
    return db.payments.find_one({"payment_id": payment_id},
                                {"_id": 0, "payment_id": 1, "status": 1, "amount": 1, "payment_mode": 1})


def flag_overdue(db, today=REFERENCE_DATE):
    """FR6. Nightly sweep: any pending bill past its due date becomes overdue (uses ix_due_date)."""
    return db.payments.update_many({"status": "pending", "due_date": {"$lt": today}},
                                   {"$set": {"status": "overdue"}}).modified_count


def member_dues(db, member_id):
    oid = _member_oid(db, member_id)
    return list(db.payments.find({"member_id": oid, "status": {"$ne": "paid"}},
                                 {"_id": 0, "payment_id": 1, "amount": 1, "status": 1, "due_date": 1}))


def delete_payment(db, payment_id):
    # only unpaid invoices raised in error may be deleted; paid records are immutable
    return db.payments.delete_one({"payment_id": payment_id, "status": "pending"}).deleted_count


# ---------------------------------------------------------------- AUTH (FR8)
def authenticate(db, email, password):
```

```
    for coll in ("members", "trainers"):
        u = db[coll].find_one({"email": email.lower()})
        if u and bcrypt.checkpw(password.encode(), u["password_hash"].encode()):
            return {"role": u["role"], "id": u.get("member_id") or u.get("trainer_id"), "name": u["name"]}
    return None
```

Appendix C: Stored Procedures, Functions and Triggers (Full Code)

MongoDB does not have stored procedures in the SQL sense. Their role is played by three mechanisms, all included here in full: (1) server-side **aggregation pipelines and the view**, which run entirely inside the database (`aggregations.py`); (2) **multi-document transactions**, the equivalent of a stored procedure wrapped in BEGIN and COMMIT (`transactions.py`); and (3) **triggers implemented with change streams** [28], the supported equivalent of CREATE TRIGGER (`triggers.py`).

C.1 Aggregation Pipelines and View (aggregations.py)

```
"""Step 5: advanced NoSQL features, aggregation framework (FR7).

Each function returns the pipeline result. Stages used across the set:
$match $group $project $addFields $lookup $unwind $sort $limit $facet $bucket
$graphLookup $cond $size $round $dateToString $filter $sum $avg $push $first
"""
from .config import REFERENCE_DATE, is_mock


# A1. Monthly revenue trend, split by membership tier ($lookup + $group + $dateToString)
def revenue_trend(db):
    return list(db.payments.aggregate([
        {"$match": {"status": "paid"}},
        {"$lookup": {"from": "members", "localField": "member_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$group": {
            "_id": {"month": {"$dateToString": {"format": "%Y-%m", "date": "$billing_period_start"}},
                    "tier": "$m.membership.type"},
            "revenue": {"$sum": "$amount"}, "payments": {"$sum": 1}}},
        {"$group": {"_id": "$_id.month",
                    "total_revenue": {"$sum": "$revenue"},
                    "payments": {"$sum": "$payments"},
                    "by_tier": {"$push": {"tier": "$_id.tier", "revenue": "$revenue"}}}},
        {"$sort": {"_id": 1}},
        {"$project": {"_id": 0, "month": "$_id", "total_revenue": 1, "payments": 1, "by_tier": 1}},
    ]))


# A2. Class popularity: attendance rate, average turnout and fill rate per class
def class_popularity(db):
    return list(db.attendance.aggregate([
        {"$group": {"_id": "$class_id",
                    "marked": {"$sum": 1},
                    "attended": {"$sum": {"$cond": [{"$in": ["$status", ["present", "late"]]}, 1, 0]}},
                    "sessions": {"$addToSet": "$session_id"}}},
        {"$lookup": {"from": "classes", "localField": "_id", "foreignField": "_id", "as": "c"}},
        {"$unwind": "$c"},
        {"$project": {
            "_id": 0, "class_name": "$c.class_name", "trainer": "$c.trainer_name",
            "enrolled": {"$size": "$c.enrolled_members"}, "capacity": "$c.capacity",
            "sessions_held": {"$size": "$sessions"},
            "avg_turnout": {"$round": [{"$divide": ["$attended", {"$size": "$sessions"}]}, 1]},
            "attendance_rate_pct": {"$round": [{"$multiply": [{"$divide": ["$attended", "$marked"]}, 100]},
                1]},
            "fill_rate_pct": {"$round": [{"$multiply": [
                {"$divide": [{"$size": "$c.enrolled_members"}, "$c.capacity"]}, 100]}, 1]}}},
```

```
        {"$sort": {"avg_turnout": -1, "class_name": 1}},
    ]))


# A3. Members at risk of churning: attendance below 60 percent
def at_risk_members(db, threshold=60):
    return list(db.attendance.aggregate([
        {"$group": {"_id": "$member_id", "name": {"$first": "$member_name"},
                    "total": {"$sum": 1},
                    "attended": {"$sum": {"$cond": [{"$eq": ["$status", "absent"]}, 0, 1]}}}},
        {"$addFields": {"rate_pct": {"$round": [{"$multiply": [{"$divide": ["$attended", "$total"]}, 100]},
            1]}}},
        {"$match": {"rate_pct": {"$lt": threshold}, "total": {"$gte": 4}}},
        {"$lookup": {"from": "members", "localField": "_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$project": {"_id": 0, "member_id": "$m.member_id", "name": 1, "tier": "$m.membership.type",
                      "sessions_marked": "$total", "attended": 1, "rate_pct": 1}},
        {"$sort": {"rate_pct": 1, "member_id": 1}},
    ]))


# A4. Outstanding dues with days overdue and contact details (FR6)
def outstanding_dues(db, today=REFERENCE_DATE):
    return list(db.payments.aggregate([
        {"$match": {"status": {"$in": ["pending", "overdue"]}}},
        {"$addFields": {"days_overdue": {"$max": [0, {"$floor": {
            "$divide": [{"$subtract": [today, "$due_date"]}, 86400000]}}]}}},
        {"$group": {"_id": "$member_id", "name": {"$first": "$member_name"},
                    "bills": {"$sum": 1}, "amount_due": {"$sum": "$amount"},
                    "max_days_overdue": {"$max": "$days_overdue"},
                    "statuses": {"$addToSet": "$status"}}},
        {"$lookup": {"from": "members", "localField": "_id", "foreignField": "_id", "as": "m"}},
        {"$unwind": "$m"},
        {"$project": {"_id": 0, "member_id": "$m.member_id", "name": 1, "phone": "$m.phone",
                      "tier": "$m.membership.type", "bills": 1, "amount_due": 1, "max_days_overdue": 1,
                      "priority": {"$cond": [{"$gte": ["$max_days_overdue", 15]}, "HIGH",
                                             {"$cond": [{"$gt": ["$max_days_overdue", 0]}, "MEDIUM",
                                        "LOW"]}]}}},
        {"$sort": {"max_days_overdue": -1, "amount_due": -1, "member_id": 1}},
    ]))


# A5. Trainer workload: members, active plans, classes (three $lookup joins)
def trainer_workload(db):
    return list(db.trainers.aggregate([
        {"$lookup": {"from": "members", "localField": "_id", "foreignField": "assigned_trainer_id",
            "as": "mem"}},
        {"$lookup": {"from": "workout_plans", "localField": "_id", "foreignField": "trainer_id",
            "as": "plans"}},
        {"$lookup": {"from": "classes", "localField": "_id", "foreignField": "trainer_id", "as": "cls"}},
        {"$project": {"_id": 0, "trainer_id": 1, "name": 1, "specialization": 1,
                      "members": {"$size": "$mem"},
                      "active_plans": {"$size": {"$filter": {"input": "$plans", "as": "p",
                                                             "cond": {"$eq": ["$$p.status", "active"]}}}},
                      "classes": {"$size": "$cls"},
                      "class_names": "$cls.class_name"}},
        {"$sort": {"members": -1, "trainer_id": 1}},
    ]))


# A6. Membership dashboard in ONE query: $facet runs three sub-pipelines, $bucket groups ages
def membership_overview(db, today=REFERENCE_DATE):
    res = list(db.members.aggregate([
        {"$addFields": {"age": {"$floor": {"$divide": [{"$subtract": [today, "$dob"]}, 31557600000]}}}},
```

```
        {"$facet": {
            "by_tier": [{"$group": {"_id": "$membership.type", "count": {"$sum": 1}}}, {"$sort": {"_id": 1}}],
            "by_status": [{"$group": {"_id": "$membership.status", "count": {"$sum": 1}}},
                {"$sort": {"_id": 1}}],
            "age_groups": [{"$bucket": {"groupBy": "$age", "boundaries": [18, 25, 35, 45, 60],
                                        "default": "60+", "output": {"count": {"$sum": 1}}}}],
            "by_city": [{"$group": {"_id": "$address.city", "count": {"$sum": 1}}}, {"$sort": {"count": -1}}],
        }},
    ]))
    return res[0]


# A7. Most prescribed exercises across all active plans ($unwind on embedded array)
def top_exercises(db, limit=8):
    return list(db.workout_plans.aggregate([
        {"$match": {"status": "active"}},
        {"$unwind": "$exercises"},
        {"$group": {"_id": "$exercises.exercise_name", "plans": {"$sum": 1},
                    "avg_sets": {"$avg": "$exercises.sets"},
                        "avg_weight_kg": {"$avg": "$exercises.weight_kg"}}},
        {"$project": {"_id": 0, "exercise": "$_id", "plans": 1,
                      "avg_sets": {"$round": ["$avg_sets", 1]}, "avg_weight_kg": {"$round": ["$avg_weight_kg",
                          1]}}},
        {"$sort": {"plans": -1, "exercise": 1}},
        {"$limit": limit},
    ]))


# A8. Peak check-in hours (string operators on check_in_time)
def peak_hours(db):
    return list(db.attendance.aggregate([
        {"$match": {"status": {"$ne": "absent"}}},
        {"$group": {"_id": {"$substr": ["$check_in_time", 0, 2]}, "check_ins": {"$sum": 1}}},
        {"$project": {"_id": 0, "hour": {"$concat": ["$_id", ":00"]}, "check_ins": 1}},
        {"$sort": {"check_ins": -1, "hour": 1}},
    ]))


# G1. Graph traversal: full referral network of a member ($graphLookup, recursive)
def referral_network(db, member_id):
    res = list(db.members.aggregate([
        {"$match": {"member_id": member_id}},
        {"$graphLookup": {"from": "members", "startWith": "$_id", "connectFromField": "_id",
                          "connectToField": "referred_by", "as": "network",
                          "maxDepth": 5, "depthField": "level"}},
        {"$project": {"_id": 0, "member_id": 1, "name": 1,
                      "network": {"$map": {"input": "$network", "as": "n",
                                           "in": {"member_id": "$$n.member_id", "name": "$$n.name",
                                                  "level": {"$add": ["$$n.level", 1]}}}}}},
    ]))
    if res:
        res[0]["network"].sort(key=lambda n: (n["level"], n["member_id"]))
        res[0]["network_size"] = len(res[0]["network"])
    return res[0] if res else None


# G2. Top referrers: who brought in the largest downstream network
def top_referrers(db, limit=5):
    return list(db.members.aggregate([
        {"$match": {"referred_by": None}},
        {"$graphLookup": {"from": "members", "startWith": "$_id", "connectFromField": "_id",
                          "connectToField": "referred_by", "as": "network", "depthField": "level"}},
        {"$project": {"_id": 0, "member_id": 1, "name": 1, "network_size": {"$size": "$network"},
                      "direct": {"$size": {"$filter": {"input": "$network", "as": "n",
```

```
                                                    "cond": {"$eq": ["$$n.level", 0]}}}},
                    "levels": {"$add": [{"$max": "$network.level"}, 1]}}},
        {"$match": {"network_size": {"$gt": 0}}},
        {"$sort": {"network_size": -1, "member_id": 1}},
        {"$limit": limit},
    ]))


# V1. Read-only view powering the member dashboard (FR3); hides password_hash
MEMBER_DASHBOARD_PIPELINE = [
    {"$lookup": {"from": "trainers", "localField": "assigned_trainer_id", "foreignField": "_id", "as": "t"}},
    {"$lookup": {"from": "payments", "localField": "_id", "foreignField": "member_id", "as": "p"}},
    {"$project": {"_id": 0, "member_id": 1, "name": 1, "tier": "$membership.type",
                  "status": "$membership.status", "trainer": {"$first": "$t.name"},
                  "unpaid_amount": {"$sum": {"$map": {
                      "input": {"$filter": {"input": "$p", "as": "x", "cond": {"$ne": ["$$x.status",
                          "paid"]}}},
                      "as": "u", "in": "$$u.amount"}}}}},
]


def create_views(db):
    if is_mock():
        return False
    db.drop_collection("member_dashboard")
    db.command({"create": "member_dashboard", "viewOn": "members", "pipeline": MEMBER_DASHBOARD_PIPELINE})
    return True


def member_dashboard(db, member_id):
    if is_mock():  # mongomock has no views; run the same pipeline directly
        res = list(db.members.aggregate([{"$match": {"member_id": member_id}}] + MEMBER_DASHBOARD_PIPELINE))
        return res[0] if res else None
    return db.member_dashboard.find_one({"member_id": member_id})
```

C.2 Transactions (transactions.py)

```
"""Step 6: multi-document ACID transactions.

Transactions need a replica set (a single-node replica set is enough; see README).
Two business operations touch several collections and must be all-or-nothing:

T1 renew_membership: insert a paid payment AND extend the member's end date.
T2 close_session:    insert attendance rows for a session AND mark it completed.
"""
from datetime import timedelta

from pymongo import ReadPreference
from pymongo.read_concern import ReadConcern
from pymongo.write_concern import WriteConcern

from .config import REFERENCE_DATE, TIER_FEES, get_client, is_mock


class TransactionAborted(Exception):
    pass


def _run(db, fn):
    """Run fn(session) inside a transaction, with automatic retry on transient errors.
    Under mongomock (no transaction support) fn runs without a session."""
    if is_mock():
        return fn(None)
    client = get_client()
```

```
    with client.start_session() as s:
        return s.with_transaction(
            fn,
            read_concern=ReadConcern("snapshot"),
            write_concern=WriteConcern("majority"),
            read_preference=ReadPreference.PRIMARY,
        )


def renew_membership(db, member_id, months, mode="UPI", fail_midway=False):
    """T1. fail_midway=True raises after the payment insert to prove the rollback."""
    def txn(s):
        m = db.members.find_one({"member_id": member_id}, session=s)
        if not m:
            raise TransactionAborted(f"{member_id} not found")
        fee = TIER_FEES[m["membership"]["type"]] * months
        old_end = m["membership"]["end_date"]
        start = max(old_end, REFERENCE_DATE) + timedelta(days=1)
        new_end = start + timedelta(days=30 * months)
        last = db.payments.find_one({}, sort=[("payment_id", -1)], projection={"payment_id": 1}, session=s)
        pid = f"PAY{int(last['payment_id'][3:]) + 1}"
        db.payments.insert_one({
            "payment_id": pid, "member_id": m["_id"], "member_name": m["name"], "amount": fee,
            "currency": "INR", "payment_mode": mode, "status": "paid",
            "billing_period_start": start, "billing_period_end": new_end,
            "payment_date": REFERENCE_DATE, "due_date": REFERENCE_DATE}, session=s)
        if fail_midway:
            raise TransactionAborted("Simulated failure after payment insert")
        db.members.update_one({"_id": m["_id"]},
                              {"$set": {"membership.end_date": new_end, "membership.status": "active"}},
                              session=s)
        return {"payment_id": pid, "amount": fee, "new_end_date": new_end.date().isoformat()}
    return _run(db, txn)


def close_session(db, session_id, marks):
    """T2. marks = {member_id: (status, check_in_time)}. Everything commits together."""
    def txn(s):
        sess = db.class_sessions.find_one({"session_id": session_id, "status": "scheduled"}, session=s)
        if not sess:
            raise TransactionAborted(f"{session_id} is not an open session")
        cls = db.classes.find_one({"_id": sess["class_id"]}, session=s)
        roster = {e["member_id"]: e["member_name"] for e in cls["enrolled_members"]}
        ids = {d["member_id"]: d["_id"] for d in
               db.members.find({"member_id": {"$in": list(marks)}}, {"member_id": 1}, session=s)}
        db.class_sessions.update_one({"_id": sess["_id"]}, {"$set": {"status": "completed"}}, session=s)
        rows = 0
        for mid, (status, t) in marks.items():
            oid = ids.get(mid)
            if oid not in roster:  # fails AFTER earlier writes: the rollback undoes them all
                raise TransactionAborted(f"{mid} is not enrolled in {cls['class_name']}")
            db.attendance.insert_one({"session_id": sess["_id"], "class_id": sess["class_id"],
                                      "member_id": oid, "member_name": roster[oid], "date": sess["date"],
                                      "status": status, "check_in_time": t}, session=s)
            rows += 1
        return {"session_id": session_id, "attendance_rows": rows}
    return _run(db, txn)
```

C.3 Triggers (triggers.py)

```
"""Database triggers implemented with MongoDB change streams.

MongoDB has no CREATE TRIGGER statement. The supported equivalent is a change stream:
the server pushes every insert, update, replace and delete on a collection to a listening
```

```
client, which reacts to it. Change streams need a replica set, which this project uses.

Trigger TR1 (audit trail): every change to members and payments is written to the
audit_log collection with the operation, the business key, the fields that changed and
the time. This gives the administrator a tamper-evident history of who paid what and
when a membership changed (NFR3 security, NFR5 consistency).

Trigger TR2 (membership reactivation): when a payment changes to status "paid" for a
member whose membership is "expired", the member is set back to "active".

    python -m gym_db.triggers            # run the triggers until Ctrl+C
"""
import threading
from datetime import datetime, timezone

WATCHED = ["members", "payments"]


def _business_key(db, coll, event):
    doc = event.get("fullDocument") or {}
    key = doc.get("member_id") if coll == "members" else doc.get("payment_id")
    if key is None and coll == "payments" and isinstance(doc.get("member_id"), str):
        key = doc["member_id"]
    return key or str(event["documentKey"]["_id"])


def handle_event(db, event):
    """Apply both triggers to one change event."""
    coll = event["ns"]["coll"]
    op = event["operationType"]
    changed = None
    if op == "update":
        desc = event.get("updateDescription", {})
        changed = sorted(list(desc.get("updatedFields", {}).keys()) + desc.get("removedFields", []))
    # TR1 audit trail
    db.audit_log.insert_one({
        "collection": coll, "operation": op, "key": _business_key(db, coll, event),
        "changed_fields": changed, "at": datetime.now(timezone.utc),
    })
    # TR2 reactivate an expired membership once a bill is paid
    doc = event.get("fullDocument") or {}
    if coll == "payments" and op in ("update", "replace") and doc.get("status") == "paid":
        res = db.members.update_one({"_id": doc["member_id"], "membership.status": "expired"},
                                    {"$set": {"membership.status": "active"}})
        if res.modified_count:
            member = db.members.find_one({"_id": doc["member_id"]}, {"member_id": 1})
            db.audit_log.insert_one({"collection": "members", "operation": "trigger:reactivate",
                                     "key": member["member_id"], "changed_fields": ["membership.status"],
                                     "at": datetime.now(timezone.utc)})


def watch(db, stop_event=None, ready_event=None):
    """Listen to members and payments until stop_event is set."""
    pipeline = [{"$match": {"ns.coll": {"$in": WATCHED}}}]
    with db.watch(pipeline, full_document="updateLookup", max_await_time_ms=200) as stream:
        if ready_event:
            ready_event.set()
        while stop_event is None or not stop_event.is_set():
            event = stream.try_next()
            if event is not None:
                handle_event(db, event)


def start_in_background(db):
```

```
    """Start the triggers on a daemon thread; returns the stop event."""
    stop, ready = threading.Event(), threading.Event()
    threading.Thread(target=watch, args=(db, stop, ready), daemon=True).start()
    ready.wait(5)
    return stop


if __name__ == "__main__":
    from gym_db.config import get_db
    print("Triggers TR1 (audit trail) and TR2 (reactivation) are running. Press Ctrl+C to stop.")
    try:
        watch(get_db())
    except KeyboardInterrupt:
        pass
```

C.4 Index Creation and Explain (indexes.py)

```
"""Step 3: indexes (NFR1 performance) and an explain() comparison.

Implements every index promised in Review 1 section 6.8, plus a text index for class
search and a partial index that only covers unpaid dues.
"""
from pymongo import ASCENDING, DESCENDING, TEXT

from .config import is_mock

INDEXES = {
    "members": [
        ([("email", ASCENDING)], {"unique": True, "name": "uq_email"}),
        ([("member_id", ASCENDING)], {"unique": True, "name": "uq_member_id"}),
        ([("assigned_trainer_id", ASCENDING)], {"name": "ix_trainer"}),
        ([("membership.type", ASCENDING), ("membership.status", ASCENDING)], {"name": "ix_tier_status"}),
        ([("referred_by", ASCENDING)], {"name": "ix_referred_by"}),
    ],
    "trainers": [
        ([("email", ASCENDING)], {"unique": True, "name": "uq_email"}),
        ([("trainer_id", ASCENDING)], {"unique": True, "name": "uq_trainer_id"}),
    ],
    "workout_plans": [
        ([("member_id", ASCENDING), ("status", ASCENDING)], {"name": "ix_member_status"}),
        ([("trainer_id", ASCENDING)], {"name": "ix_trainer"}),
    ],
    "classes": [
        ([("trainer_id", ASCENDING)], {"name": "ix_trainer"}),
        ([("enrolled_members.member_id", ASCENDING)], {"name": "ix_enrolled_member"}),  # multikey
        ([("class_name", TEXT), ("category", TEXT)], {"name": "tx_class_search"}),
    ],
    "class_sessions": [
        ([("class_id", ASCENDING), ("date", ASCENDING)], {"name": "ix_class_date"}),
    ],
    "attendance": [
        ([("member_id", ASCENDING), ("date", DESCENDING)], {"name": "ix_member_date"}),
        ([("session_id", ASCENDING), ("member_id", ASCENDING)], {"unique": True,
            "name": "uq_session_member"}),
        ([("class_id", ASCENDING), ("status", ASCENDING)], {"name": "ix_class_status"}),
    ],
    "payments": [
        ([("member_id", ASCENDING), ("status", ASCENDING)], {"name": "ix_member_status"}),
        ([("due_date", ASCENDING)], {"name": "ix_due_date"}),
        ([("status", ASCENDING), ("due_date", ASCENDING)],
         {"name": "px_unpaid_dues", "partialFilterExpression": {"status": {"$in": ["pending", "overdue"]}}}),
    ],
}
```

```
def create_indexes(db):
    created = []
    for coll, specs in INDEXES.items():
        for keys, opts in specs:
            if is_mock() and ("partialFilterExpression" in opts or keys[0][1] == TEXT):
                continue  # mongomock does not support partial or text indexes
            db[coll].create_index(keys, **opts)
            created.append(f"{coll}.{opts['name']}")
    return created


def explain_comparison(db):
    """Run the same query with and without an index and compare the query plans.

    Query: a member's attendance history, newest first (dashboard query).
    """
    if is_mock():
        return None
    member_id = db.members.find_one({"member_id": "MEM1001"})["_id"]
    query = {"member_id": member_id}

    def stats(hint):
        plan = db.attendance.find(query).sort("date", -1).hint(hint).explain()
        es = plan["executionStats"]
        stage = plan["queryPlanner"]["winningPlan"]
        stage = stage.get("queryPlan", stage)  # MongoDB 7+ (slot-based engine) nests the plan
        # walk down to the leaf stage (COLLSCAN or IXSCAN)
        while "inputStage" in stage:
            stage = stage["inputStage"]
        return {"stage": stage["stage"], "docs_examined": es["totalDocsExamined"],
                "keys_examined": es["totalKeysExamined"], "returned": es["nReturned"],
                "time_ms": es["executionTimeMillis"]}

    return {"without_index": stats([("$natural", 1)]), "with_index": stats("ix_member_date")}
```

Appendix D: Complete Query Set with Outputs

D.1 Query Set for mongosh (queries.js)

```
// Generated by queries.py --export-js. Run after the database has been created and seeded:
//   python demo.py --section setup
//   mongosh "mongodb://localhost:27017/?directConnection=true" mongosh/queries.js

db = db.getSiblingDB("gym_db");

// Q1  Selection: Active VIP members living in Chennai
// SQL equivalent:
//   SELECT member_id, name, end_date FROM members
//   WHERE tier = 'VIP' AND status = 'active' AND city = 'Chennai'
//   ORDER BY member_id;
print("\n--- Q1 Active VIP members living in Chennai");
printjson(db.members.find(
  { "membership.type": "VIP", "membership.status": "active", "address.city": "Chennai" },
  { _id: 0, member_id: 1, name: 1, "membership.end_date": 1 }
).sort({ member_id: 1 }).toArray());

// Q2  Selection: Trainers who teach Yoga or CrossFit with at least 6 years of experience
// SQL equivalent:
//   SELECT DISTINCT t.trainer_id, t.name, t.experience_years
//   FROM trainers t JOIN trainer_specializations s ON s.trainer_id = t.trainer_id
//   WHERE s.skill IN ('Yoga', 'CrossFit') AND t.experience_years >= 6
//   ORDER BY t.experience_years DESC;
print("\n--- Q2 Trainers who teach Yoga or CrossFit with at least 6 years of experience");
printjson(db.trainers.find(
  { specialization: { "$in": ["Yoga", "CrossFit"] }, experience_years: { "$gte": 6 } },
  { _id: 0, trainer_id: 1, name: 1, specialization: 1, experience_years: 1 }
).sort({ experience_years: -1 }).toArray());

// Q3  Selection: Unpaid bills already past their due date (oldest first)
// SQL equivalent:
//   SELECT payment_id, member_name, amount, status, due_date FROM payments
//   WHERE status IN ('pending', 'overdue') AND due_date < DATE '2026-09-24'
//   ORDER BY due_date, payment_id LIMIT 8;
print("\n--- Q3 Unpaid bills already past their due date (oldest first)");
printjson(db.payments.find(
  {
    status: { "$in": ["pending", "overdue"] },
    due_date: { "$lt": ISODate("2026-09-24T00:00:00Z") }
  },
  { _id: 0, payment_id: 1, member_name: 1, amount: 1, status: 1, due_date: 1 }
).sort({ due_date: 1, payment_id: 1 }).limit(8).toArray());

// Q4  Join (inner): Active VIP members with their assigned trainer
// SQL equivalent:
//   SELECT m.member_id, m.name, t.name AS trainer, t.certification
//   FROM members m INNER JOIN trainers t ON t.trainer_pk = m.assigned_trainer_id
//   WHERE m.tier = 'VIP' AND m.status = 'active'
//   ORDER BY m.member_id LIMIT 8;
print("\n--- Q4 Active VIP members with their assigned trainer");
printjson(db.members.aggregate([
  { "$match": { "membership.type": "VIP", "membership.status": "active" } },
  {
    "$lookup": { from: "trainers", localField: "assigned_trainer_id", foreignField: "_id", as: "t" }
  },
  { "$unwind": "$t" },
```

```
  {
    "$project": { _id: 0, member_id: 1, name: 1, trainer: "$t.name", certification: "$t.certification" }
  },
  { "$sort": { member_id: 1 } },
  { "$limit": 8 }
]).toArray());

// Q5  Join (three tables): Attendance history of MEM1001 with class name and session time
// SQL equivalent:
//   SELECT s.date, c.class_name, s.start_time, a.status, a.check_in_time
//   FROM members m
//   JOIN attendance a ON a.member_id = m.member_pk
//   JOIN class_sessions s ON s.session_pk = a.session_id
//   JOIN classes c ON c.class_pk = a.class_id
//   WHERE m.member_id = 'MEM1001' ORDER BY s.date DESC;
print("\n--- Q5 Attendance history of MEM1001 with class name and session time");
printjson(db.members.aggregate([
  { "$match": { member_id: "MEM1001" } },
  { "$lookup": { from: "attendance", localField: "_id", foreignField: "member_id", as: "a" } },
  { "$unwind": "$a" },
  {
    "$lookup": { from: "class_sessions", localField: "a.session_id", foreignField: "_id", as: "s" }
  },
  { "$unwind": "$s" },
  { "$lookup": { from: "classes", localField: "a.class_id", foreignField: "_id", as: "c" } },
  { "$unwind": "$c" },
  {
    "$project": {
      _id: 0,
      date: "$s.date",
      class_name: "$c.class_name",
      start_time: "$s.start_time",
      status: "$a.status",
      check_in_time: "$a.check_in_time"
    }
  },
  { "$sort": { date: -1 } }
]).toArray());

// Q6  Join (left outer): Every trainer with the number of classes they teach, including trainers with none
// SQL equivalent:
//   SELECT t.trainer_id, t.name, COUNT(c.class_id) AS classes
//   FROM trainers t LEFT OUTER JOIN classes c ON c.trainer_id = t.trainer_pk
//   GROUP BY t.trainer_id, t.name ORDER BY classes DESC, t.trainer_id;
print("\n--- Q6 Every trainer with the number of classes they teach, including trainers with none");
printjson(db.trainers.aggregate([
  { "$lookup": { from: "classes", localField: "_id", foreignField: "trainer_id", as: "c" } },
  {
    "$project": { _id: 0, trainer_id: 1, name: 1, classes: { "$size": "$c" }, class_names: "$c.class_name" }
  },
  { "$sort": { classes: -1, trainer_id: 1 } }
]).toArray());

// Q7  Grouping and aggregation: Money collected by payment mode (COUNT, SUM, AVG)
// SQL equivalent:
//   SELECT payment_mode, COUNT(*) AS payments, SUM(amount) AS total, AVG(amount) AS average
//   FROM payments WHERE status = 'paid'
//   GROUP BY payment_mode ORDER BY total DESC;
print("\n--- Q7 Money collected by payment mode (COUNT, SUM, AVG)");
printjson(db.payments.aggregate([
  { "$match": { status: "paid" } },
  {
    "$group": {
      _id: "$payment_mode",
```

```
      payments: { "$sum": 1 },
      total: { "$sum": "$amount" },
      average: { "$avg": "$amount" }
    }
  },
  {
    "$project": {
      _id: 0,
      payment_mode: "$_id",
      payments: "$payments",
      total: "$total",
      average: { "$round": ["$average", 0] }
    }
  },
  { "$sort": { total: -1 } }
]).toArray());

// Q8  Grouping with HAVING: Cities with more than 10 members, split by tier
// SQL equivalent:
//   SELECT city, COUNT(*) AS members,
//          SUM(CASE WHEN tier = 'VIP' THEN 1 ELSE 0 END) AS vip
//   FROM members GROUP BY city HAVING COUNT(*) > 10 ORDER BY members DESC;
print("\n--- Q8 Cities with more than 10 members, split by tier");
printjson(db.members.aggregate([
  {
    "$group": {
      _id: "$address.city",
      members: { "$sum": 1 },
      basic: { "$sum": { "$cond": [{ "$eq": ["$membership.type", "Basic"] }, 1, 0] } },
      premium: { "$sum": { "$cond": [{ "$eq": ["$membership.type", "Premium"] }, 1, 0] } },
      vip: { "$sum": { "$cond": [{ "$eq": ["$membership.type", "VIP"] }, 1, 0] } }
    }
  },
  { "$match": { members: { "$gt": 10 } } },
  {
    "$project": {
      _id: 0,
      city: "$_id",
      members: "$members",
      basic: "$basic",
      premium: "$premium",
      vip: "$vip"
    }
  },
  { "$sort": { members: -1 } }
]).toArray());

// Q9  Grouping and aggregation: Attendance outcome by class category
// SQL equivalent:
//   SELECT c.category, COUNT(*) AS marked,
//          SUM(CASE WHEN a.status = 'present' THEN 1 ELSE 0 END) AS present,
//          SUM(CASE WHEN a.status = 'late' THEN 1 ELSE 0 END) AS late,
//          SUM(CASE WHEN a.status = 'absent' THEN 1 ELSE 0 END) AS absent
//   FROM attendance a JOIN classes c ON c.class_pk = a.class_id
//   GROUP BY c.category ORDER BY marked DESC;
print("\n--- Q9 Attendance outcome by class category");
printjson(db.attendance.aggregate([
  { "$lookup": { from: "classes", localField: "class_id", foreignField: "_id", as: "c" } },
  { "$unwind": "$c" },
  {
    "$group": {
      _id: "$c.category",
      marked: { "$sum": 1 },
      present: { "$sum": { "$cond": [{ "$eq": ["$status", "present"] }, 1, 0] } },
```

```
      late: { "$sum": { "$cond": [{ "$eq": ["$status", "late"] }, 1, 0] } },
      absent: { "$sum": { "$cond": [{ "$eq": ["$status", "absent"] }, 1, 0] } }
    }
  },
  {
    "$project": {
      _id: 0,
      category: "$_id",
      marked: "$marked",
      present: "$present",
      late: "$late",
      absent: "$absent"
    }
  },
  { "$sort": { marked: -1 } }
]).toArray());

// Q10  Subquery (scalar): Members whose attendance rate is below the gym-wide average
// SQL equivalent:
//   SELECT member_id, AVG(CASE WHEN status <> 'absent' THEN 100 ELSE 0 END) AS rate
//   FROM attendance GROUP BY member_id
//   HAVING rate < (SELECT AVG(CASE WHEN status <> 'absent' THEN 100 ELSE 0 END)
//                  FROM attendance)
//   ORDER BY rate LIMIT 8;
print("\n--- Q10 Members whose attendance rate is below the gym-wide average");
printjson(db.attendance.aggregate([
  { "$addFields": { attended: { "$cond": [{ "$eq": ["$status", "absent"] }, 0, 100] } } },
  {
    "$facet": {
      overall: [{ "$group": { _id: null, avg: { "$avg": "$attended" } } }],
      per_member: [
        {
          "$group": { _id: "$member_name", rate: { "$avg": "$attended" }, marked: { "$sum": 1 } }
        }
      ]
    }
  },
  { "$unwind": "$overall" },
  { "$unwind": "$per_member" },
  { "$match": { "$expr": { "$lt": ["$per_member.rate", "$overall.avg"] } } },
  {
    "$project": {
      name: "$per_member._id",
      marked: "$per_member.marked",
      rate_pct: { "$round": ["$per_member.rate", 1] },
      gym_average_pct: { "$round": ["$overall.avg", 1] }
    }
  },
  { "$project": { _id: 0 } },
  { "$sort": { rate_pct: 1, name: 1 } },
  { "$limit": 8 }
]).toArray());

// Q11  Subquery (correlated): Members with more than one unpaid bill
// SQL equivalent:
//   SELECT m.member_id, m.name,
//          (SELECT COUNT(*) FROM payments p
//           WHERE p.member_id = m.member_pk AND p.status IN ('pending','overdue')) AS unpaid
//   FROM members m
//   WHERE (SELECT COUNT(*) FROM payments p
//          WHERE p.member_id = m.member_pk AND p.status IN ('pending','overdue')) > 1
//   ORDER BY m.member_id;
print("\n--- Q11 Members with more than one unpaid bill");
printjson(db.members.aggregate([
```

```
  {
    "$lookup": {
      from: "payments",
      let: { mid: "$_id" },
      as: "unpaid",
      pipeline: [
        {
          "$match": {
            "$expr": {
              "$and": [
                { "$eq": ["$member_id", "$$mid"] },
                { "$in": ["$status", ["pending", "overdue"]] }
              ]
            }
          }
        }
      ]
    }
  },
  { "$match": { "$expr": { "$gt": [{ "$size": "$unpaid" }, 1] } } },
  {
    "$project": {
      _id: 0,
      member_id: 1,
      name: 1,
      tier: "$membership.type",
      unpaid_bills: { "$size": "$unpaid" },
      amount_due: { "$sum": "$unpaid.amount" }
    }
  },
  { "$sort": { member_id: 1 } }
]).toArray());

// Q12  Subquery (EXISTS): Classes taught by trainers who coach more than 8 members
// SQL equivalent:
//   SELECT c.class_name, c.trainer_name FROM classes c
//   WHERE EXISTS (SELECT 1 FROM members m
//                 WHERE m.assigned_trainer_id = c.trainer_id
//                 GROUP BY m.assigned_trainer_id HAVING COUNT(*) > 8);
print("\n--- Q12 Classes taught by trainers who coach more than 8 members");
printjson(db.classes.aggregate([
  {
    "$lookup": { from: "members", localField: "trainer_id", foreignField: "assigned_trainer_id", as: "m" }
  },
  { "$match": { "$expr": { "$gt": [{ "$size": "$m" }, 8] } } },
  { "$project": { _id: 0, class_name: 1, trainer_name: 1, trainer_members: { "$size": "$m" } } },
  { "$sort": { class_name: 1 } }
]).toArray());

// Q13  Set operation (UNION): Members enrolled in Morning Yoga or Evening Yoga (each counted once)
// SQL equivalent:
//   SELECT member_name FROM enrollments WHERE class_name = 'Morning Yoga'
//   UNION
//   SELECT member_name FROM enrollments WHERE class_name = 'Evening Yoga';
print("\n--- Q13 Members enrolled in Morning Yoga or Evening Yoga (each counted once)");
printjson(db.classes.aggregate([
  {
    "$facet": {
      a: [
        { "$match": { class_name: "Morning Yoga" } },
        { "$project": { n: "$enrolled_members.member_name" } }
      ],
      b: [
        { "$match": { class_name: "Evening Yoga" } },
```

```
        { "$project": { n: "$enrolled_members.member_name" } }
      ]
    }
  },
  {
    "$project": {
      morning: { "$size": { "$arrayElemAt": ["$a.n", 0] } },
      evening: { "$size": { "$arrayElemAt": ["$b.n", 0] } },
      result: { "$setUnion": [{ "$arrayElemAt": ["$a.n", 0] }, { "$arrayElemAt": ["$b.n", 0] }] }
    }
  },
  {
    "$project": {
      morning_yoga: "$morning",
      evening_yoga: "$evening",
      union_size: { "$size": "$result" },
      result: 1
    }
  }
]).toArray());

// Q14  Set operation (INTERSECT): Members enrolled in both CrossFit Burn and Strength 101
// SQL equivalent:
//   SELECT member_name FROM enrollments WHERE class_name = 'CrossFit Burn'
//   INTERSECT
//   SELECT member_name FROM enrollments WHERE class_name = 'Strength 101';
print("\n--- Q14 Members enrolled in both CrossFit Burn and Strength 101");
printjson(db.classes.aggregate([
  {
    "$facet": {
      a: [
        { "$match": { class_name: "CrossFit Burn" } },
        { "$project": { n: "$enrolled_members.member_name" } }
      ],
      b: [
        { "$match": { class_name: "Strength 101" } },
        { "$project": { n: "$enrolled_members.member_name" } }
      ]
    }
  },
  {
    "$project": {
      result: { "$setIntersection": [{ "$arrayElemAt": ["$a.n", 0] }, { "$arrayElemAt": ["$b.n", 0] }] }
    }
  },
  { "$project": { intersect_size: { "$size": "$result" }, result: 1 } }
]).toArray());

// Q15  Set operation (EXCEPT): Active members who are not enrolled in any class
// SQL equivalent:
//   SELECT name FROM members WHERE status = 'active'
//   EXCEPT
//   SELECT member_name FROM enrollments;
print("\n--- Q15 Active members who are not enrolled in any class");
printjson(db.members.aggregate([
  {
    "$facet": {
      active: [
        { "$match": { "membership.status": "active" } },
        { "$group": { _id: null, n: { "$addToSet": "$name" } } }
      ],
      enrolled: [
        {
          "$lookup": {
```

```
            from: "classes",
            localField: "_id",
            foreignField: "enrolled_members.member_id",
            as: "c"
          }
        },
        { "$match": { "$expr": { "$gt": [{ "$size": "$c" }, 0] } } },
        { "$group": { _id: null, n: { "$addToSet": "$name" } } }
      ]
    }
  },
  {
    "$project": {
      result: {
        "$setDifference": [{ "$arrayElemAt": ["$active.n", 0] }, { "$arrayElemAt": ["$enrolled.n", 0] }]
      },
      active_members: { "$size": { "$arrayElemAt": ["$active.n", 0] } },
      enrolled_somewhere: { "$size": { "$arrayElemAt": ["$enrolled.n", 0] } }
    }
  },
  {
    "$project": { active_members: 1, enrolled_somewhere: 1, except_size: { "$size": "$result" }, result: 1 }
  }
]).toArray());
```

D.2 Outputs of the Query Set on MongoDB 8.3.7 (query_output.txt)

```
Connected to MongoDB; database 'gym_db' recreated and seeded; reference date 2026-09-24

================================================================================
Q1  [Selection]  Active VIP members living in Chennai
================================================================================
member_id  name           membership.end_date
---------  -------------  -------------------
MEM1014    Divya Das      2026-12-31
MEM1043    Gautham Nair   2027-05-31
MEM1046    Nandini Menon  2027-02-28
(3 matching documents)

================================================================================
Q2  [Selection]  Trainers who teach Yoga or CrossFit with at least 6 years of experience
================================================================================
trainer_id  name             specialization              experience_years
----------  --------------  --------------------------  ----------------
TRN03       Deepa Krishnan  Yoga, Pilates               9
TRN08       Vivek Nair      CrossFit, Olympic Lifting   8
TRN07       Kavitha Reddy   Yoga, Mobility              7
TRN01       Priya Menon     Strength Training, CrossFit  6
(4 matching documents)

================================================================================
Q3  [Selection]  Unpaid bills already past their due date (oldest first)
================================================================================
payment_id  member_name     amount  status   due_date
----------  --------------  ------  -------  ----------
PAY7054     Divya Menon     1500    overdue  2026-08-05
PAY7023     Isha Joshi      1500    pending  2026-09-05
PAY7027     Gautham Bhat    2500    pending  2026-09-05
PAY7047     Divya Das       4000    pending  2026-09-05
PAY7061     Aishwarya Das   4000    pending  2026-09-05
PAY7073     Karthik Sharma  2500    pending  2026-09-05
PAY7085     Ananya Das      2500    pending  2026-09-05
PAY7089     Vikram Kumar    1500    pending  2026-09-05
(17 matching documents, first 8 shown)
```

```
================================================================================
Q4  [Join (inner)]  Active VIP members with their assigned trainer
================================================================================
member_id  name              trainer        certification
---------  ----------------  -------------  -------------
MEM1001    Harish Kumar      Priya Menon    ACE-CPT
MEM1012    Isha Das          Sanjay Pillai  ISSA-CPT
MEM1014    Divya Das         Rohit Sharma   NASM-CPT
MEM1018    Aishwarya Das     Kavitha Reddy  RYT-200
MEM1028    Lakshmi Krishnan  Rohit Sharma   NASM-CPT
MEM1033    Rahul Sharma      Vivek Nair     CF-L2
MEM1035    Dev Varma         Vivek Nair     CF-L2
MEM1038    Nandini Das       Farhan Ali     ACSM-EP
(8 matching documents)

================================================================================
Q5  [Join (three tables)]  Attendance history of MEM1001 with class name and session time
================================================================================
date        class_name    start_time  status   check_in_time
----------  ------------  ----------  -------  -------------
2026-09-23  Evening Yoga  19:00       present  18:52
2026-09-19  Evening Yoga  17:00       present  16:51
2026-09-16  Evening Yoga  19:00       present  18:50
2026-09-12  Evening Yoga  17:00       present  16:57
2026-09-09  Evening Yoga  19:00       present  19:00
(5 matching documents)

================================================================================
Q6  [Join (left outer)]  Every trainer with the number of classes they teach, including trainers with none
================================================================================
trainer_id  name            classes  class_names
----------  --------------  -------  --------------------------
TRN03       Deepa Krishnan  2        Morning Yoga, Pilates Core
TRN02       Rohit Sharma    1        HIIT Express
TRN04       Sanjay Pillai   1        Strength 101
TRN05       Meenakshi Iyer  1        Power Zumba
TRN06       Farhan Ali      1        Spin Cycle
TRN07       Kavitha Reddy   1        Evening Yoga
TRN08       Vivek Nair      1        CrossFit Burn
TRN01       Priya Menon     0
(8 matching documents)

================================================================================
Q7  [Grouping and aggregation]  Money collected by payment mode (COUNT, SUM, AVG)
================================================================================
payment_mode  payments  total   average
------------  --------  ------  -------
UPI           101       237500  2351
card          37        97000   2622
cash          15        37500   2500
(3 matching documents)

================================================================================
Q8  [Grouping with HAVING]  Cities with more than 10 members, split by tier
================================================================================
city       members  basic  premium  vip
---------  -------  -----  -------  ---
Chennai    16       6      6        4
Bengaluru  13       6      5        2
Hosur      12       4      2        6
(3 matching documents)

================================================================================
```

```
Q9  [Grouping and aggregation]  Attendance outcome by class category
================================================================================
category  marked  present  late  absent
--------  ------  -------  ----  ------
Yoga      160     103      8     49
Zumba     95      69       8     18
CrossFit  75      51       10    14
Strength  70      46       7     17
HIIT      68      46       8     14
Pilates   60      46       5     9
Cardio    52      36       4     12
(7 matching documents)

================================================================================
Q10  [Subquery (scalar)]  Members whose attendance rate is below the gym-wide average
================================================================================
name              marked  rate_pct  gym_average_pct
----------------  ------  --------  ---------------
Isha Nair         5       20        77.1
Keerthana Iyer    10      40        77.1
Meera Varma       24      45.8      77.1
Gautham Joshi     14      50        77.1
Isha Joshi        10      50        77.1
Karthik Krishnan  8       50        77.1
Siddharth Varma   24      50        77.1
Nandini Menon     10      60        77.1
(8 matching documents)

================================================================================
Q11  [Subquery (correlated)]  Members with more than one unpaid bill
================================================================================
member_id  name                tier     unpaid_bills  amount_due
---------  ------------------  -------  ------------  ----------
MEM1008    Isha Joshi          Basic    2             3000
MEM1009    Gautham Bhat        Premium  2             5000
MEM1014    Divya Das           VIP      2             8000
MEM1016    Divya Menon         Basic    2             3000
MEM1018    Aishwarya Das       VIP      2             8000
MEM1021    Karthik Sharma      Premium  2             5000
MEM1024    Ananya Das          Premium  2             5000
MEM1025    Vikram Kumar        Basic    2             3000
MEM1038    Nandini Das         VIP      2             8000
MEM1044    Kavya Kumar         Premium  2             5000
MEM1045    Siddharth Pillai    Basic    2             3000
MEM1046    Nandini Menon       VIP      2             8000
MEM1047    Varun Das           Basic    2             3000
MEM1049    Ashwin Krishnan     Premium  2             5000
MEM1051    Karthik Shetty      Premium  2             5000
MEM1052    Isha Nair           Premium  2             5000
MEM1054    Keerthana Krishnan  Premium  2             5000
(17 matching documents)

================================================================================
Q12  [Subquery (EXISTS)]  Classes taught by trainers who coach more than 8 members
================================================================================
class_name    trainer_name    trainer_members
------------  --------------  ---------------
Morning Yoga  Deepa Krishnan  12
Pilates Core  Deepa Krishnan  12
Spin Cycle    Farhan Ali      10
Strength 101  Sanjay Pillai   10
(4 matching documents)

================================================================================
```

```
Q13  [Set operation (UNION)]  Members enrolled in Morning Yoga or Evening Yoga (each counted once)
================================================================================
morning_yoga: 13  evening_yoga: 19  union_size: 27
result: Aditya Reddy, Dev Varma, Divya Das, Gautham Joshi, Gautham Subramanian, Harish Kumar,
    Harish Subramanian, Isha Das, Isha Joshi, Isha Nair, Karthik Sharma, Karthik Shetty, Keerthana Iyer,
    Keerthana Krishnan, Lakshmi Bhat, Lakshmi Krishnan, Manoj Shetty, Meera Varma, Nandini Das, Pranav Varma,
    Rahul Reddy, Shreya Iyer, Siddharth Pillai, Siddharth Sharma, Siddharth Varma, Sneha Kumar, Varun Das

================================================================================
Q14  [Set operation (INTERSECT)]  Members enrolled in both CrossFit Burn and Strength 101
================================================================================
intersect_size: 5
result: Dev Varma, Divya Menon, Meera Varma, Rahul Reddy, Siddharth Sharma

================================================================================
Q15  [Set operation (EXCEPT)]  Active members who are not enrolled in any class
================================================================================
active_members: 50  enrolled_somewhere: 47  except_size: 3
result: Ashwin Krishnan, Rahul Sharma, Swathi Varma
```

D.3 Output of the Index Benchmark (benchmark_output.txt)

```
MongoDB 8.3.7: building gym_bench with 200,000 attendance and 60,000 payment documents ...

query                        plan                     docs_examined  keys_examined  returned  median_ms
B1 member attendance history  COLLSCAN                       200,000              0       105      80.46
B1 member attendance history  ix_member_date                    105            105       105       2.1
B2 absentees of one class     COLLSCAN                       200,000              0       998      84.97
B2 absentees of one class     ix_class_status                    998            998       998       5.53
B3 overdue dues report        COLLSCAN                        60,000              0     2,965      39.95
B3 overdue dues report        px_unpaid_dues (partial)         2,965          2,966     2,965      13.38
B3 overdue dues report        full index                       2,965          2,967     2,965      12.78

Index size: px_unpaid_dues 60.0 KB vs full index 604.0 KB
```

Appendix E: Full-Size ER Diagram and Relational Schema

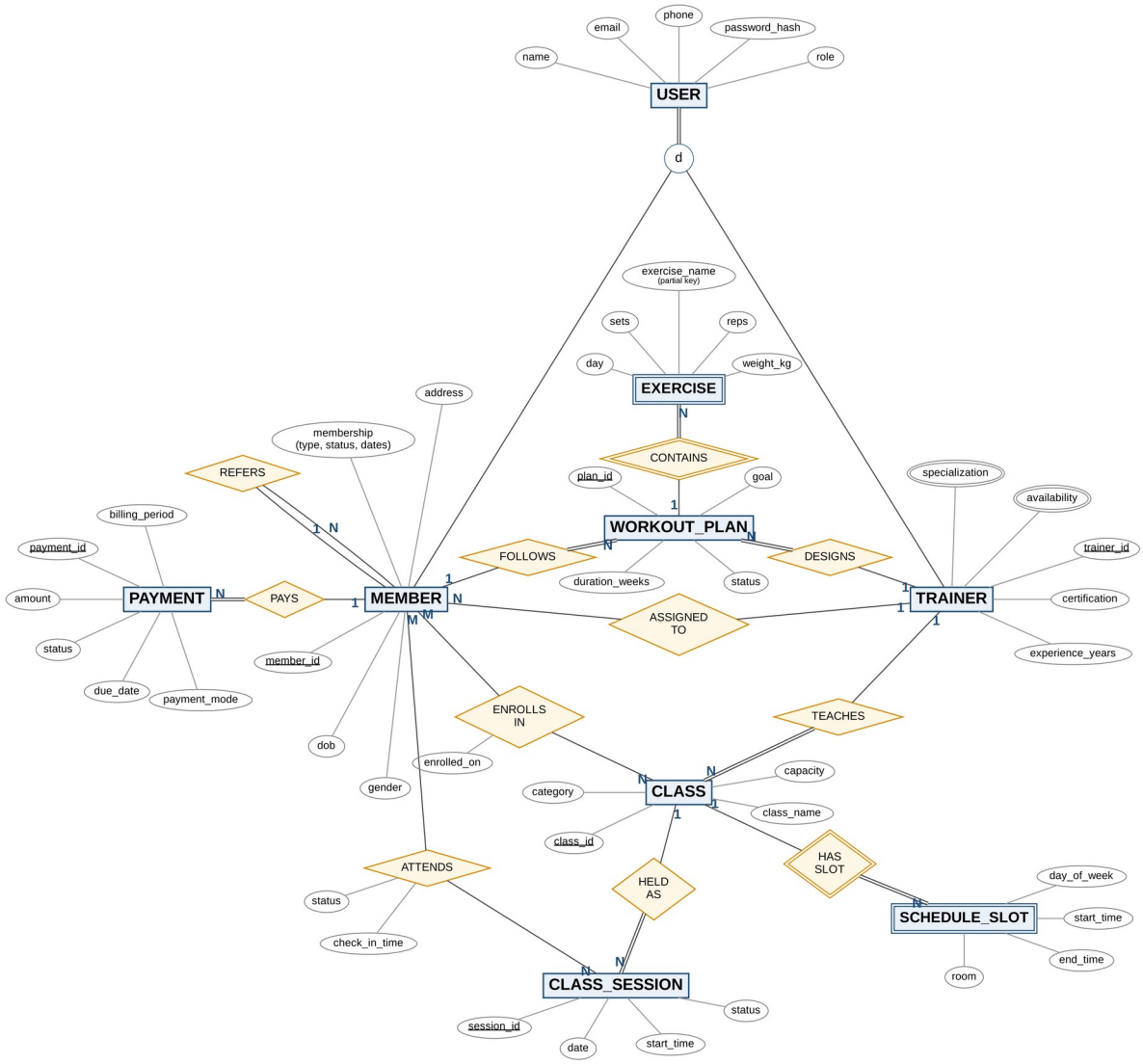


Figure E.1: Complete EER diagram with all attributes

The complete diagram is embedded at high resolution; zoom in on the PDF to read individual attributes.

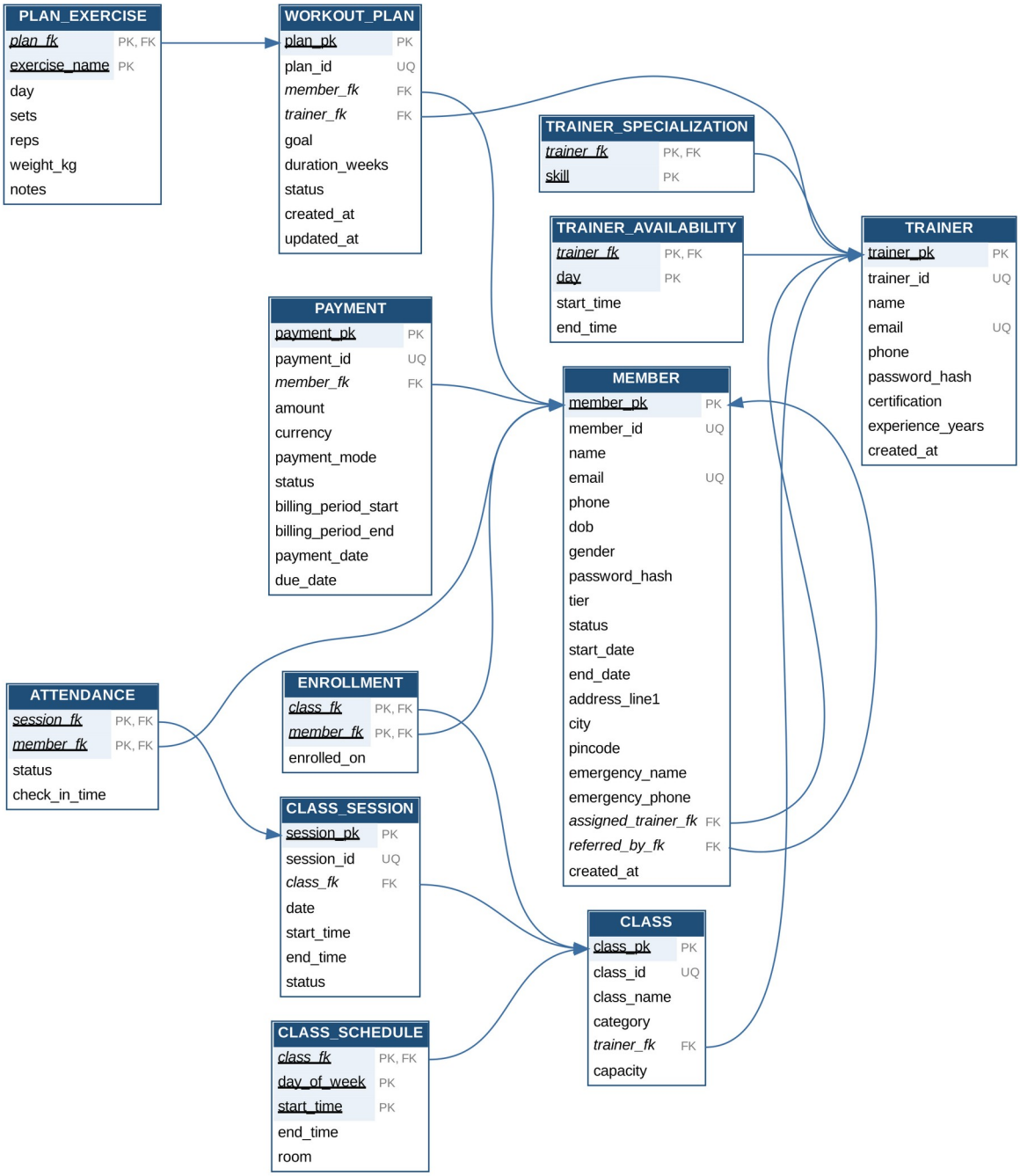


Figure E.2: Full-size relational schema (3NF)



Figure E.3: Full-size MongoDB document model (rotated to fit the page)

Appendix F: Application Screenshots

The application is used through the terminal programs `demo.py`, `queries.py` and `benchmark.py`. Figures F.1 to F.9 are screenshots of the live run on macOS with MongoDB 8.3.11. Figures F.10 to F.13 show the output of the query set, benchmark and triggers from the live run on MongoDB 8.3.7.

```
--- Relationship check: one member resolved across all referenced collections
MEM1001 Harish Kumar (VIP) -> trainer TRN01 Priya Menon
  workout plans: 1 | classes: 1 | attendance rows: 5 | payments: 4

--- Validator in action: inserting a member with an invalid tier
REJECTED by validator: 121 Document failed validation
```

Figure F.1: Validator rejecting a member with an invalid tier

```
============================================================================
1. DATABASE IMPLEMENTATION: collections, validators, sample data, indexes
============================================================================
Collections created with $jsonSchema validators:
  members, trainers, workout_plans, classes, class_sessions, attendance, payments

--- Sample data loaded (document counts)
collection      documents
--------------  ---------
trainers        8
members         60
workout_plans   76
classes         8
class_sessions  64
attendance      580
payments        220
TOTAL           1016
```

Figure F.2: Collections created and 1,016 sample documents loaded

```
================================================================================
2. CRUD OPERATIONS
================================================================================

--- CREATE: new trainer, new member (referred by MEM1003), workout plan, class, session, invoice
trainer TRN09 | member MEM1061 | plan WP5077 | class CLS09 | session SES9065 | invoice PAY7221

--- CREATE rejected: duplicate email (unique index uq_email)
DuplicateKeyError: email already registered

--- CREATE rejected: member assigned to a trainer that does not exist (app-level integrity)
IntegrityError: Trainer TRN77 not found

--- READ: member profile MEM1061 (password_hash excluded by projection)
{'_id': ObjectId('6ab528fd6d2803a432e142d5'),
 'member_id': 'MEM1061',
 'name': 'Rhea Thomas',
 'email': 'rhea.thomas@example.com',
 'phone': '9876501234',
 'dob': datetime.datetime(2004, 3, 15, 0, 0),
 'gender': 'Female',
 'role': 'member',
 'membership': {'type': 'Premium',
                'status': 'active',
                'start_date': datetime.datetime(2026, 9, 1, 0, 0),
                'end_date': datetime.datetime(2027, 9, 1, 0, 0)},
 'assigned_trainer_id': ObjectId('6ab528fd6d2803a432e142d4'),
 'referred_by': ObjectId('6ab528f96d2803a432e13ee5'),
 'address': {'line1': '', 'city': 'Vellore', 'pincode': '632014'},
 'emergency_contact': {'name': '', 'phone': ''},
 'created_at': datetime.datetime(2026, 9, 24, 0, 0)}

--- READ: VIP members, page 1 (filter + sort + skip/limit pagination)
11 active VIP members; first 5:
member_id  name             city
---------  ---------------  ---------
MEM1001    Harish Kumar     Bengaluru
MEM1012    Isha Das         Vellore
MEM1014    Divya Das        Chennai
MEM1018    Aishwarya Das    Hosur
MEM1028    Lakshmi Krishnan  Bengaluru

--- READ: trainers who teach Yoga (query inside embedded array)
trainer_id  name
----------  --------------
TRN03       Deepa Krishnan
TRN07       Kavitha Reddy

--- READ: MEM1001 last 5 attendance records (uses index ix_member_date)
date                status   check_in_time
------------------  -------  -------------
2026-09-23 00:00:00  present  18:52
2026-09-19 00:00:00  present  16:51
2026-09-16 00:00:00  present  18:50
2026-09-12 00:00:00  present  16:57
2026-09-09 00:00:00  present  19:00
```

Figure F.3: CRUD demonstration, create and read operations

```
--- READ: MEM1001 last 5 attendance records (uses index ix_member_date)
date                status   check_in_time
------------------  -------  -------------
2026-09-23 00:00:00  present  18:52
2026-09-19 00:00:00  present  16:51
2026-09-16 00:00:00  present  18:50
2026-09-12 00:00:00  present  16:57
2026-09-09 00:00:00  present  19:00

--- UPDATE: enroll into a class of capacity 2 (atomic capacity guard)
3rd enrollment blocked -> Cannot enroll MEM1001 in CLS09: class full or already enrolled
roster: ['Rhea Thomas', 'Swathi Sharma']

--- UPDATE: embedded arrays ($push, positional $inc, $addToSet, $pull)
day  exercise_name  sets  reps  weight_kg
---  -------------  ----  ----  ---------
Mon  Squat          4     10    35
Wed  Bench Press    4     10    25
Fri  Deadlift       3     8     40
specialization: ['Kickboxing', 'HIIT', 'Boxing']

--- UPDATE: tier upgrade Premium -> VIP, and name change propagated to denormalized copies
members: Rhea T. Thomas | tier: VIP
workout_plans.member_name: Rhea T. Thomas
payments.member_name: Rhea T. Thomas
classes.enrolled_members.member_name: ['Rhea T. Thomas']

--- UPDATE: attendance upsert (mark late, then correct to present; still one row)
rows for (SES9065, MEM1061): 1 | status: present

--- UPDATE: payments (record payment, overdue sweep)
record_payment: {'payment_id': 'PAY7221', 'amount': 2500, 'payment_mode': 'UPI', 'status': 'paid'}
bills newly flagged overdue by sweep: 16

--- DELETE: remove exercise, withdraw from class, delete attendance, delete member with cascade
exercises now: ['Squat', 'Bench Press']
withdraw: 1 | delete attendance: 1
delete MEM1016 blocked -> MEM1016 has 2 unpaid bill(s); settle before deleting
delete MEM1061 -> {'plans_deleted': 1, 'classes_updated': 0}
member still exists? False | orphan plans left: 0
delete TRN01 blocked -> TRN01 still has 6 members and 0 classes; reassign first

--- AUTH (FR8): bcrypt login check
correct password: {'role': 'trainer', 'id': 'TRN01', 'name': 'Priya Menon'}
wrong password:   None
```

Figure F.4: CRUD demonstration, update, delete and authentication

```
======================================================================
3a. AGGREGATION PIPELINES (FR7)
======================================================================

--- A1 Monthly revenue trend by tier (paid only)
2026-07  INR  147,000  (60 payments)  [Basic 36,000, Premium 55,000, VIP 56,000]
2026-08  INR  145,500  (59 payments)  [Basic 34,500, Premium 55,000, VIP 56,000]
2026-09  INR   79,500  (34 payments)  [Basic 24,000, Premium 27,500, VIP 28,000]

--- A2 Class popularity
class_name     trainer         enrolled  capacity  fill_rate_pct  sessions_held  avg_turnout  attendance_rate_pct
-------------  --------------  --------  --------  -------------  -------------  -----------  -------------------
Power Zumba    Meenakshi Iyer  19        25        76.0           5              15.4         81.1
HIIT Express   Rohit Sharma    17        18        94.4           4              13.5         79.4
Evening Yoga   Kavitha Reddy   19        20        95.0           5              12.8         67.4
CrossFit Burn  Vivek Nair      15        15        100.0          5              12.2         81.3
Strength 101   Sanjay Pillai   14        14        100.0          5              10.6         75.7
Pilates Core   Deepa Krishnan  12        12        100.0          5              10.2         85.0
Spin Cycle     Farhan Ali      13        16        81.2           4              10.0         76.9
Morning Yoga   Deepa Krishnan  13        20        65.0           5              9.4          72.3

--- A3 Members at risk (attendance < 60%)
member_id  name              tier     sessions_marked  attended  rate_pct
---------  ----------------  -------  ---------------  --------  --------
MEM1052    Isha Nair         Premium  5                1         20.0
MEM1020    Keerthana Iyer    Premium  10               4         40.0
MEM1030    Meera Varma       Basic    24               11        45.8
MEM1008    Isha Joshi        Basic    10               5         50.0
MEM1019    Siddharth Varma   Basic    24               12        50.0
MEM1053    Karthik Krishnan  Basic    8                4         50.0
MEM1039    Gautham Joshi     Premium  14               7         50.0

--- A4 Outstanding dues (collections worklist)
member_id  name                tier     bills  amount_due  max_days_overdue  priority
---------  ------------------  -------  -----  ----------  ----------------  --------
MEM1016    Divya Menon         Basic    2      3000        50.0              HIGH
MEM1046    Nandini Menon       VIP      2      8000        19.0              HIGH
MEM1038    Nandini Das         VIP      2      8000        19.0              HIGH
MEM1018    Aishwarya Das       VIP      2      8000        19.0              HIGH
MEM1014    Divya Das           VIP      2      8000        19.0              HIGH
MEM1021    Karthik Sharma      Premium  2      5000        19.0              HIGH
MEM1054    Keerthana Krishnan  Premium  2      5000        19.0              HIGH
MEM1044    Kavya Kumar         Premium  2      5000        19.0              HIGH
MEM1052    Isha Nair           Premium  2      5000        19.0              HIGH
MEM1009    Gautham Bhat        Premium  2      5000        19.0              HIGH
MEM1024    Ananya Das          Premium  2      5000        19.0              HIGH
MEM1049    Ashwin Krishnan     Premium  2      5000        19.0              HIGH

--- A5 Trainer workload (3 x $lookup)
trainer_id  name            members  active_plans  classes
----------  --------------  -------  ------------  -------
TRN03       Deepa Krishnan  12       12            2
TRN04       Sanjay Pillai   10       10            1
TRN06       Farhan Ali      10       10            1
TRN08       Vivek Nair      8        8             1
TRN07       Kavitha Reddy   7        7             1
TRN01       Priya Menon     6        6             0
TRN02       Rohit Sharma    5        5             1
TRN05       Meenakshi Iyer  2        2             1
TRN09       Nisha Varghese  0        0             1
```

Figure F.5: Aggregation reports A1 to A5

```
--- A6 Membership overview ($facet + $bucket in one query)
{'by_tier': [{'_id': 'Basic', 'count': 24},
             {'_id': 'Premium', 'count': 22},
             {'_id': 'VIP', 'count': 14}],
 'by_status': [{'_id': 'active', 'count': 50},
               {'_id': 'expired', 'count': 6},
               {'_id': 'frozen', 'count': 4}],
 'age_groups': [{'_id': 18, 'count': 12},
                {'_id': 25, 'count': 15},
                {'_id': 35, 'count': 22},
                {'_id': 45, 'count': 11}],
 'by_city': [{'_id': 'Chennai', 'count': 16},
             {'_id': 'Bengaluru', 'count': 13},
             {'_id': 'Hosur', 'count': 12},
             {'_id': 'Katpadi', 'count': 10},
             {'_id': 'Vellore', 'count': 9}]}

--- A7 Most prescribed exercises ($unwind embedded array)
exercise             plans  avg_sets  avg_weight_kg
-------------------  -----  --------  -------------
Plank                23     4.0       0.0
Rowing Machine       20     3.7       0.0
Squat                19     4.2       67.6
Bench Press          17     4.0       55.9
Deadlift             17     3.9       74.4
Pull Ups             11     4.0       0.0
Treadmill Intervals  11     4.0       0.0
Barbell Row          10     3.6       39.5

--- A8 Peak check-in hours
hour   check_ins
-----  ---------
18:00  115
06:00  98
17:00  82
16:00  51
09:00  39
07:00  24

--- V1 member_dashboard view
{'member_id': 'MEM1005',
 'name': 'Rahul Rao',
 'tier': 'Premium',
 'status': 'expired',
 'trainer': 'Vivek Nair',
 'unpaid_amount': 0}
```

Figure F.6: Aggregation reports A6 to A8 and the member_dashboard view

```
================================================================================
3b. GRAPH TRAVERSAL: referral network with $graphLookup
================================================================================

--- G2 Top referrers (size of downstream network)
member_id  name            direct  network_size  levels
---------  --------------  ------  ------------  ------
MEM1036    Lakshmi Bhat    1       5             3
MEM1016    Divya Menon     2       4             2
MEM1030    Meera Varma     1       4             3
MEM1020    Keerthana Iyer  1       3             3
MEM1051    Karthik Shetty  2       3             2

--- G1 Full referral tree of MEM1036
  L1 MEM1041 Arjun Rao
    L2 MEM1046 Nandini Menon
    L2 MEM1047 Varun Das
      L3 MEM1049 Ashwin Krishnan
      L3 MEM1050 Aishwarya Menon
```

Figure F.7: Referral network with $graphLookup

```
================================================================================
3c. INDEXING
================================================================================

--- Indexes on attendance and payments
  attendance._id_: [('_id', 1)]
  attendance.ix_member_date: [('member_id', 1), ('date', -1)]
  attendance.uq_session_member: [('session_id', 1), ('member_id', 1)] (unique)
  attendance.ix_class_status: [('class_id', 1), ('status', 1)]
  payments._id_: [('_id', 1)]
  payments.ix_member_status: [('member_id', 1), ('status', 1)]
  payments.ix_due_date: [('due_date', 1)]
  payments.px_unpaid_dues: [('status', 1), ('due_date', 1)] (partial)

--- explain(): member attendance history, collection scan vs index scan
plan           stage     docs_examined  keys_examined  returned  time_ms
-------------  --------  -------------  -------------  --------  -------
without_index  COLLSCAN  580            0              5         0
with_index     IXSCAN    5              5              5         0

--- Text index search: classes matching 'yoga'
class_name    category
------------  --------
Evening Yoga  Yoga
Morning Yoga  Yoga
```

Figure F.8: Indexes, explain() comparison and text search

```
========================================================================
3d. MULTI-DOCUMENT ACID TRANSACTIONS
========================================================================

--- T1 renew_membership with a simulated failure -> rollback
aborted: Simulated failure after payment insert
payments count 220 -> 220 (unchanged) | end_date still 2027-03-31

--- T1 renew_membership, 3 months -> commit
{'payment_id': 'PAY7221', 'amount': 4500, 'new_end_date': '2027-06-30'}
end_date 2027-03-31 -> 2027-06-30 | payments 220 -> 221

--- T2 close_session: attendance rows + session status in one transaction
SES9006 Morning Yoga 2026-09-24: {'session_id': 'SES9006', 'attendance_rows': 3}
session status now: completed

--- T2 with a member not on the roster -> whole transaction rolled back
aborted: MEM1001 is not enrolled in Pilates Core
attendance rows 583 -> 583 | SES9046 status: scheduled

Demo complete.
```

Figure F.9: Transactions T1 and T2, commit and rollback

```
(.venv) gym-membership-db % python queries.py --only Q11   (Q12 shown below)
================================================================================
Q11  [Subquery (correlated)]  Members with more than one unpaid bill
================================================================================
member_id  name                tier     unpaid_bills  amount_due
---------  ------------------  -------  ------------  ----------
MEM1008    Isha Joshi          Basic    2             3000
MEM1009    Gautham Bhat        Premium  2             5000
MEM1014    Divya Das           VIP      2             8000
MEM1016    Divya Menon         Basic    2             3000
MEM1018    Aishwarya Das       VIP      2             8000
MEM1021    Karthik Sharma      Premium  2             5000
MEM1024    Ananya Das          Premium  2             5000
MEM1025    Vikram Kumar        Basic    2             3000
MEM1038    Nandini Das         VIP      2             8000
MEM1044    Kavya Kumar         Premium  2             5000
MEM1045    Siddharth Pillai    Basic    2             3000
MEM1046    Nandini Menon       VIP      2             8000
MEM1047    Varun Das           Basic    2             3000
MEM1049    Ashwin Krishnan     Premium  2             5000
MEM1051    Karthik Shetty      Premium  2             5000
MEM1052    Isha Nair           Premium  2             5000
MEM1054    Keerthana Krishnan  Premium  2             5000
(17 matching documents)

================================================================================
Q12  [Subquery (EXISTS)]  Classes taught by trainers who coach more than 8 members
================================================================================
class_name    trainer_name    trainer_members
------------  --------------  ---------------
Morning Yoga  Deepa Krishnan  12
Pilates Core  Deepa Krishnan  12
Spin Cycle    Farhan Ali      10
Strength 101  Sanjay Pillai   10
(4 matching documents)
```

Figure F.10: Correlated and EXISTS subqueries Q11 and Q12

```
(.venv) gym-membership-db % python queries.py   (set operations Q13 to Q15)
================================================================================
Q13  [Set operation (UNION)]  Members enrolled in Morning Yoga or Evening Yoga (each counted once)
================================================================================
morning_yoga: 13  evening_yoga: 19  union_size: 27
result: Aditya Reddy, Dev Varma, Divya Das, Gautham Joshi, Gautham Subramanian, Harish Kumar,
        Harish Subramanian, Isha Das, Isha Joshi, Isha Nair, Karthik Sharma, Karthik Shetty,
        Keerthana Iyer, Keerthana Krishnan, Lakshmi Bhat, Lakshmi Krishnan, Manoj Shetty, Meera Varma,
        Nandini Das, Pranav Varma, Rahul Reddy, Shreya Iyer, Siddharth Pillai, Siddharth Sharma,
        Siddharth Varma, Sneha Kumar, Varun Das

================================================================================
Q14  [Set operation (INTERSECT)]  Members enrolled in both CrossFit Burn and Strength 101
================================================================================
intersect_size: 5
result: Dev Varma, Divya Menon, Meera Varma, Rahul Reddy, Siddharth Sharma

================================================================================
Q15  [Set operation (EXCEPT)]  Active members who are not enrolled in any class
================================================================================
active_members: 50  enrolled_somewhere: 47  except_size: 3
result: Ashwin Krishnan, Rahul Sharma, Swathi Varma
```

Figure F.11: Set operations Q13 to Q15

```
(.venv) gym-membership-db % python benchmark.py
MongoDB 8.3.7: building gym_bench with 200,000 attendance and 60,000 payment documents ...

query                       plan                      docs_examined  keys_examined  returned  median_ms
B1 member attendance history  COLLSCAN                      200,000              0       105      80.46
B1 member attendance history  ix_member_date                   105            105       105       2.1
B2 absentees of one class     COLLSCAN                      200,000              0       998      84.97
B2 absentees of one class     ix_class_status                   998            998       998       5.53
B3 overdue dues report        COLLSCAN                       60,000              0     2,965      39.95
B3 overdue dues report        px_unpaid_dues (partial)        2,965          2,966     2,965      13.38
B3 overdue dues report        full index                      2,965          2,967     2,965      12.78

Index size: px_unpaid_dues 60.0 KB vs full index 604.0 KB
```

Figure F.12: Index benchmark at 200,000 documents

```
(.venv) gym-membership-db % python demo.py --section trig
================================================================================
3e. TRIGGERS: change streams on members and payments
================================================================================

--- TR2: MEM1004 was expired; after paying PAY7223 the trigger set status to active

--- TR1: audit_log written by the trigger
collection  operation           key      changed_fields
----------  ------------------  -------  --------------------------------
payments    insert              PAY7223
payments    update              PAY7223  payment_date, payment_mode, status
members     trigger:reactivate  MEM1004  membership.status
members     update              MEM1002  phone
members     update              MEM1004  membership.status

Demo complete.
```

Figure F.13: Change-stream triggers TR1 and TR2

Appendix G: User Manual

G.1 Installation

1. Install Python 3.10 or later and Git.
2. Install MongoDB Community Server 8 and mongosh. On macOS: `brew tap mongodb/brew`, `brew trust mongodb/brew`, `brew install mongodb-community`. On Windows: download the MSI installer from mongodb.com and the separate mongosh installer.
3. Enable the replica set (needed for transactions and triggers). Add these lines to `mongod.conf` (macOS: `/opt/homebrew/etc/mongod.conf`; Windows: `C:\Program Files\MongoDB\Server\8.0\bin\mongod.cfg`):

```
replication:
  replSetName: rs0
```

4. Restart MongoDB (`brew services restart mongodb-community` on macOS, `net stop MongoDB` then `net start MongoDB` as administrator on Windows) and run `mongosh --eval "rs.initiate()"` once. `rs.status().ok` must print 1.
5. Clone and set up the project:

```
git clone https://github.com/malikabalaji/gym-membership-db.git
cd gym-membership-db
python3 -m venv .venv
source .venv/bin/activate          (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
```

G.2 Running the System

Table G.1: Commands

Command	What it does
`python demo.py`	Recreates gym_db, loads the sample data and runs every section
`python demo.py --section setup`	Collections, validators, data, indexes only
`python demo.py --section crud`	Create, read, update and delete demonstration
`python demo.py --section agg`	Reports A1 to A8 and the member_dashboard view
`python demo.py --section graph`	Referral network queries
`python demo.py --section index`	Index list, explain comparison, text search
`python demo.py --section txn`	Transactions with commit and rollback
`python demo.py --section trig`	Change-stream triggers
`python queries.py`	Runs the 15 representative queries
`python queries.py --only Q11`	Runs one query
`python benchmark.py`	Index benchmark at 200,000 documents (about 10 seconds)
`python -m pytest -q`	Runs the automated tests (no server needed)
`python demo.py --mock`	Runs the demonstration without a server (in memory)
`mongosh "mongodb://localhost:27017/?directConnection=true" mongosh/queries.js`	Runs the query set in the shell

G.3 Everyday Tasks

The functions below are what the planned web screens call. They can be used today from a Python shell:

```
from datetime import datetime
from gym_db.config import get_db
from gym_db import crud as C, aggregations as A, transactions as T
db = get_db()

# register a member
C.create_member(db, "Asha Rao", "asha.rao@example.com", "9876500000", datetime(2002, 5, 1),
                "Female", "Secret@123", tier="Premium", trainer_id="TRN03")
C.enroll_member(db, "CLS01", "MEM1061")                          # book a class
C.mark_attendance(db, "SES9007", "MEM1061", "present", "07:02")  # roll call
T.renew_membership(db, "MEM1061", months=3, mode="UPI")          # renew and pay
C.flag_overdue(db)                                               # nightly sweep
A.outstanding_dues(db)                                           # dues worklist
A.member_dashboard(db, "MEM1061")                                # member screen
```

Test logins: trainers use their email and `Trainer@NN` (for example priya.menon@gym.com and Trainer@01); members use `Member@` followed by their number (for example Member@1001).

G.4 Backup and Restore

```
mongodump --db gym_db --out backup/                # full backup
mongorestore --drop backup/                        # restore
python export_data.py                              # JSON export to data/*.json
mongoimport --db gym_db --collection members --jsonArray --file data/members.json
```

G.5 Troubleshooting

Table G.2: Common problems

Message	Fix
ServerSelectionTimeoutError	MongoDB is not running: start the service
"This server is a standalone mongod" or "Transaction numbers are only allowed on a replica set member"	Replica set not enabled: add `replSetName: rs0` and run `rs.initiate()`
ModuleNotFoundError	Virtual environment not active or dependencies missing: activate `.venv` and run `pip install -r requirements.txt`
externally-managed-environment (macOS)	Use a virtual environment as in G.1
Refusing to load formula from untrusted tap (Homebrew)	Run `brew trust mongodb/brew`, then install again
Document failed validation	The data breaks a rule in Table 4.13; correct the value